

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Владимирский государственный университет имени
Александра Григорьевича и Николая Григорьевича Столетовых»
(ВлГУ)

ВЫПУСКНАЯ
КВАЛИФИКАЦИОННАЯ РАБОТА

Студент Шутова Таисия Евгеньевна

Институт Информационных технологий и электроники

Направление 09.03.04 – Программная инженерия

Тема выпускной квалификационной работы

Информационная система арт галереи. Модуль рекомендаций

Руководитель _____

подпись инициалы, фамилия

Студент _____

подпись инициалы, фамилия

Допустить выпускную квалификационную работу
к защите в государственной экзаменационной комиссии

Заведующий кафедрой ИСПИ _____ И.Е. Жигалов

подпись инициалы, фамилия

«__» _____ 20 ____ г.

АННОТАЦИЯ

ВКР содержит 111 страниц, 27 рисунков, 20 таблиц, 2 приложения, 52 источника литературы.

Выпускная квалификационная работа посвящена разработке модуля персонализированных рекомендаций для системы «Арт-галерея» на Python. Модуль использует гибридный подход: коллаборативную фильтрацию (SVD) и контентную (TF-IDF). Для поиска визуально похожих произведений применяется модель OpenCLIP. Интеграция с Java-приложением на Spring MVC осуществляется через ProcessBuilder с обменом данными в JSON. Система формирует персонализированную ленту и блок визуально похожих работ.

ABSTRACT

In this diploma thesis is contained 111 pages, 27 illustrations, 20 tables, 2 appendices, 52 bibliography.

The final qualification work is devoted to the development of a personalized recommendation module for the Art Gallery system in Python. The module uses a hybrid approach: collaborative filtering (SVD) and content-based filtering (TF-IDF). The OpenCLIP model is used to search for visually similar works. Integration with a Java application using Spring MVC is performed through ProcessBuilder with JSON data exchange. The system generates a personalized feed and a block of visually similar works.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	7
ВВЕДЕНИЕ	10
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	11
1.1 Постановка проблемы. Исследовательские вопросы	11
1.2 Значимость и преимущества исследования.....	12
1.3 Цель и задачи	13
1.4 Существующий системный анализ и сравнение.....	14
1.4.1 Качественные выводы	16
1.4.2 SWOT-анализ разрабатываемой системы.....	16
1.5 Анализ требований.....	18
1.5.1 Функциональные требования.....	18
1.5.2 Нефункциональные требования	21
2 ПРОЕКТИРОВАНИЕ ПРОГРАММНО-ИНФОРМАЦИОННОЙ СИСТЕМЫ..	23
2.1 Обоснование архитектурных и технологических решений.....	23
2.1.1 Выбор архитектурного подхода	23
2.1.2 Выбор стандартов визуального моделирования	25
2.1.3 Обоснование выбора СУБД	25
2.1.4 Обоснование платформы реализации	26
2.1.5 Расчётное обоснование технических характеристик	27
2.2 Архитектурное проектирование	27
2.2.1 Диаграмма развёртывания.....	33

					<i>ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ</i>		
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подп.</i>	<i>Дата</i>			
<i>Разраб.</i>		Шутова Т.Е.			Информационная система арт галереи. Модуль рекомендаций Пояснительная записка	<i>Лит.</i>	<i>Лист</i>
<i>Пров.</i>		Озерова М.И.				У	4
<i>Н. контр.</i>		Андреанова В.И.					
<i>Утв.</i>		Жигалов И.Е.				ПРИ-122	
						<i>Листов</i>	113

2.3	Проектирование базы данных.....	36
2.4	Проектирование пользовательского интерфейса.....	39
2.5	Проектирование модуля рекомендаций.....	41
2.5.1	Архитектурная организация персонализированных рекомендаций.....	42
2.5.2	Архитектурная организация поиска похожих произведений.....	43
2.5.3	Обоснование алгоритмического выбора.....	44
2.5.4	Метрики и валидация.....	46
2.6	Проектирование системы безопасности и аудита	46
2.7	Диаграмма прецедентов.....	47
2.8	Диаграмма классов.....	49
2.9	Диаграмма последовательности	50
3	РЕАЛИЗАЦИЯ ПРОГРАММНО-ИНФОРМАЦИОННОЙ СИСТЕМЫ	53
3.1	Математическая модель персонализированных рекомендаций.....	53
3.1.1	Коллаборативная фильтрация (SVD)	53
3.1.2	Контентная фильтрация (TF-IDF и косинусная сходство)	55
3.1.3	Гибридная композиция и нормализация.....	56
3.2	Математическая модель поиска визуально похожих произведений	57
3.2.1	Извлечение признаков	57
3.2.2	Оценка визуального сходства	58
3.3	Реализация вычислительных модулей на Python.....	59
3.3.1	Скрипт обучения модели model_trainer.py	59
3.3.2	Скрипт модели recommendation_engine.py	60
3.3.3	Скрипт извлечения признаков openclip_extractor.py	62
3.4	Интеграция Python-модулей с Java-приложением.....	62
3.5	Оценка качества и тестирование	64
3.5.1	Метрики персонализированных рекомендаций.....	64
3.5.2	Валидация поиска похожих работ	66
3.5.3	Нагрузочное тестирование	66
3.6	Визуальное отображение модуля рекомендаций в веб-приложение.....	69
4	ИНФОРМАЦИОННЫЙ МЕНЕДЖМЕНТ	70

4.1 Обоснование проекта	71
4.2 Анализ окружения проекта	73
4.3 Основные положения устава проекта	75
4.4 Содержание проекта	76
4.5 Организационная структура проекта	80
4.6 Календарный план проекта	84
4.7 План управления рисками	87
4.8 План управления изменениями.....	93
4.9 Управленческая отчетность по проекту.....	96
ЗАКЛЮЧЕНИЕ	100
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ	103
ПРИЛОЖЕНИЕ А. ДИАГРАММА КОМПОНЕНТОВ.....	109
ПРИЛОЖЕНИЕ Б. СТРАНИЦЫ ВЕБ-ПРИЛОЖЕНИЯ	111

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Определения, обозначения, сокращения	Расшифровка
API (Application Programming Interface)	Интерфейс программирования приложений, набор правил и инструментов для взаимодействия между программными компонентами.
C4 model	Модель для визуализации архитектуры программного обеспечения, состоящая из четырёх уровней абстракции (Context, Containers, Components, Code).
CLIP (Contrastive Language-Image Pre-training)	Модель, обучаемая на парах "изображение-текст", способная находить связи между визуальным и текстовым описаниями.
CRUD (Create, Read, Update, Delete)	Базовые операции для управления данными: создание, чтение, обновление и удаление.
DTO (Data Transfer Object)	Объект передачи данных, используемый для инкапсуляции данных и их передачи между подсистемами.
ERD (Entity-Relationship Diagram)	Диаграмма «сущность-связь», используемая для моделирования данных и представления логической структуры базы данных.
ETL (Extract, Transform, Load)	Процесс извлечения данных из источников, их преобразования и загрузки в целевое хранилище.
ИСП (Иерархическая структура работ)	Документ, определяющий полный перечень работ и задач по проекту.

ML (Machine Learning)	Машинное обучение, область искусственного интеллекта, изучающая методы построения алгоритмов, способных обучаться на данных.
MVC (Model-View-Controller)	Архитектурный шаблон, разделяющий приложение на три компонента: модель (данные), представление (интерфейс) и контроллер (логика).
MVP (Minimum Viable Product)	Минимально жизнеспособный продукт, версия продукта, обладающая минимальными, но достаточными для использования функциями.
ORM (Object-Relational Mapping)	Технология объектно-реляционного отображения, связывающая реляционные базы данных с объектно-ориентированными языками программирования.
OpenCLIP	Открытая реализация моделей семейства CLIP, предназначенная для обучения и формирования персонализированных рекомендаций на больших данных.
pHash (Perceptual Hash)	Перцептивный хеш, метод создания «отпечатка» изображения, позволяющий находить визуально похожие копии.
REST (Representational State Transfer)	Архитектурный стиль взаимодействия компонентов распределённого приложения в сети.
S3 (Simple Storage Service)	Протокол для взаимодействия с объектными хранилищами, изначально разработанный Amazon Web Services.
SADT (Structured Analysis and Design Technique)	Методология структурного анализа и проектирования систем.

SQL (Structured Query Language)	Язык структурированных запросов, используемый для управления данными в реляционных базах данных.
SSD (Solid-State Drive)	Твердотельный накопитель, запоминающее устройство на основе микросхем flash-памяти.
SVD (Singular Value Decomposition)	Сингулярное разложение, метод факторизации матрицы, широко используемый в рекомендательных системах.
SWOT-анализ	Метод стратегического планирования, заключающийся в выявлении факторов внутренней и внешней среды организации.
TF-IDF (Term Frequency — Inverse Document Frequency)	Статистическая мера, используемая для оценки важности слова в контексте документа и коллекции.
UML (Unified Modeling Language)	Унифицированный язык графического описания для моделирования программных систем.
UI (User Interface)	Пользовательский интерфейс, совокупность средств, при помощи которых пользователь взаимодействует с системой.
URL (Uniform Resource Locator)	Единый указатель ресурса, стандартизированная строка, определяющая местоположение ресурса в сети.

ВВЕДЕНИЕ

В рамках настоящей выпускной квалификационной работы был выполнен полный цикл разработки информационной системы для цифровой арт-галереи. Работа охватывает анализ предметной области, проектирование архитектуры, реализацию серверной части системы, включая модуль интеллектуальных рекомендаций, а также аспекты информационного менеджмента. Разработка нацелена на создание надежной и масштабируемой платформы, которая служит цифровой основой для деятельности галереи.

Разработанный программный продукт предоставляет функционал для публикации и каталогизации произведений искусства (публикаций), управления пользовательскими профилями и коллекциями (выставками). Ключевой особенностью системы является реализованный модуль персонализированных рекомендаций, который использует гибридный алгоритм на основе методов машинного обучения: коллаборативная фильтрация с сингулярным разложением (SVD) и контентная фильтрация на основе TF-IDF признаков (автор, категории). Для улучшения качества рекомендаций также применяется анализ визуального содержания изображений с помощью модели OpenCLIP, позволяющей выделять цветовые характеристики, гистограммы и объекты. Модуль интегрирован в основное Java-приложение через вызов Python-скриптов, что обеспечивает гибкость и возможность переобучения модели без остановки системы.

В ходе работы особое внимание уделено проектированию и оптимизации именно серверных компонентов, обеспечивающих высокую производительность и безопасность при обработке данных. Результатом проекта является готовое к внедрению решение, которое может быть использовано арт-пространствами для расширения аудитории и эффективного представления художественных произведений в цифровой среде.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		10

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Постановка проблемы. Исследовательские вопросы

В условиях цифровой трансформации культурного сектора арт-галереи сталкиваются с необходимостью не просто переноса каталога произведений в онлайн-среду, но и создания комплексной платформы, способной обеспечить устойчивое взаимодействие с аудиторией. Существующие типовые решения, такие как системы управления контентом или универсальные фотогалереи, часто оказываются недостаточными, поскольку не учитывают специфику художественного контента и не решают задачу персонализированного представления произведений пользователям. Отсутствие интеллектуального механизма, адаптирующего выдачу контента под индивидуальные предпочтения, приводит к снижению вовлечённости.

В рамках данной работы решается задача создания модуля рекомендаций, который бы эффективно решал задачу сортировки контента по уместности в условиях ограниченных и зашумленных данных о взаимодействиях пользователей с произведениями искусства. Это порождает ряд взаимосвязанных исследовательских вопросов, касающихся как общей архитектуры, так и частных реализаций.

Во-первых, требуется определить оптимальную архитектуру серверной части, построенной на Java Spring MVC, обеспечивающую высокопроизводительный API для веб-интерфейса и поддерживающую интеграцию с отдельным скриптом на Python, реализующим логику рекомендаций.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		11

Во-вторых, необходим обоснованный выбор между использованием готовых библиотек машинного обучения и разработкой собственных алгоритмов с учётом таких критериев, как гибкость, точность, масштабируемость и ресурсоёмкость.

В-третьих, особое внимание требует вопрос выбора методов коллаборативной и контентной фильтрации, их совмещения в гибридную схему, а также способа извлечения визуальных признаков из изображений для улучшения рекомендаций в ситуации «холодного старта» [1, 2].

Дополнительно можно рассмотреть неперсонализированные рекомендации на основе схожести выбранной публикации за счёт общих категорий, авторов, цветовой гистограммы и общих объектов на изображении.

Наконец, важным аспектом остаётся методика обучения и переобучения модели, включая формирование тренировочных данных на основе событий пользовательской активности (лайки, комментарии), выбор архитектуры матричного разложения и определение объективных метрик для оценки качества рекомендаций (Precision@k, Recall@k, Coverage) [3, 4, 5].

Решение этих вопросов в совокупности направлено на создание не просто функционирующего приложения, а целостной, интеллектуальной системы, которая повышает ценность цифрового присутствия арт-галереи за счёт персонализации пользовательского опыта.

1.2 Значимость и преимущества исследования

Проведённое исследование и разработанный программный продукт обладают значимой практической и технической ценностью.

С практической точки зрения, работа вносит вклад в цифровизацию сферы культуры, предлагая готовое, ориентированное на специфику арт-пространств

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		12

решение, которое выходит за рамки базового каталога. Внедрение подобной системы позволяет галерее не только автоматизировать процессы управления контентом и пользователями, но и перейти к модели активного, персонализированного взаимодействия с аудиторией. Интеллектуальный модуль рекомендаций, основанный на методах машинного обучения, выступает ключевым конкурентным преимуществом, напрямую влияя на вовлеченность посетителей, увеличивая время их присутствия на платформе и способствуя более полному освоению коллекции.

С технической стороны, значимость работы заключается в комплексном решении задачи интеграции высоконагруженной серверной части на Java Spring MVC с гибридным алгоритмом машинного обучения, что демонстрирует практически значимый подход к построению современных расширяемых информационных систем. Разработанные архитектурные решения, обоснование выбора фреймворка для модуля рекомендаций и методика обучения для задачи ранжирования в конкретной предметной области представляют собой воспроизводимый опыт, который может быть адаптирован для других проектов, требующих внедрения подобной функциональности.

1.3 Цель и задачи

Целью данной выпускной квалификационной работы является улучшение качества пользовательского опыта за счёт проектирования персонализированных рекомендательных алгоритмов, основанных на методах машинного обучения и машинного зрения, в информационную систему с последующей её реализацией, развертыванием и оценкой эффективности.

Для достижения поставленной цели в работе последовательно решаются следующие задачи:

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		13

- 1) провести анализ существующих подходов к построению рекомендательных систем, выбрать методы (SVD, TF-IDF) [6, 7] и обосновать их применимость для предметной области;
- 2) разработать алгоритм гибридных рекомендаций, объединяющий коллаборативную и контентную фильтрацию, с возможностью использования визуальных признаков (OpenCLIP) [8, 9];
- 3) реализовать на Python модуль, включающий подготовку данных, обучение модели и формирование рекомендаций;
- 4) интегрировать Python-модуль в основное Java-приложение через вызов скриптов с передачей данных в формате JSON;
- 5) реализовать механизм рандомизированной выдачи рекомендаций (выбор 12 из ТОП-50 для рекомендаций, выбор 8 из топ-50 для схожих работ) для повышения разнообразия при каждом обновлении страницы;
- 6) обеспечить возможность ручного и автоматического переобучения модели и пересканирования изображений по запросу администратора;
- 7) оценить качество работы модели с использованием метрик Precision@5, Recall@5, Coverage [3, 4, 5].

1.4 Существующий системный анализ и сравнение

В рамках обоснования архитектурных и функциональных решений разрабатываемой системы был проведён анализ существующих популярных платформ для публикации и обмена визуальным контентом, которые потенциально могут использоваться художниками и арт-сообществами. Целью анализа являлось выявление типовых и недостающих функций, определение рыночных ниш и формирование чёткого ценностного предложения для нового

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		14

продукта. Анализ носил количественный и качественный характер, а его итоги обобщены в рамках SWOT-анализа [10, 11].

Для сравнения были выбраны платформы, условно разделённые на две категории: специализированные платформы для цифровых художников и дизайнеров (DeviantArt, ArtStation) и универсальный фотохостинг (Pinterest) [12, 13, 14]. Ключевые сравнительные критерии включали: возможность публикации статичных изображений, наличие премодерации, социальные функции (лайки, комментарии), инструменты категоризации и организации работ (альбомы, коллекции, проекты), монетизация, а также наличие и тип системы рекомендаций. Результаты сравнения приведены в таблице 1.1.

Таблица 1.1 – Сравнительный анализ платформ

Критерий / Платформа	Pinterest	DeviantArt	ArtStation	Разр. ИС
Фокус на искусстве	Визуальный агрегатор	Сообщество художников	Профессиональные цифровые художники	Арт-галерея, сообщество художников
Премодерация	Нет (жалобы)	Выборочная, жалобы	Нет (жалобы)	На основе нейронной сети
Лайки / Комментарии	Да (сохранения) / Ограниченно	Да / Да	Да / Да	Да / Да
Категории/Теги	Теги, доски	Категории, теги	Теги, проекты	Категории
Коллекции	Доски	Галереи, коллекции	Проекты	Выставки
Профиль как портфолио	Сборник досок	Очень развит	Развит	Развит
Монетизация/Продажи	Партнерские ссылки	Продажа принтов, комиссии	Продажа работ, найм	Партнерские ссылки
Тип рекомендаций	Популярное, похожие пины	Популярное, подписки	Популярное, теги	Гибридный (SVD + TF-IDF + визуальное сходство)

1.4.1 Качественные выводы

Универсальная платформа Pinterest предлагают широкий охват, но их алгоритмы и интерфейсы не адаптированы для структурированного представления художественных портфолио. Модерация носит реактивный характер, а функции организации работ вторичными.

Специализированные платформы (ArtStation, DeviantArt) предоставляют отличные инструменты для создания портфолио и профессионального нетворкинга. Однако их социальные функции часто ограничены комментариями и лайками, а механизмы курирования контента (создание тематических коллекций с гибким управлением правами) либо отсутствуют, либо реализованы фрагментарно (например, только для премиум-пользователей). Системы рекомендаций на этих платформах, как правило, основаны на подписках, тегах и популярности, а не на глубоком анализе индивидуальных предпочтений.

Ни одна из рассмотренных платформ не использует гибридный подход, объединяющий коллаборативную фильтрацию (анализ поведения) с контентной фильтрацией (авторы, категории, визуальные признаки), что позволяет добиться более высокой персонализации и адаптации к интересам конкретного пользователя.

1.4.2 SWOT-анализ разрабатываемой системы

На основе проведённого анализа составлен SWOT-анализ [10, 11] разрабатываемой системы, который подтверждает актуальность создания специализированного решения, занимающего нишу между социальными сетями и портфолио-хостингами.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		16

Сильные стороны (Strengths):

- интеграция управляемой галереи с премодерацией, социальных функций и интеллектуальной рекомендательной системы в едином продукте;
- уникальные возможности по созданию тематических выставок с различными моделями доступа (публичные, частные), что поддерживает как индивидуальных авторов, так и сообщества;
- обязательная премодерация всех публикаций формирует доверительную среду и высокий стандарт контента, отличаясь от реактивной модерации на крупных платформах;
- использование гибридного подхода (SVD + TF-IDF + визуальные признаки) для персонализации.

Слабые стороны (Weaknesses):

- отсутствие начальной пользовательской базы и узнаваемости по сравнению с гигантами рынка;
- необходимость дискового пространства для хранения большого объёма данных;
- ограниченность медиаформатов (только статичные изображения) на начальном этапе;
- зависимость качества рекомендаций от объёма накопленных взаимодействий.

Возможности (Opportunities):

- привлечение профессиональных художников, кураторов и малых арт-галерей, недовольных обезличенностью крупных платформ и нуждающихся в структурированной, контролируемой среде;
- возможность интеграции с художественными школами, вузами и офлайн-галереями для создания их цифровых филиалов;
- введение платных подписок для расширения возможностей (например, аналитика, продвижение выставок), комиссионной модели за продажу работ или интеграции с печатными сервисами.

Угрозы (Threats):

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		17

- доминирование крупных устоявшихся платформ с огромными бюджетами на развитие и маркетинг;
- риск того, что целевая аудитория продолжит использовать привычные, хотя и менее специализированные, платформы из-за уже сложившейся аудитории;
- сложность поддержания и периодического переобучения рекомендательной системы, необходимость защиты от манипуляций (накруток).

Данный анализ подтверждает актуальность разработки специализированной системы, занимающей четкую нишу между социальными сетями и портфолио-хостингами, делая ставку на контроль качества, кураторство и персонализацию.

1.5 Анализ требований

Анализ требований к разрабатываемой информационной системе для арт-галереи проведен с учетом ее функционального назначения, целевой аудитории и ключевого дифференцирующего модуля – системы рекомендаций. Особое внимание уделено реализуемости и проверке всех требований в условиях ограниченных ресурсов, характерных для студенческого проекта, что подразумевает использование бесплатных инструментов, локального развертывания и эмуляции нагрузки.

1.5.1 Функциональные требования

Функциональные требования определяют основные возможности системы, которые должны быть реализованы и продемонстрированы в ходе защиты ВКР.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		18

Требования к базовой серверной части (Java Spring MVC приложение):

- система должна предоставлять REST API для веб-клиента;
- должна быть реализована регистрация и аутентификация пользователей по логину/паролю; поддержка ролей: Посетитель, Художник, Администратор;
- для авторизованных пользователей должна быть предусмотрена возможность загрузки статичных изображений (JPEG, PNG, WEBP) в виде черновика публикации с обязательным заполнением названия, описания и выбора одной или нескольких категорий из фиксированного системного списка;
- публикации в статусе черновика должны быть видны только их автору и администраторам;
- пользователи с ролью Администратор должны иметь интерфейс (веб-или API) для просмотра очереди черновиков, их одобрения или отклонения с указанием причины (после одобрения публикация становится видимой для всех авторизованных пользователей);
- для опубликованных работ должны быть доступны функции: постановка/снятие лайка, добавление/удаление/редактирование текстового комментария;
- пользователи должны иметь возможность создавать именованные коллекции («Выставки»); для каждой выставки владелец должен иметь возможность настраивать политику добавления работ: Только владелец или Все авторизованные пользователи;
- должен быть реализован профиль пользователя, отображающий сетку его опубликованных работ и список созданных им выставок;
- административный интерфейс (может быть реализован как набор специфических API-эндпоинтов) должен позволять управлять списком категорий, публикаций, пользователями и просматривать системные журналы событий.

Требования к модулю рекомендаций:

- модуль должен быть реализован как набор скриптов на языке Python (версии 3.9 и выше), интегрированных в основное веб-приложение на Java Spring

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		19

MVC посредством вызова через ProcessBuilder [15] с передачей параметров командной строки;

- входными данными для формирования рекомендаций является идентификатор текущего авторизованного пользователя (целое число) или текущая выбранная работа;

- выходными данными должен быть ранжированный список идентификаторов публикаций (произведений искусства), которые пользователь ещё не просматривал и с которыми не взаимодействовал (не ставил лайки, не комментировал), – для рекомендаций, или которые похожи на текущую — для схожих работ;

- система должна поддерживать два режима работы: режим обучения (переобучения) модели и режим получения рекомендаций;

- обучение модели должно проводиться на основе истории взаимодействий пользователей с системой (лайки, просмотры) и метаданных работ (категории);

- модель должна быть реализована с использованием бесплатного фреймворка машинного обучения с открытым исходным кодом, совместимого с развертыванием в среде с ограниченными вычислительными ресурсами (без GPU);

- должна быть предусмотрена возможность периодического переобучения модели на новых данных в офлайн-режиме;

- алгоритм рекомендаций должен быть основан на гибридном подходе, объединяющем методы машинного обучения без использования глубоких нейронных сетей; в частности, требуется реализовать коллаборативную фильтрацию, контентную фильтрацию, гибридную оценку и визуальные признаки;

- модуль должен обеспечивать контроль качества рекомендаций путём расчёта метрик Precision@5, Recall@5 и Coverage [3, 4, 5] на тестовой выборке (25% от всех взаимодействий, выделяемых случайным образом); целевыми значениями для учебного проекта являются $\text{Precision@5} \geq 0,2$, $\text{Recall@5} \geq 0,5$ и

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		20

Coverage $\geq 0,15$ при условии, что в базе данных присутствует не менее 50 одобренных работ и 50 взаимодействий;

– время формирования персонализированных рекомендаций (включая запуск Python-скрипта, загрузку модели, вычисление предсказаний и возврат результата) на серверной машине с процессором Intel Core i5 или аналогичным не должно превышать 5 секунд для одного пользователя; время обучения модели (включая загрузку данных из БД, построение TF-IDF матрицы, обучение SVD и сохранение кэша) не должно превышать 60 секунд при базе данных до 1000 работ и 5000 взаимодействий.

1.5.2 Нефункциональные требования

Данные требования сформулированы с учетом необходимости их валидации в учебных условиях.

Производительность и масштабируемость:

– система должна стабильно работать на локальной машине или бесплатном облачном виртуальном сервере (например, с 1-2 ГБ ОЗУ);

– время отклика API на основные запросы (получение ленты, профиля) в условиях локального развертывания и тестовой базы данных (до 1000 записей) не должно превышать 5 секунд;

– модуль рекомендаций должен формировать ответ для одного пользователя не более чем за 5 секунд в условиях использования CPU.

Надежность и информационный менеджмент:

– должно быть обеспечено регулярное резервное копирование базы данных (достаточно скрипта для создания дампа SQL);

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		21

– критическая бизнес-логика (регистрация, модерация, создание публикаций) должна быть покрыта модульными и интеграционными тестами с использованием JUnit и in-memory базы данных (H2);

– система должна предусматривать фиксирование в журнале событий ключевых событий (ошибки, действия модератора) в файл.

Безопасность:

– все пароли должны храниться в базе данных исключительно в хэшированном виде (используя алгоритм bcrypt или scrypt);

– API-эндпоинты, кроме публичных (логин, регистрация, просмотр одобренных работ), должны требовать валидный JWT-токен;

– загружаемые файлы должны валидироваться на тип и размер (макс. 5-10 МБ для учебных целей); должна быть реализована защита от перебора паролей.

Каждое функциональное требование должно быть проверяемо через:

– ручное тестирование с помощью Swagger UI/Postman для API;

– написание автотестов для критических сценариев;

– демонстрацию работы на защите ВКР;

– работа модуля рекомендаций должна быть оценена количественно на синтетически сгенерированном или небольшом ручном датасете (имитация 50 пользователей и 50 взаимодействий) с расчетом базовых метрик (Precision@5, Recall@5) [3, 4, 5];

– все компоненты системы (основное приложение, база данных, сервис рекомендаций) должны быть запускаемы с помощью Tomcat 9 [16] на персональном компьютере, что гарантирует воспроизводимость результата.

Приведённый набор требований принят как минимально необходимый и достаточный для демонстрации работоспособности системы и достижения заявленной цели в рамках выпускной квалификационной работы. Акцент сделан на требования, которые могут быть объективно проверены без привлечения коммерческой инфраструктуры или больших объемов реальных данных.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		22

2 ПРОЕКТИРОВАНИЕ ПРОГРАММНО-ИНФОРМАЦИОННОЙ СИСТЕМЫ

2.1 Обоснование архитектурных и технологических решений

Ключевой этап проектирования – выбор архитектурного стиля, базовых технологий и стандартов моделирования, определяющих все последующие проектные работы. Все решения принимались с учётом функциональных и нефункциональных требований, сформулированных в разделе 1, а также реальных условий выполнения ВКР: разработка ведётся малой командой, развёртывание осуществляется на локальной инфраструктуре, демонстрация системы запланирована через веб-клиент.

2.1.1 Выбор архитектурного подхода

Рассматривались два основных варианта организации серверной части:

- монолитное приложение [17, 18] с модульной внутренней структурой, в котором весь бизнес-функционал, включая модуль рекомендаций, реализован в рамках единого развёртываемого артефакта;
- микросервисная архитектура [17, 18], предполагающая выделение системы рекомендаций в отдельный независимый сервис со своим технологическим стеком и базой данных.

Сравнение проводилось по критериям, важным для учебного проекта: простота разработки и отладки, требования к инфраструктуре, скорость интеграции, поддерживаемость силами одного-двух разработчиков. Оценка

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		23

выполнялась методом взвешенной суммы баллов (таблица 1.2). Каждый критерий оценивался по шкале от 1 (наихудшее значение) до 5 (наилучшее). Веса критериев определены экспертным путём с учётом ограничений ВКР.

Таблица 1.2 – Сравнение архитектурных стилей

Критерий	Вес	Монолит (оценка)	Микросервисы (оценка)
Простота разработки и конфигурирования	0,25	5	2
Требования к инфраструктуре (ОЗУ, CPU)	0,20	4	2
Удобство отладки и тестирования	0,20	5	3
Скорость реализации MVP	0,25	5	2
Гибкость замены ML-компонента в будущем	0,10	3	5
Взвешенная сумма		4,65	2,75

Монолитная архитектура с вызовом Python-скриптов через ProcessBuilder [15] показала значительное преимущество для текущих условий (взвешенная сумма 4,65 против 2,75). Она позволяет сосредоточить усилия на функциональной полноте, избежать накладных расходов на сетевое взаимодействие, сериализацию и развёртывание нескольких сервисов. Важным преимуществом является возможность использования всех библиотек экосистемы Python (pandas, scikit-learn, open_clip, torch) без необходимости создавать дополнительные HTTP-обёртки. Отладка упрощается, так как весь код находится в одном репозитории, а скрипты могут быть протестированы изолированно. Единственным недостатком является некоторая связанность с конкретным языком, но в рамках учебного проекта это не является критичным. При этом модульная структура Java-приложения (разделение на контроллеры, сервисы и репозитории) сохраняет возможность последующего выделения рекомендательного модуля в отдельный микросервис, если это потребуется при масштабировании.

На основании проведённого анализа принято решение строить систему как монолитное веб-приложение на базе Java Spring MVC с интеграцией Python-скриптов рекомендаций через стандартные средства выполнения внешних процессов.

2.1.2 Выбор стандартов визуального моделирования

Для документирования архитектуры выбрана комбинация нотаций:

- для верхних уровней (системный контекст, контейнеры) применяется подход C4, обеспечивающий наглядное представление системы для широкой аудитории без перегрузки деталями;
- для детального проектирования компонентов, структуры базы данных и поведения системы используется унифицированный язык моделирования UML (диаграммы классов, компонентов, развёртывания).

Такой подход соответствует рекомендациям ГОСТ Р 59795–2021 и современной практике документирования программных систем.

2.1.3 Обоснование выбора СУБД

Для хранения структурированных данных требуется реляционная база данных. Сравнивались MySQL 8.0 и PostgreSQL 14 [19, 20]. Обе СУБД являются открытыми, поддерживают транзакции, индексы и ограничения целостности. Решающим фактором стал опыт команды и наличие готовой инфраструктуры под MySQL в окружении разработки. Кроме того, MySQL демонстрирует более низкое потребление памяти при небольших объёмах данных, что важно при

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		25

развёртывании на локальном сервере или слабом VPS. Таким образом, в качестве СУБД выбрана MySQL 8.0.

2.1.4 Обоснование платформы реализации

Серверная часть реализуется на Java 17 с использованием фреймворка Spring, веб-сервера Tomcat 9, механизма внедрения зависимостей, безопасности (Spring Security) и Spring Data JPA/Hibernate. Выбор обусловлен требованиями к надёжности, богатой экосистемой и наличием компетенций у разработчиков. Для клиентской части используется HTML5, CSS3, взаимодействующая с сервером по REST API. Мобильное приложение для Android имеет функционирующий прототип, однако в рамках ВКР будет продемонстрирован только веб-клиент. Модуль машинного обучения написан на Python 3.11 с библиотеками scikit-learn [21, 22] (для TF-IDF и косинусной близости [23, 24]), Surprise (для метрик качества) [25, 26], pandas [27, 28] и numpy [29, 30] (для обработки данных), open_clip [31, 32] и torch [33, 34] (для извлечения визуальных признаков, таких как цвета и объекты), mysql-connector-python (для доступа к базе данных). Для реализации коллаборативной фильтрации планируется описать собственный алгоритм стохастического градиентного спуска, что необходимо для полного контроля над процессом, позволит настроить связи с существующими структурами данными по собственным предпочтениям и минимизировать внешние зависимости. Интеграция в Java-приложение осуществляется путём вызова Python-скрипта через ProcessBuilder; обмен данными происходит через JSON.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		26

2.1.5 Расчётное обоснование технических характеристик

Хотя проект является учебным и эксплуатируется в ограниченной среде, выполнена прикидка основных параметров. Предполагаемое количество активных пользователей на этапе опытной эксплуатации – до 100, среднее количество загружаемых изображений в день – 20. Средний размер одного изображения после оптимизации принят 2 МБ. Тогда суточный объём новых данных: $20 * 2 = 40$ МБ, годовой – около 15 ГБ. С учётом резервных копий и журналов активности, дисковой подсистемы объёмом 100 ГБ SSD достаточно на несколько лет. Серверное приложение обрабатывает преимущественно CRUD-запросы и периодический запуск ML-скрипта; при пиковой нагрузке до 50 одновременных запросов достаточно 2 ГБ оперативной памяти и 2 виртуальных ядер CPU. Эти характеристики закладываются в конфигурацию тестового стенда.

2.2 Архитектурное проектирование

Архитектура задокументирована в соответствии с выбранным подходом C4. Все диаграммы C4 выполнены с помощью PlantUML [35].

Диаграмма контекста (рисунок 2.1) отображает информационную систему арт-галереи как единое целое и её взаимодействие с внешними пользователями и системами. Выделены три типа пользователей:

- «Посетитель» – неавторизованный пользователь, имеющий доступ только к просмотру опубликованных произведений, выставок и категорий;
- «Художник» – авторизованный пользователь, который дополнительно может загружать произведения, создавать выставки, оставлять комментарии, ставить лайки и получать персонализированные рекомендации;

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		27

– «Администратор» – пользователь с правами модерации контента, управления категориями и администрирования системы, включая запуск переобучения рекомендательной модели.

Система взаимодействует с тремя внешними сущностями:

– MinIO (S3) – объектное хранилище для загрузки и хранения изображений произведений, а также для хранения вспомогательных файлов [36, 37] (например, NSFW-модели);

– ML Recommendation Engine – компонент, отвечающий за построение персонализированных рекомендаций и поиск визуально похожих произведений на основе признаков, извлечённых с помощью OpenCLIP; содержит два основных скрипта: `recommendation_engine.py` (обеспечивает получение персонализированных рекомендаций) и `model_trainer.py` (отвечает за переобучение модели); для извлечения визуальных признаков изображений (цветовых гистограмм и обнаружения объектов) используется библиотека OpenCLIP с предобученной моделью ViT-B-32, вызов которой осуществляется через вспомогательный скрипт `openclip_extractor`;

– Yandex Vision API – внешний облачный сервис, используемый для автоматической модерации загружаемых изображений (определение контента 18+).

Все связи между пользователями и системой, а также между системой и внешними сервисами осуществляются по протоколу HTTPS/REST, за исключением взаимодействия с базой данных (JDBC).

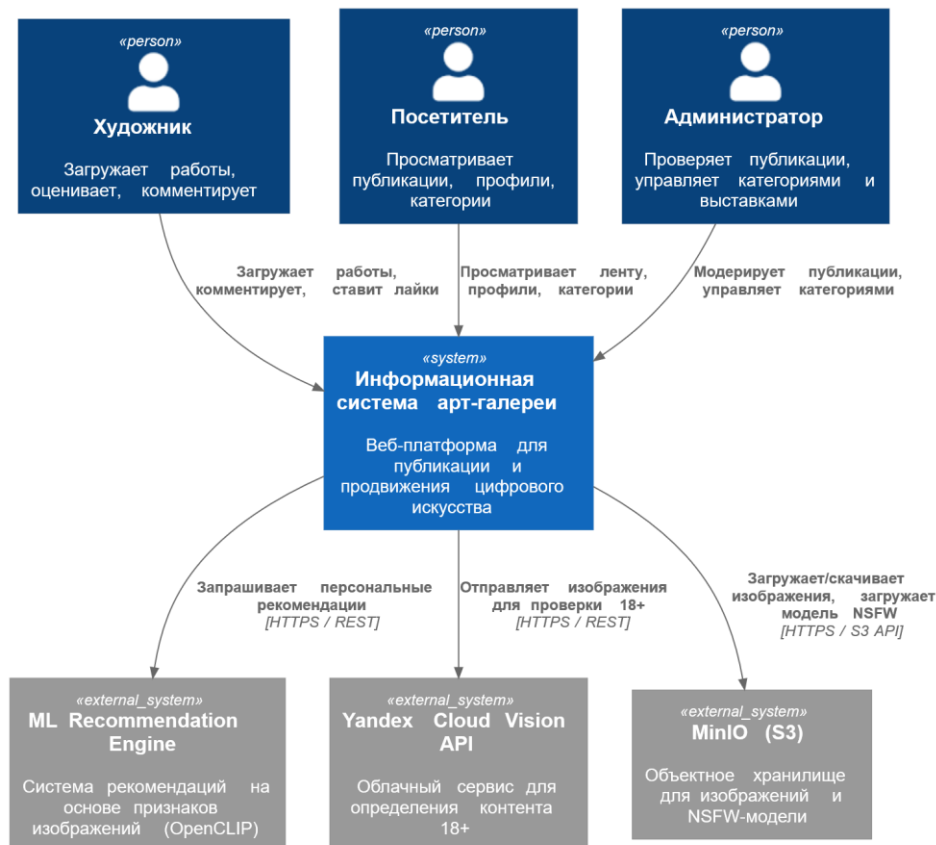


Рисунок 2.1 – Диаграмма контекста

Диаграмма контейнеров (рисунок 2.2) детализирует внутреннюю структуру системы, показывая основные исполняемые единицы (контейнеры) и их взаимосвязи. В границах информационной системы выделены следующие контейнеры:

- Web Browser – клиентская часть, реализованная на HTML5, CSS3, JavaScript с использованием шаблонизатора Thymeleaf; обеспечивает интерфейс для просмотра галереи, управления произведениями, выставками и получения рекомендаций; взаимодействует с серверным приложением через HTTP/HTTPS запросы;

- Spring MVC Web Application – основное серверное приложение, написанное на Java 11 с использованием фреймворка Spring 5.3.20; реализует REST API, бизнес-логику, обработку запросов, модуль модерации (с интеграцией Yandex Vision API), а также клиентскую логику для взаимодействия с ML-

сервисом и MinIO; приложение развёртывается в контейнере сервлетов Apache Tomcat 9;

- MySQL Database – реляционная база данных MySQL 8.0.33, хранящая все сущности: пользователей, роли, произведения, категории, выставки, комментарии, лайки, уведомления и хеши изображений (в том числе вычисленные признаки); доступ к БД осуществляется через JDBC;
- ML Recommendation Service – контейнер, реализующий подключение к рекомендательным скриптам на Python; предоставляет HTTP-эндпоинты, вызываемые основным Spring-приложением.

Внешними по отношению к системе сущностями остаются MinIO (объектное хранилище) и Yandex Vision API. Все внутренние контейнеры развёртываются на одном сервере или на связанных виртуальных машинах в рамках учебного стенда.

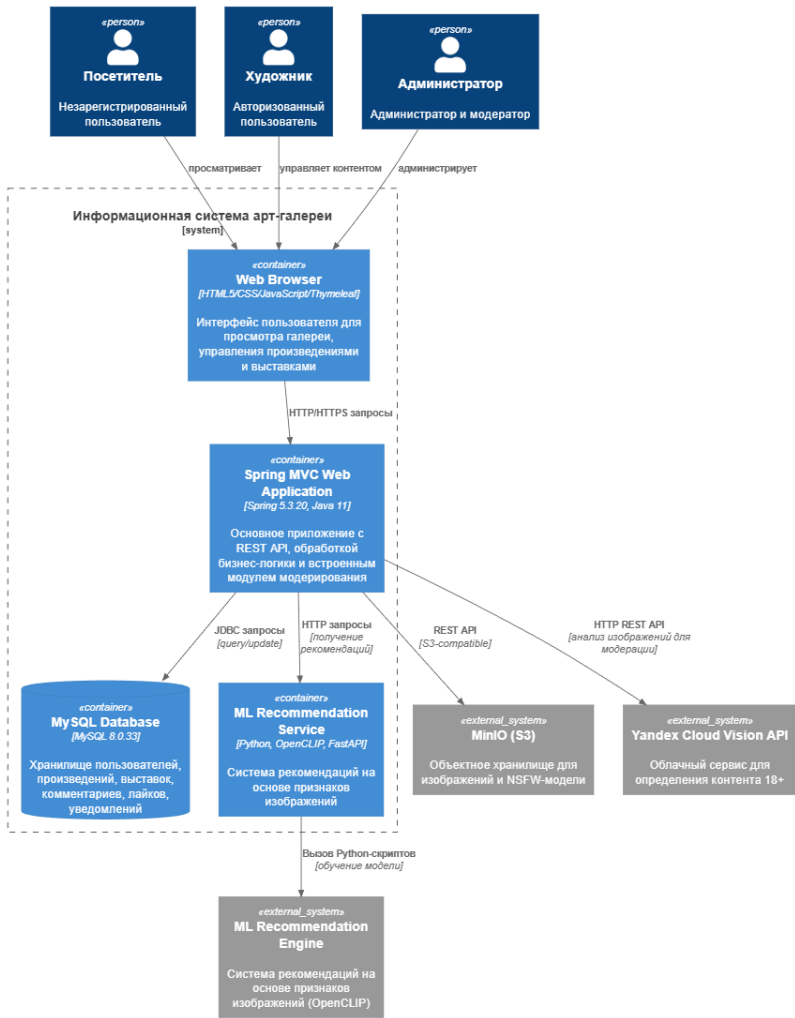


Рисунок 2.2 – Диаграмма контейнеров

Диаграмма компонентов (Приложение А) раскрывает внутреннее устройство основного контейнера – Spring MVC Web Application. Выделены следующие группы компонентов:

- контроллеры – обеспечивают обработку входящих HTTP-запросов и формирование ответов; к ним относятся: AuthController (аутентификация и регистрация), ArtworkController (CRUD-операции с произведениями), CategoryController (управление категориями), ExhibitionController (управление выставками), AdminController (административные функции), UploadController (загрузка файлов), UserController (управление профилем), HomeController (главная страница) и RecommendationController (API для получения персонализированных рекомендаций и похожих произведений). Все контроллеры используют соответствующие сервисы;

- сервисы – содержат бизнес-логику; ключевыми для модуля рекомендаций являются: RecommendationService – отвечает за вызов ML-сервиса, получение списка рекомендованных произведений и последующую рандомизацию (выбор 12 случайных из ТОП-50); OpenClipService – обеспечивает извлечение признаков изображений (цветовые гистограммы, векторы объектов) с помощью вызова внешнего ML-движка и сохраняет эти признаки в базу данных через ImageHashRepository; ImageFeatureService – обрабатывает хеши изображений и признаки для поиска дубликатов и визуально похожих произведений; ModerationService – координирует процесс модерации, включая вызов YandexVisionClient; также присутствуют сервисы для работы с пользователями, произведениями, категориями, выставками и уведомлениями;

- репозитории – обеспечивают абстракцию доступа к базе данных; для каждой сущности определён отдельный репозиторий: UserRepository, ArtworkRepository, CategoryRepository, ExhibitionRepository, CommentRepository, LikeRepository, NotificationRepository, ImageHashRepository; репозитории используют Spring Data JPA (Hibernate) и взаимодействуют с MySQL через JDBC;

– модели и DTO; в пакете моделей (User, Role, Artwork, Category, Exhibition, Comment, Like, Notification, ImageHash) описаны сущности, соответствующие таблицам базы данных; DTO (например, RecommendationDTO) используются для передачи данных между контроллерами и клиентом, а также между сервисами;

– интеграционные клиенты; MinioClient инкапсулирует взаимодействие с объектным хранилищем MinIO через S3-совместимый API; YandexVisionClient отвечает за вызов облачного сервиса модерации; MLClient (HTTP-клиент) обеспечивает вызов Python ML-сервиса для получения персонализированных рекомендаций и для извлечения признаков изображений (последнее может использоваться как синхронно при загрузке нового произведения, так и асинхронно для периодического обновления);

– служебные компоненты; CustomUserDetailsService реализует механизм аутентификации Spring Security; валидаторы и утилиты (Validation/Utility) обеспечивают проверку входных данных и вспомогательные функции.

Взаимодействие между компонентами при получении персонализированных рекомендаций происходит следующим образом: клиент (веб-браузер) отправляет GET-запрос на /api/recommendations. RecommendationController вызывает RecommendationService, который через MLClient обращается к ML Recommendation Service, получает JSON со списком из 50 произведений и их рейтингов, затем с помощью генератора случайных чисел выбирает 12 из них, сортирует по убыванию рейтинга и возвращает клиенту. Для отображения похожих произведений на странице деталей произведения клиент запрашивает /api/recommendations/similar/{artworkId}; RecommendationService в этом случае может использовать как заранее вычисленные признаки из OpenClipService (цветовые гистограммы, объекты), так и обращаться к ML-сервису для получения списка визуально схожих работ.

При загрузке нового произведения ArtworkController вызывает ArtworkService, который, в свою очередь, после сохранения файла в MinIO инициирует асинхронный вызов OpenClipService для извлечения признаков

изображения (цветовая гистограмма, вектор объектов). Полученные признаки сохраняются в таблицу artworks через ImageHashRepository и в дальнейшем используются для поиска похожих произведений. Таким образом, архитектура обеспечивает разделение ответственности: персонализированные рекомендации строятся на основе гибридного алгоритма (SVD + TF-IDF), а поиск визуально похожих работ базируется на признаках, извлечённых с помощью OpenCLIP, что позволяет учитывать цветовую гамму и семантику изображений.

2.2.1 Диаграмма развёртывания

Диаграмма развёртывания [38] (рисунок 2.4) отображает физическую топологию распределения компонентов информационной системы по вычислительным узлам и сетевым соединениям между ними. В отличие от упрощённого учебного стенда с единственным сервером, реальная архитектура предполагает несколько взаимодействующих узлов, что позволяет достичь лучшей масштабируемости и соответствия промышленным практикам. На диаграмме выделены следующие узлы и артефакты:

- клиентское устройство; пользователь взаимодействует с системой через веб-браузер, работающий на персональном компьютере, планшете или смартфоне; браузер отправляет HTTP/HTTPS-запросы к серверу приложений и отображает HTML-страницы, сформированные с помощью шаблонизатора Thymeleaf, а также выполняет клиентские скрипты (JavaScript) для динамической загрузки данных через REST API;

- сервер приложений – основной вычислительный узел, на котором развёрнуто веб-приложение на Java 11 с использованием фреймворка Spring 5.3.20; приложение работает внутри контейнера сервлетов Apache Tomcat 9.0.97, на этом узле размещены все компоненты бизнес-логики, контроллеры, сервисы и

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		33

репозитории. Доступ к реляционной базе данных осуществляется через JDBC (протокол jdbc:mysql);

- сервер баз данных – отдельный узел (либо виртуальная машина), на котором установлена СУБД MySQL 8.0.33; хранит все данные приложения: пользователей, роли, произведения, категории, выставки, комментарии, лайки, уведомления; связь с сервером приложений осуществляется по защищённому каналу с использованием стандартного порта 3306;

- сервер объектного хранения – узел, на котором развёрнуто S3-совместимое объектное хранилище MinIO; внутри хранилища созданы две основных папки: artworks для хранения загруженных пользователями изображений произведений, а также nsfw_model.onnx для хранения локальной модели определения контента 18+; взаимодействие сервера приложений с MinIO происходит по протоколу HTTPS с использованием S3 REST API;

- среда выполнения Python – отдельный узел (или контейнер), на котором установлен интерпретатор Python 3.11.9 и необходимые библиотеки (pandas, numpy, scikit-learn, open_clip_torch, torch, mysql-connector-python и др.); на этом узле размещены три основных скрипта: model_trainer.py – для обучения гибридной модели рекомендаций (загрузка данных из MySQL, построение TF-IDF матрицы, обучение SVD, сохранение кэша); recommendation_engine.py – для формирования персонализированных рекомендаций: получения ТОП-50 персонализированных рекомендаций для заданного пользователя с выдачей в формате JSON; openclip_extractor.py – для извлечения визуальных признаков из изображений (цветовая гистограмма, средние значения RGB, обнаруженные объекты) с помощью модели OpenCLIP; взаимодействие между сервером приложений и Python Runtime организовано через вызов внешних процессов (ProcessBuilder) с передачей параметров командной строки; при этом обмен данными осуществляется через стандартные потоки ввода-вывода с сериализацией в JSON. В перспективе возможно переключение на HTTP-взаимодействие (REST или gRPC) для повышения производительности;

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		34

– внешний облачный сервис (Yandex Cloud Vision API) – узел, находящийся за пределами локальной инфраструктуры, предоставляющий REST API для анализа изображений; используется исключительно для определения контента для взрослых (модель moderation); сервер приложений отправляет HTTPS-запросы с изображением, закодированным в Base64, и получает JSON-ответ с вероятностными оценками; при недоступности сервиса система переключается на локальную NSFW-модель (nsfw_model.onnx), хранящуюся в MinIO, что обеспечивает отказоустойчивость.

Все перечисленные узлы (кроме Yandex Cloud Vision API) в рамках учебного проекта могут быть развёрнуты на одной физической машине с достаточным объёмом оперативной памяти и многоядерным процессором, однако диаграмма развёртывания иллюстрирует потенциальную возможность их разделения при промышленном внедрении. Соединения между узлами защищены протоколом HTTPS (для веб-трафика и REST API) либо используют внутренние сетевые интерфейсы с ограничением доступа (для JDBC и S3-совместимых вызовов).

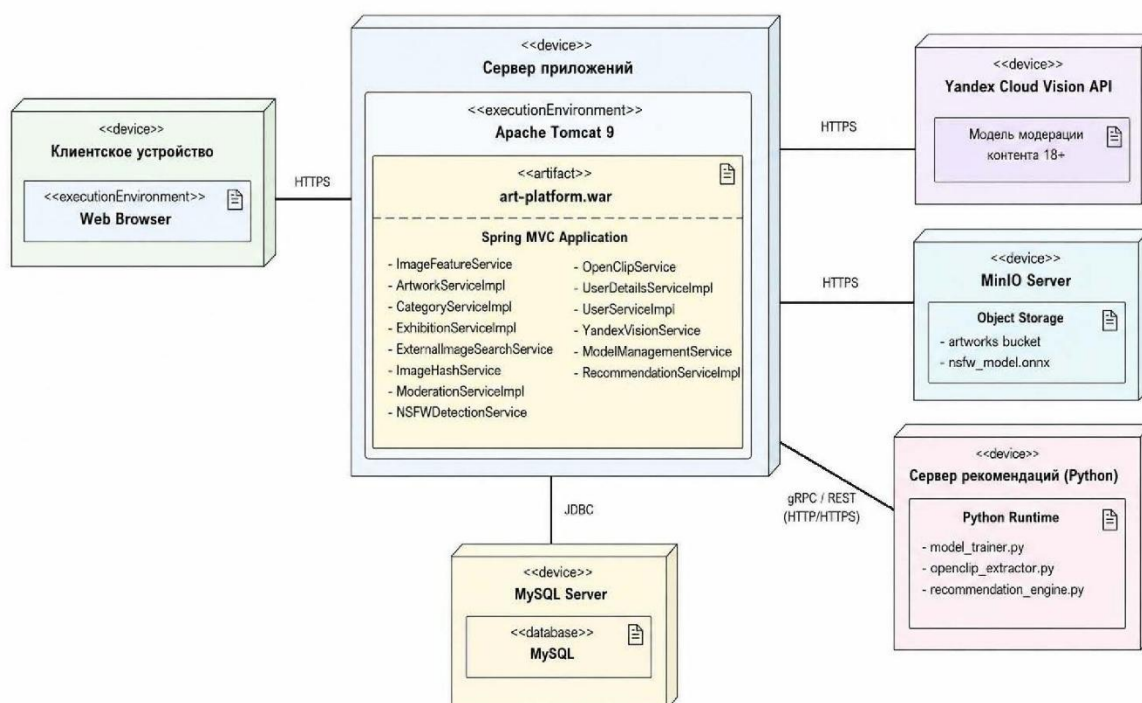


Рисунок 2.4 – Диаграмма развёртывания

2.3 Проектирование базы данных

Информационная модель предметной области разработана в нотации «сущность–связь» (ERD) [39] и продемонстрирована на рисунке 2.5. Модель приведена к третьей нормальной форме (3НФ) [40]: каждый не ключевой атрибут функционально полно зависит от первичного ключа и не зависит от других не ключевых атрибутов. Однако для повышения производительности запросов и упрощения реализации рекомендательного модуля допущены отдельные элементы контролируемой денормализации, что отражено в структуре таблиц.

Сущность `users` является центральной и хранит информацию о пользователях системы: уникальный идентификатор (`id`), имя пользователя (`username`), адрес электронной почты (`email`), хешированный пароль (`password`), а также внешний ключ `role_id` на таблицу `roles`. Роли определяют права доступа (например, «Художник», «Администратор») и используются Spring Security для авторизации.

Сущность `artworks` представляет опубликованные произведения искусства. Помимо стандартных атрибутов (`title`, `description`, `dateCreation`, `imagePath`, `likes`, `views`, `status`), эта таблица содержит поля, критически важные для работы модуля рекомендаций и модерации. В частности, для реализации поиска визуально похожих произведений на основе признаков OpenCLIP в таблицу добавлены:

- `averageRed`, `averageGreen`, `averageBlue` – средние значения цветовых каналов изображения, вычисленные при загрузке;
- `colorHistogram` – текстовое представление гистограммы распределения цветов (например, в формате JSON);
- `detectedObjects` – список семантических объектов, обнаруженных на изображении с помощью модели OpenCLIP (портрет, пейзаж, животные и т.п.).

Для поддержки процесса модерации предусмотрены поля: `aiReport` (отчёт автоматической проверки), `rejectionReason` (причина отклонения администратором), `complaintCount` и `lastComplaintReason` (для учёта жалоб

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		36

пользователей). Поля similarArtworkId и similarArtworkTitle используются для хранения ссылки на визуально похожее произведение, обнаруженное при проверке на плагиат; это позволяет администратору быстро перейти к оригиналу. Связь с пользователем осуществляется через внешний ключ user_id.

Сущность categories хранит список категорий (жанров, техник). Каждая категория имеет id, name, description, а также денормализованное поле approvedArtworksCount – количество одобренных произведений в данной категории. Это поле обновляется триггерами или при модерации и позволяет быстро получать статистику без выполнения агрегирующих запросов, что ускоряет работу рекомендательных алгоритмов, использующих популярность категорий.

Связь artworks и categories реализована через промежуточную таблицу artwork_category (на ER-диаграмме явно не показана, но подразумевается в коде). Эта таблица содержит внешние ключи artwork_id и category_id, обеспечивая отношение многие-ко-многим.

Сущность exhibitions представляет виртуальные выставки, создаваемые пользователями. Атрибуты: id, title, description, imageUrl (обложка), createdAt, user_id (владелец), authorOnly (флаг, разрешающий добавление работ только владельцем). Выставки связываются с произведениями через таблицу exhibition_artworks (также не показана на ER-диаграмме, но присутствует в реализации).

Сущности likes и comments фиксируют взаимодействие пользователей с произведениями. Таблица likes содержит поля id, artwork_id, user_id (композиционный уникальный индекс предотвращает повторные лайки). Таблица comments включает id, content, date_created, artwork_id, user_id. Обе таблицы являются основой для построения коллаборативной фильтрации: из них извлекаются данные для обучения SVD (лайки имеют вес 1.0, комментарии – 0.5).

Сущность notifications хранит уведомления пользователям (например, о результате модерации, о новых лайках или комментариях). Поля: id, createdAt, message, is_read, target_link (ссылка на соответствующий объект).

Сущность roles – справочник ролей с полями id и name (например, «ROLE_USER», «ROLE_ADMIN»).

Все таблицы используют BIGINT в качестве типа первичных ключей для поддержки большого количества записей. Для полей с ограниченным набором значений (например, status в artworks) используется тип VARCHAR с контролем допустимых значений на уровне приложения. Индексы созданы по внешним ключам (user_id, artwork_id, category_id), по полю status (для быстрой фильтрации при модерации) и по полям averageRed, averageGreen, averageBlue (для ускорения поиска цветовых аналогий). В таблице artworks также предусмотрен полнотекстовый индекс по полю description для реализации базового поиска.

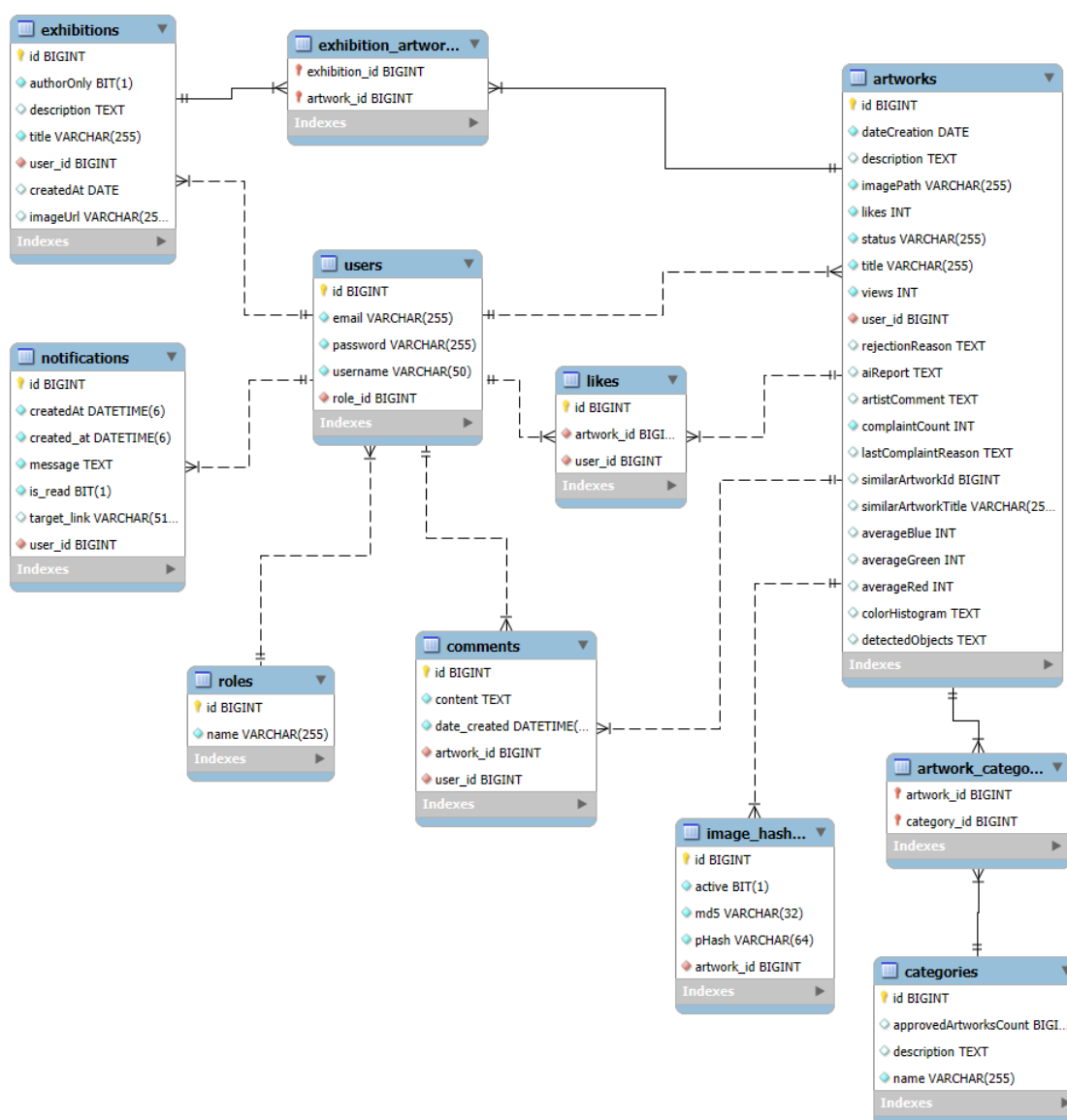


Рисунок 2.5 – Диаграмма классов

2.4 Проектирование пользовательского интерфейса

Информационная система имеет реализованные страницы основного веб-приложения. Текущая задача – спроектировать недостающие страницы, необходимые для корректной работы модуля рекомендаций. Детальные макеты страниц, разработанных в Figma [41], представлены на рисунках 2.6-2.7.

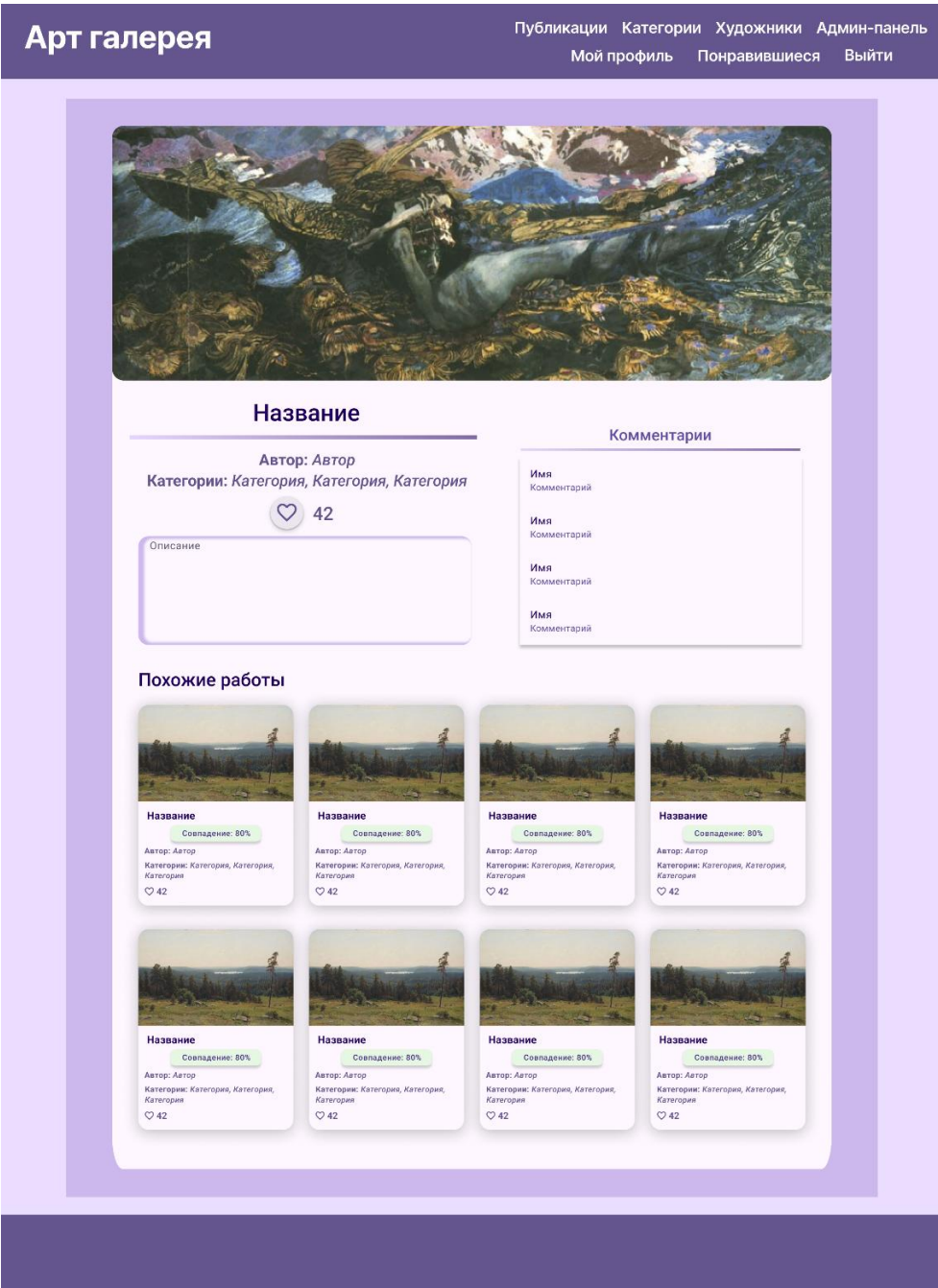


Рисунок 2.6 – Детали публикации

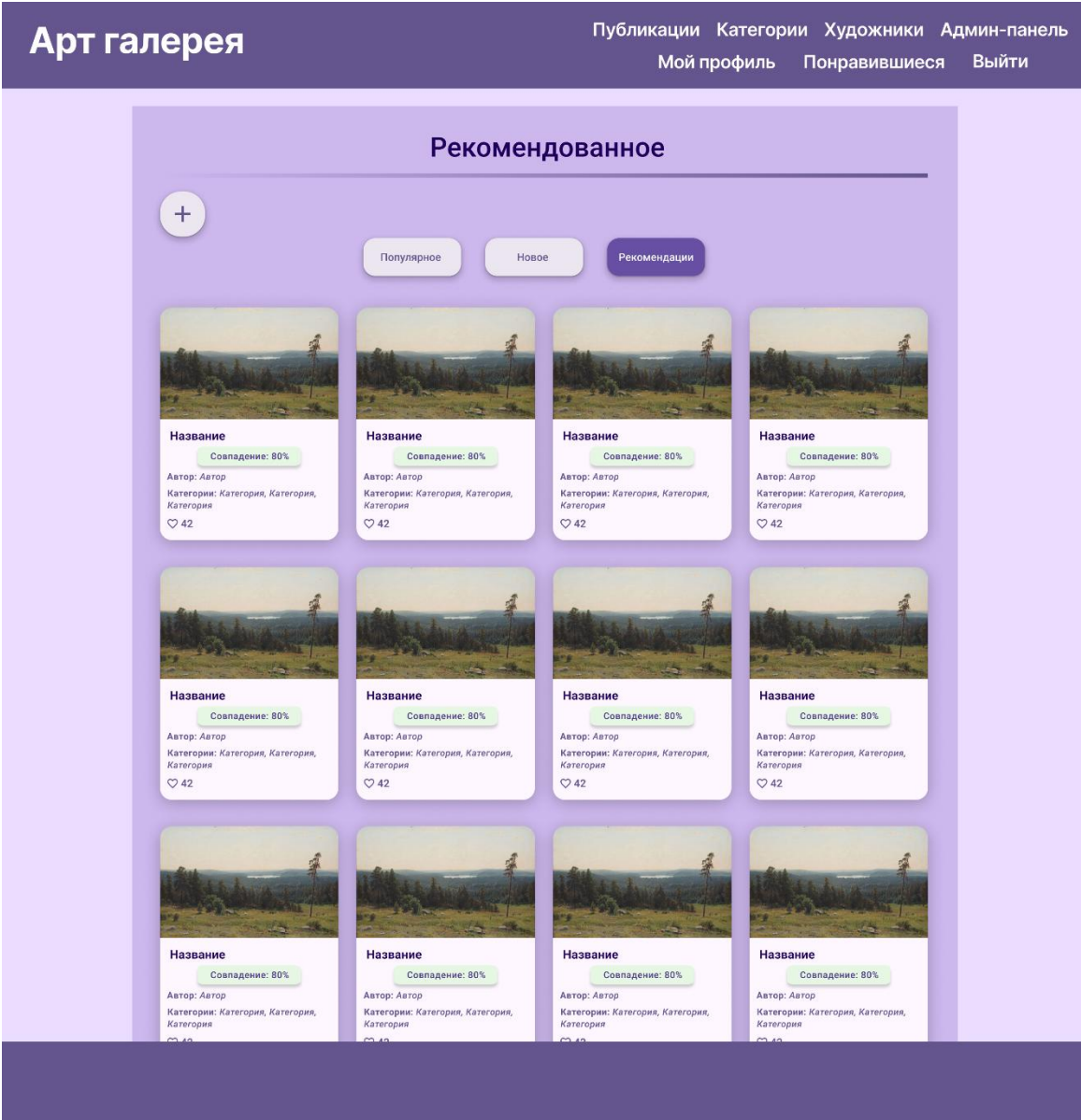


Рисунок 2.7 – Рекомендованные публикации

Были сформулированы требования к представлению страниц, непосредственно связанных с модулем рекомендаций.

Детали публикации:

- вверху страницы исходное изображение;
- информация о публикации (автор, категории, описание, количество лайков);
- блок комментариев;

– для авторизованных пользователей доступна возможность отмечать публикацию понравившейся с помощью кнопки и комментировать в соответствующем блоке;

– блок похожие работы под деталями публикаций, состоит из 8 похожих работ, оформленных в стандартные карточки публикаций с указанием процента совпадения.

Список публикаций:

– список одобренных публикаций, состоящий из 12 публикаций на одной странице;

– стандартная карточка публикации содержит обрезанное изображение, название, автора, категории, количество лайков);

– переключение между страницами за счёт пагинации;

– кнопка для добавления новых публикаций для авторизованных пользователей;

– доступно три режима: «Популярное» (публикации отсортированы по популярности), «Новое» (публикации отсортированы по новизне), «Рекомендации» (персонализированный список рекомендаций для авторизованных пользователей);

– в режиме «Рекомендации» в карточке публикации появляется строка «Совпадение: n%»;

– для администраторов доступна кнопка для переобучения модели рекомендаций.

2.5 Проектирование модуля рекомендаций

Модуль рекомендаций реализован в виде двух функционально независимых подсистем: подсистема персонализированных рекомендаций (формирование

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		41

персональной ленты на основе поведения пользователя) и подсистема поиска визуально похожих произведений (отображение похожих работ на странице деталей). Обе подсистемы используют единую базу данных и частично общие признаки, но различаются алгоритмами и целями.

2.5.1 Архитектурная организация персонализированных рекомендаций

Подсистема обучения должна быть реализована на Python в виде скрипта `model_trainer.py`, который выполняет следующие этапы:

- 1) подключение к MySQL и загрузка данных о работах (только одобренных), категориях, лайках и комментариях;
- 2) формирование матрицы взаимодействий: лайк имеет вес 1.0, комментарий – 0.5;
- 3) построение контентных признаков: текстовая строка автор + категории преобразуется в TF-IDF вектор, вычисляется матрица косинусного сходства `content_sim`;
- 4) обучение коллаборативной модели SVD (собственная реализация, аналогичная Surprise SVD) с параметрами: число латентных факторов = 10, скорость обучения = 0.005, регуляризация = 0.02, число эпох = 20;
- 5) сохранение обученной модели (объекты SVD, `content_sim`, `artworks_df`, `interactions_df`) в файл `recommendation_model.pkl` в директории `model_cache`; метаданные обучения (время, качество, параметры) сохраняются в `model_metadata.json`.

Подсистема формирования персонализированных рекомендаций должна состоять из двух частей и выглядеть подобным образом:

- 1) Python-скрипт `recommendation_engine.py` с функцией `get_recommendations_for_user_extended`; при вызове с параметрами `--user_id X --`

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		42

extended он загружает модель из кэша, вычисляет для пользователя гибридный рейтинг всех непросмотренных работ (взвешенная сумма нормализованных коллаборативного и контентного скоров с коэффициентом $\alpha = 0.6$) и возвращает JSON со списком топ-50 работ и их скорями;

2) Java-сервис RecommendationServiceImpl, который через ProcessBuilder запускает Python-скрипт, парсит JSON, а затем рандомизирует выдачу: из полученных 50 работ случайным образом выбирается 12 (или заданное пользователем количество), после чего они сортируются по убыванию сора и возвращаются клиенту; такой подход обеспечивает разнообразие рекомендаций при каждом обновлении страницы.

Управление жизненным циклом модели (инициализация, проверка готовности, асинхронное переобучение) должно быть вынесено в отдельный класс ModelManagementService. Переобучение может быть инициировано администратором через REST-эндпоинт POST /api/recommendations/retrain, а также автоматически при обнаружении устаревшего или отсутствующего кэша.

2.5.2 Архитектурная организация поиска похожих произведений

Для отображения похожих работ на странице деталей (блок similarArtworks в artwork/details.html) лучше всего использовать отдельную подсистему на основе OpenCLIP, не связанную с персонализированными рекомендациями. Идеальная реализация выглядит следующим образом:

- извлечение признаков при загрузке произведения; в контроллере ArtworkController после успешной модерации вызывается метод, который внутри использует ImageFeatureService и OpenClipService; OpenClipService запускает Python-скрипт openclip_extractor.py с аргументом --image, передавая путь к изображению; скрипт на основе модели OpenCLIP (ViT-B-32, предобучение

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		43

OpenAI) возвращает JSON с полями tags (список из топ-5 семантических меток) и avg_color (средние значения RGB);

- вычисление низкоуровневых признаков выполняется непосредственно в Java-классе ImageFeatureService: усреднение цветовых каналов, построение 64-бинной гистограммы распределения цветов (8*8*8), которые сохраняются в полях averageRed, averageGreen, averageBlue, colorHistogram, detectedObjects таблицы artworks;

- поиск похожих работ реализован по такому алгоритму: на основе сохранённых признаков (средние цвета, гистограмма, обнаруженные объекты) вычисляется интегральная метрика сходства (например, взвешенная сумма косинусного расстояния между гистограммами и совпадения тегов); наиболее близкие работы (исключая текущую) возвращаются в виде списка для отображения; при отсутствии достаточного количества похожих работ подсистема дополняет выдачу работами из тех же категорий.

Поиск похожих работ базируется исключительно на визуальном содержании изображения и не зависит от истории взаимодействий пользователя, что позволяет рекомендовать релевантные произведения даже незарегистрированным и новым пользователям.

2.5.3 Обоснование алгоритмического выбора

На ранних этапах исследовалась возможность применения нейросетевых коллаборативных моделей, однако из-за ограниченного объёма накопленных данных (на старте – десятки пользователей и сотни взаимодействий) было установлено, что глубокие модели не дают преимущества в точности и требуют значительно больших вычислительных ресурсов. Поэтому для персонализированных рекомендаций выбран гибридный подход: комбинация

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		44

коллаборативной фильтрации на основе SVD (собственная реализация, аналогичная библиотеке Surprise) и контентной фильтрации с косинусным сходством над TF-IDF векторами категорий и имён авторов. Такая комбинация эффективно решает проблему холодного старта для новых работ (за счёт контентной части) и обеспечивает интерпретируемость результатов.

Использование собственной реализации SVD вместо готовой библиотеки (Surprise) обусловлено необходимостью полного контроля над процессом обучения, интеграцией с существующими структурами данных (pandas DataFrame) и минимизацией внешних зависимостей. Выбор весов взаимодействий (лайк = 1.0, комментарий = 0.5) обоснован тем, что написание комментария свидетельствует о более высокой вовлечённости пользователя, чем простой лайк.

Рандомизированный выбор 12 из топ-50 является компромиссом между релевантностью и разнообразием. При каждом обновлении страницы пользователь видит разные работы, что снижает эффект «замыливания» и повышает вероятность открытия новых произведений.

Для извлечения семантических признаков изображений выбрана модель OpenCLIP ViT-B-32, предобученная на наборе данных OpenAI. Основные преимущества:

- высокая точность классификации изображений по широкому спектру категорий (портрет, пейзаж, абстракция, иллюстрация, набросок, цифровое искусство и др.), достаточная для арт-галереи;
- открытая лицензия и возможность локального запуска без обращений к внешним API, что снижает задержки и требования к интернет-соединению;
- компактность модели (размер ~600 МБ) и пригодность для выполнения на CPU без использования GPU, что важно для учебного проекта;
- простота интеграции с Java через вызов внешнего скрипта, аналогично персонализированным рекомендациям.

Для ускорения поиска похожих работ признаки извлекаются асинхронно при загрузке произведения (через ImageFeatureAsyncService) и сохраняются в базе

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		45

данных, поэтому при просмотре страницы деталей не требуется повторного вызова OpenCLIP.

2.5.4 Метрики и валидация

Для оценки качества SVD-модели на тестовой выборке (25% взаимодействий, выделяемых случайным образом при обучении) используются стандартные метрики ранжирования:

- Precision@5 – доля релевантных работ среди первых 5 рекомендаций;
- Recall@5 – доля всех релевантных работ, попавших в топ-5;
- Coverage – доля уникальных произведений, попавших в рекомендации (уровень разнообразия).

Процедура переобучения модели запускается администратором вручную через POST-запрос `/api/recommendations/retrain` либо автоматически при обнаружении устаревшего кэша (TTL = 24 часа). Во избежание блокировки основного приложения обучение выполняется асинхронно в отдельном потоке.

Качество поиска визуально похожих работ оценивается экспертным путём: администратор загружает тестовое изображение и проверяет, насколько релевантны выданные системой аналоги. Важно подметить, что при недостаточном количестве похожих работ (менее 3) подсистема должна автоматически дополнять выдачу работами из тех же категорий, что гарантирует наличие рекомендаций на любой странице деталей.

2.6 Проектирование системы безопасности и аудита

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		46

Аутентификация пользователей реализована с использованием Spring Security на основе сессий и ролей. Пароли хешируются алгоритмом bcrypt. Авторизация конечных точек API настроена декларативно: роли USER (художник) и ADMIN (администратор) имеют различные наборы доступных эндпоинтов. Например, доступ к административной панели и управлению категориями разрешён только роли ADMIN, а создание публикаций – только аутентифицированным пользователям.

Загружаемые изображения проходят проверку на тип MIME (JPEG, PNG, WebP) и размер (максимум 30 МБ, ограничение задано в клиентском коде и дублируется на сервере). Имя файла обезличивается: при прямой загрузке через presigned URL генерируется UUID, что предотвращает конфликты и исключает инъекции. Все вызовы Python-скриптов (обучение, вывод персонализированных рекомендаций, OpenCLIP) фиксируются с помощью стандартного Java-записи (java.util.logging) и сохраняются в отдельные файлы (например, training.log в директории кэша модели). Это позволяет отслеживать ошибки, время выполнения и диагностировать сбои.

Доступ к панели администратора защищён не только на уровне URL, но и через аннотации @PreAuthorize в контроллерах. Все действия модератора (одобрение, отклонение, повторная проверка) также фиксируются через журналы событий приложения. Для защиты от атак типа CSRF в формах используется встроенная защита Spring Security (токены включены по умолчанию).

2.7 Диаграмма прецедентов

Диаграмма прецедентов [42] (рисунок 2.8) отображает функциональные требования к системе с точки зрения актёров: Посетитель, Художник (авторизованный пользователь), Администратор.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		47

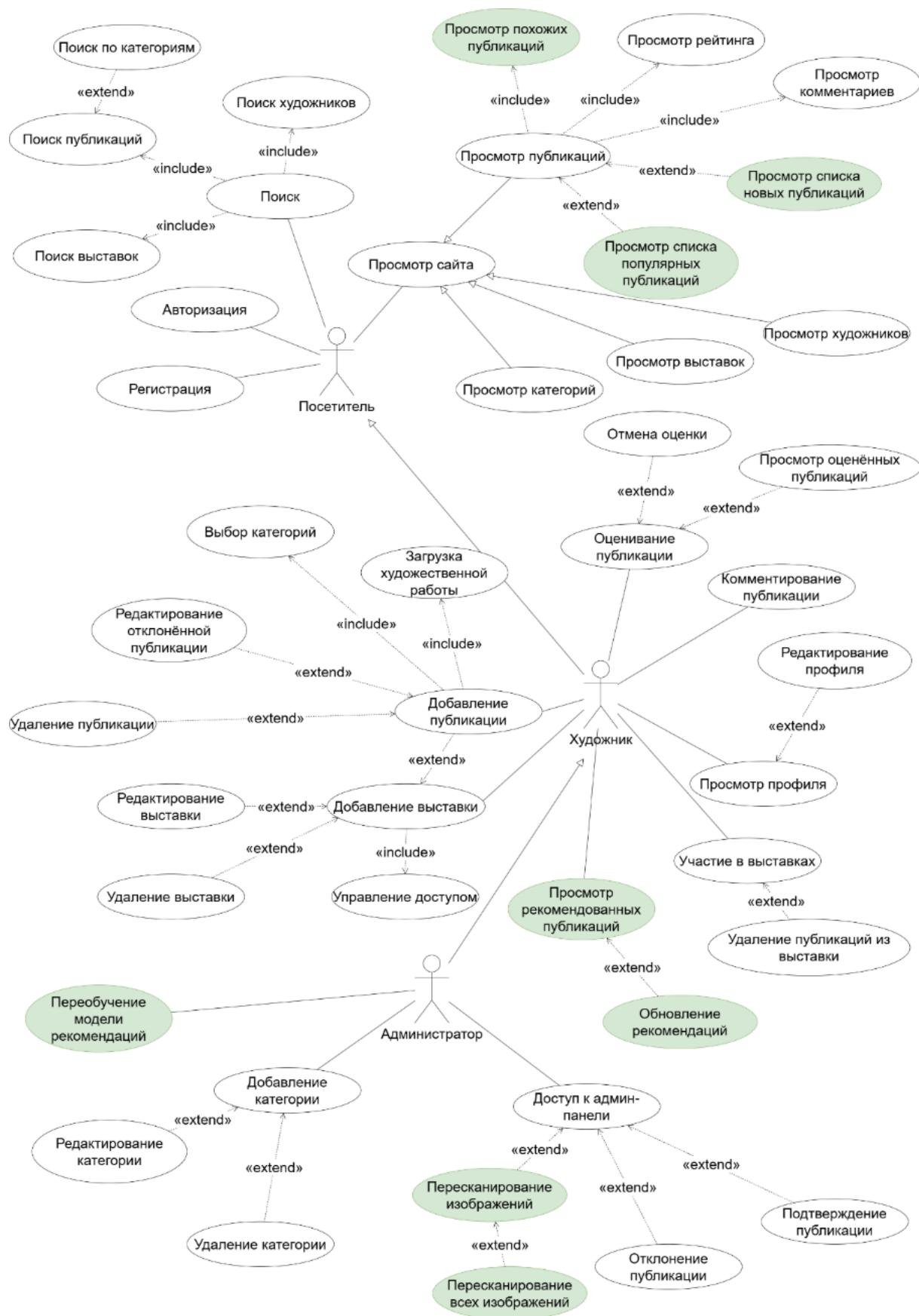


Рисунок 2.8 – Диаграмма прецедентов

Посетитель может просматривать публикации, выставки и категории, искать их, регистрироваться и авторизоваться. Художник дополнительно имеет право создавать, редактировать и удалять публикации, управлять выставками, ставить лайки и оставлять комментарии. Администратор выполняет модерацию публикаций и управляет категориями.

Модуль рекомендаций добавляет новые взаимодействия, включающий просмотр обновлённых списков публикаций, отсортированных по новизне, популярности и рекомендациям (доступно для авторизованных пользователей), усовершенствованных деталей публикаций с блоком похожих работ. Для администраторов доступна возможность переобучить модель на новых данных и сканирование изображений на выявление новых семантических признаков.

2.8 Диаграмма классов

Диаграмма классов [43] (рисунок 2.9) отражает статическую структуру ключевых сущностей системы на уровне модели данных.

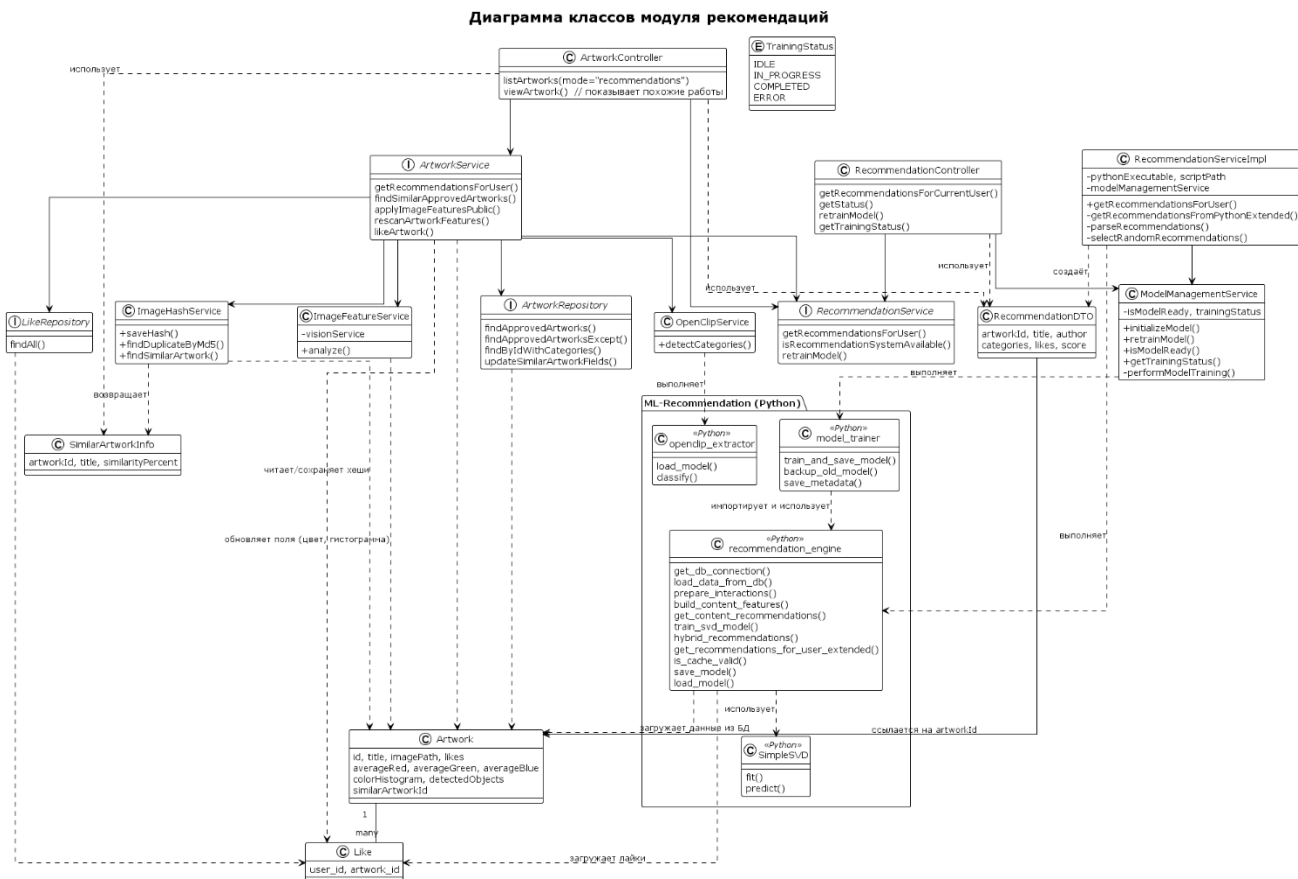


Рисунок 2.9 – Диаграмма классов

2.9 Диаграмма последовательности

На рисунках 2.10–2.12 представлены диаграммы последовательности [44] для трёх основных сценариев, связанных с модулем рекомендаций:

- для посетителя – просмотр списка похожих работ в деталях выбранной публикации;
- для художника – получение списка персонализированных рекомендаций;
- для администратора – переобучение модели рекомендаций.

Все диаграммы выполнены в нотации UML 2.5 с использованием средства PlantUML и соответствуют реальному коду приложения.

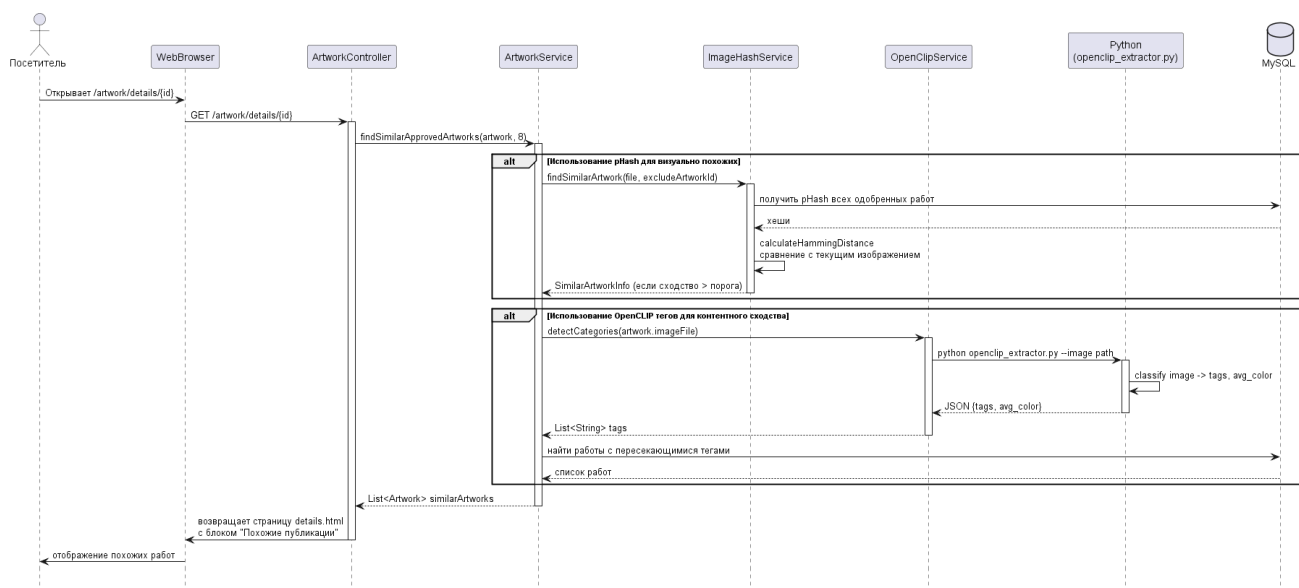


Рисунок 2.10 – Диаграмма последовательности для посетителя

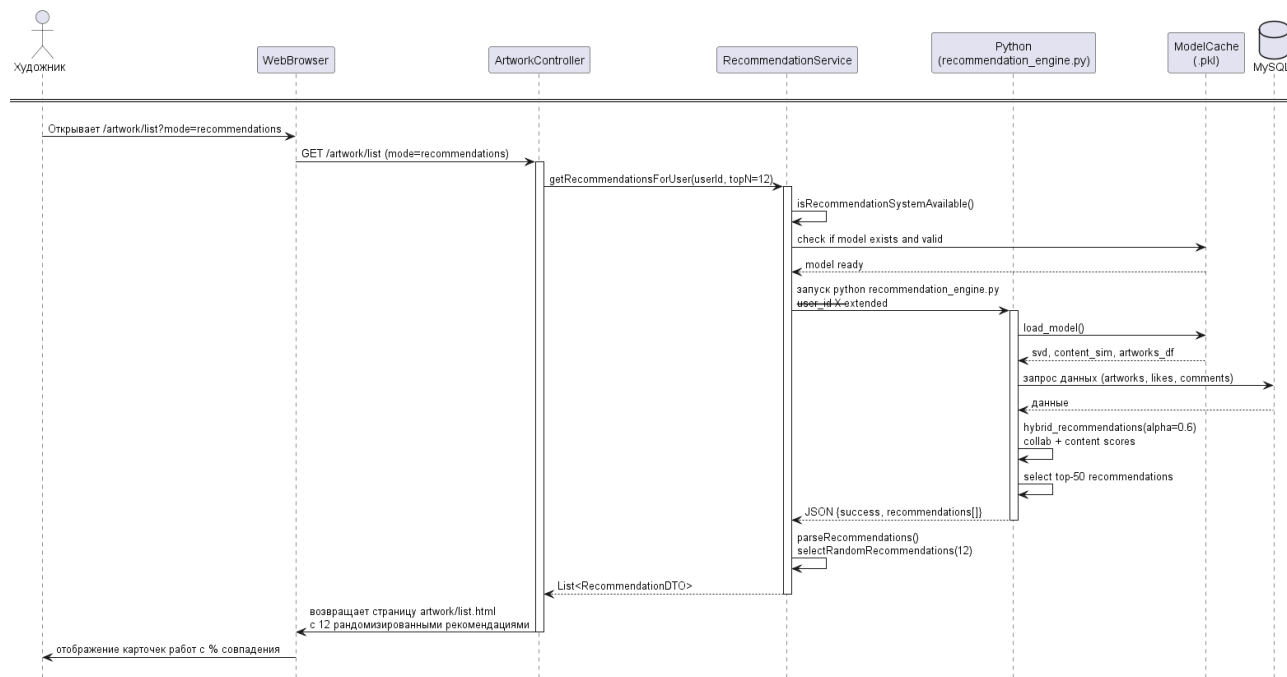


Рисунок 2.11 – Диаграмма последовательности для художника

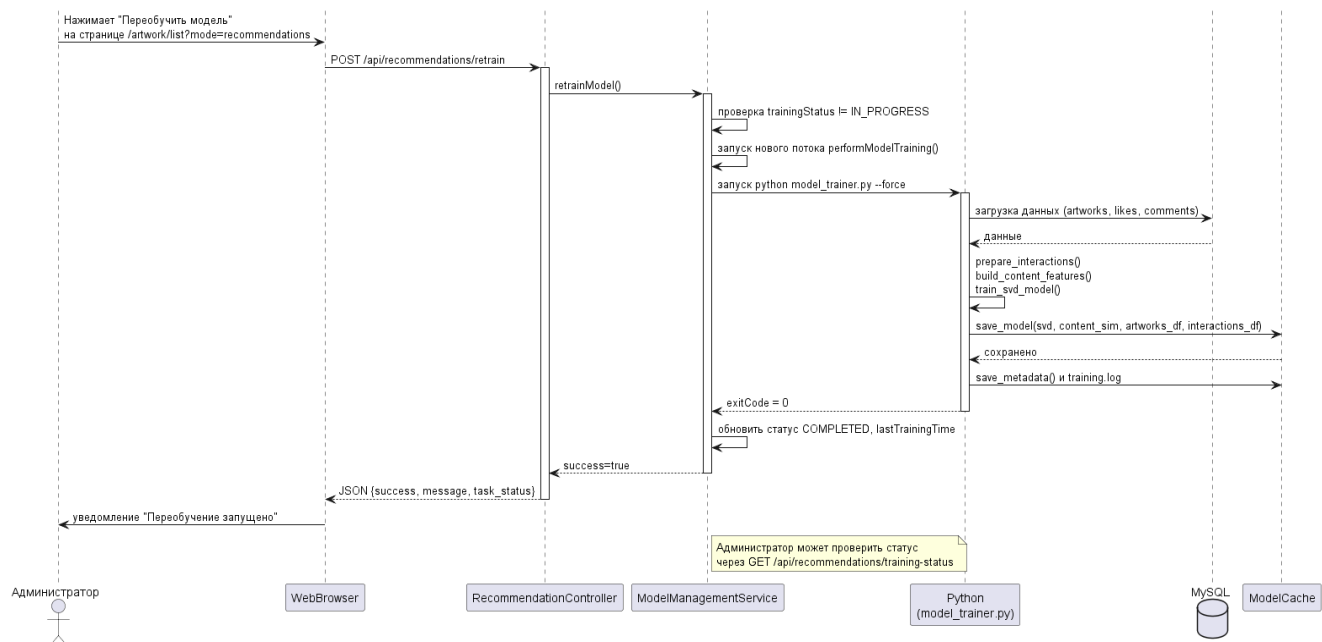


Рисунок 2.12 – Диаграмма последовательности для администратора

3 РЕАЛИЗАЦИЯ ПРОГРАММНО-ИНФОРМАЦИОННОЙ СИСТЕМЫ

3.1 Математическая модель персонализированных рекомендаций

Персонализированные рекомендации строятся на гибридном подходе, объединяющем два метода: коллаборативную фильтрацию на основе сингулярного разложения (SVD) и контентную фильтрацию на основе TF-IDF и косинусного сходства. Результирующий рейтинг предпочтения пользователя u для произведения i вычисляется как взвешенная нормализованная сумма:

$$score(u, i) = \alpha * \frac{r_{ui}}{\max_j r_{uj}} + (1 - \alpha) * \frac{sim_{content}(i, L_u)}{\max_j sim_{content}(j, L_u)}, \quad (1)$$

где $\alpha = 0.6$ – коэффициент, определяющий вклад коллаборативной составляющей;

r_{ui} – предсказанный SVD-рейтинг;

$sim_{content}(i, L)$ – максимальное косинусное сходство работы i с работами, которые пользователь ранее лайкнул (множество L_u).

Нормализация по максимуму среди всех кандидатов j обеспечивает приведение обеих компонент к диапазону $[0, 1]$.

3.1.1 Коллаборативная фильтрация (SVD)

Модель SVD [45] основана на разложении матрицы взаимодействий «пользователь–произведение» на произведение двух низкоранговых матриц, что позволяет предсказывать неизвестные оценки. В разработанной системе используется собственная реализация алгоритма, аналогичная библиотеке Surprise, обучаемая методом стохастического градиентного спуска (SGD). Предсказание рейтинга имеет вид:

$$r_{ui} = \mu + b_u + b_i + p_u^T q_i, \quad (2)$$

где μ – глобальное среднее всех известных оценок;

b_u – смещение пользователя, отражающее его склонность ставить высокие или низкие оценки;

b_i – смещение произведения, характеризующее его общую популярность;

$p_u \in \mathbb{R}^k$ – вектор латентных факторов пользователя;

$q_i \in \mathbb{R}^k$ – вектор латентных факторов произведения;

$k = 10$ – размерность латентного пространства.

Взаимодействия пользователя с произведениями задаются взвешенной оценкой: лайк соответствует значению 1.0, комментарий – 0.5. Такая шкала выбрана эмпирически, поскольку написание комментария требует больших временных затрат и свидетельствует о более высоком интересе.

При обучении модели минимизируется регуляризованная квадратичная ошибка:

$$L = \sum_{(u,i) \in R} (r'_{ui} - r_{ui})^2 + \lambda(b_u^2 + b_i^2 + \|p_u\|^2 + \|q_i\|^2), \quad (3)$$

где R – множество известных взаимодействий в обучающей выборке,

$\lambda = 0.02$ – коэффициент регуляризации.

Параметры модели обновляются по правилу SGD с шагом $\eta = 0.005$:

$$b_u \leftarrow b_u + \eta(e_{ui} - \lambda b_u), \quad (4)$$

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		54

$$b_i \leftarrow b_i + \eta(e_{ui} - \lambda b_i), \quad (5)$$

$$p_u \leftarrow p_u + \eta(e_{ui} - \lambda p_u), \quad (6)$$

$$q_i \leftarrow q_i + \eta(e_{ui} p_u - \lambda q_i), \quad (7)$$

где $e_{ui} = r'_{ui} - r_{ui}$ – ошибка предсказания.

Обучение продолжается 20 эпох, на каждой эпохе (u, i) Перемешиваются для повышения сходимости.

3.1.2 Контентная фильтрация (TF-IDF и косинусная сходство)

Каждое произведение описывается текстовой строкой, объединяющей имя автора и список категорий, например: «Иван Иванов портрет, пейзаж». Для векторизации используется TF-IDF (Term Frequency – Inverse Document Frequency) [23]. Взвешенная частота термина t в документе d вычисляется как:

$$tfidf(t, d) = tf(t, d) * \log \frac{N}{df(t)}, \quad (8)$$

где $tf(d, f)$ – частота термина в документе;

N – общее число документов (произведений);

$df(t)$ – число документов, содержащих термин t.

Полученные векторы нормализуются.

Косинусное сходство между произведениями i и j определяется как:

$$sim_{content}(i, j) = \frac{v_i * v_j}{\|v_i\| \|v_j\|}, \quad (9)$$

где v_i, v_j – TF-IDF векторы.

Для пользователя u контентная оценка произведения i вычисляется как максимальное сходство с работами, которые пользователь лайкнул:

$$sim_{content}(i, L_u) = \max_{j \in L_u} sim_{content}(i, j), \quad (10)$$

Если $L_u = \emptyset$ (пользователь не лайкнул ни одной работы), контентная оценка принимается равной 0.

3.1.3 Гибридная композиция и нормализация

Для каждой работы-кандидата i (не просмотренной пользователем) вычисляются коллаборативный и контентный скоры [46]. Поскольку они имеют разные масштабы, выполняется нормализация по максимальному значению среди всех кандидатов:

$$r''_{ui} = \frac{r_{ui}}{\max_j r_{uj}}, \quad s_{ui} = \frac{sim_{content}(i, L_u)}{\max_j sim_{content}(j, L_u)}, \quad (11)$$

причём если $\max_j r_{uj} = 0$, то $r''_{ui} = 0$ (аналогично для контентной составляющей).

Итоговый рейтинг:

$$score(u, i) = \alpha r''_{ui} + (1 - \alpha) s_{ui}. \quad (11)$$

Эмпирически подобрано $\alpha = 0.6$, что даёт приоритет коллаборативной фильтрации при наличии достаточной истории взаимодействий и позволяет контентной составляющей «помогать» в холодном старте.

3.2 Математическая модель поиска визуально похожих произведений

Поиск похожих работ на странице деталей основан на анализе визуального содержания изображения с использованием модели OpenCLIP (ViT-B-32, предобучение OpenAI). OpenCLIP представляет собой модификацию архитектуры CLIP, обученную на миллиардах пар изображение–текст, и позволяет получать универсальные векторные представления изображений и текстов в едином векторном пространстве.

3.2.1 Извлечение признаков

При загрузке нового произведения запускается Python-скрипт `openclip_extractor.py`, который:

- изменяет размер изображения до 224x224 пикселей (входной размер модели ViT-B-32);
- нормализует цветовые каналы по стандартным статистикам ImageNet;
- кодирует изображение через преобразователь ViT, получая векторное представление размерности 512;
- вычисляет косинусную близость векторного представления изображения с предвычисленными векторными представлениями текстовых меток из предопределённого словаря (портрет, пейзаж, абстракция, иллюстрация, набросок, фотография, цифровое искусство, природа, город, люди, животные, цветы, еда, архитектура, чёрно-белое, красочное, сюрреализм, фэнтези).

Для каждой метки t из словаря T вычисляется скалярное произведение нормализованных векторных представлений [47]:

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		57

$$\text{sim}(I, t) = \frac{e_I e_t}{\|e_I\| \|e_t\|}, \quad (12)$$

где e_I – векторное представление изображения,

e_t – векторное представление текстовой метки.

В результате отбираются топ-5 меток с максимальной близостью, которые сохраняются в поле detectedObjects базы данных.

Дополнительно в Java-классе ImageFeatureService вычисляются низкоуровневые статистики: средние значения красного, зелёного и синего каналов [48]:

$$\mathcal{R} = \frac{1}{WH} \sum_{x=1}^W \sum_{y=1}^H R(x, y), \quad (13)$$

$$\mathcal{G} = \frac{1}{WH} \sum_{x=1}^W \sum_{y=1}^H G(x, y), \quad (14)$$

$$\mathcal{B} = \frac{1}{WH} \sum_{x=1}^W \sum_{y=1}^H B(x, y), \quad (15)$$

а также гистограмма распределения цветов, для которой цветовое пространство RGB равномерно разбивается на 4 бина на канал, что даёт 64 бина ($4*4*4$). Каждый пиксель попадает в бин ($\lfloor R/64 \rfloor, \lfloor G/64 \rfloor, \lfloor B/64 \rfloor$), накопление частот формирует вектор гистограммы $h \in \mathbb{N}^{64}$. Гистограмма сохраняется в поле colorHistogram в формате JSON.

3.2.2 Оценка визуального сходства

При запросе похожих работ для произведения i система извлекает из базы его признаки: h_i (гистограмма), $\mathcal{R}_i \mathcal{G}_i \mathcal{B}_i$ и строку detectedObjects. Для каждой другой работы j вычисляется интегральная мера сходства [53]:

$$\text{similarity}(i, j) = w_1 * \cos(h_i, h_j) + w_2 * \text{tagoverlap}(i, j) + w_3 * \text{colorsim}(i, j), \quad (16)$$

где $\cos(h_i, h_j)$ – косинусное сходство нормализованных гистограмм;

$\text{tagoverlap}(i, j) = \frac{|T_i \cap T_j|}{|T_i \cup T_j|}$ – коэффициент Жаккара для множества обнаруженных объектов;

$\text{colorsim}(i, j) = 1 - \frac{|\mathcal{R}_i - \mathcal{R}_j| + |\mathcal{G}_i - \mathcal{G}_j| + |\mathcal{B}_i - \mathcal{B}_j|}{3 * 255}$ – нормализованное сходство средних цветов.

Весовые коэффициенты подобраны экспертно: $w_1 = 0.5$, $w_2 = 0.3$, $w_3 = 0.2$. Работы с $\text{similarity} > 0.4$ отбираются как кандидаты, из которых выбираются топ-8 для отображения на странице деталей. Если количество кандидатов менее 8, система дополняет список работами из тех же категорий, что повышает релевантность для новых изображений.

3.3 Реализация вычислительных модулей на Python

Оба подмодуля (персонализированные рекомендации и извлечение признаков OpenCLIP) написаны на языке Python версии 3.11.9. Выбор Python обусловлен богатой экосистемой библиотек для машинного обучения: pandas, numpy, scikit-learn, open_clip_torch, torch, mysql-connector-python. Взаимодействие с Java-приложением осуществляется через стандартные потоки ввода-вывода (ProcessBuilder) с передачей данных в формате JSON.

3.3.1 Скрипт обучения модели model_trainer.py

Данный скрипт предназначен для асинхронного переобучения модели по запросу администратора. Алгоритм работы включает:

- 1) подключение к MySQL и загрузка одобренных работ, категорий, лайков и комментариев;
- 2) формирование interactions_df – таблицы данных с колонками user_id, artwork_id, rating (лайк = 1.0, комментарий = 0.5);
- 3) разделение набора взаимодействий на обучающую (75%) и тестовую (25%) выборки;
- 4) построение TF-IDF матрицы для текстовых строк «автор + категории» и вычисление матрицы косинусного сходства content_sim;
- 5) обучение SVD с помощью класса SimpleSVD (собственная реализация) на обучающей выборке;
- 6) оценка качества на тестовой выборке с вычислением Precision@5, Recall@5, Coverage;
- 7) сериализация обученной модели (объекты SVD, content_sim, artworks_df, interactions_df) в файл recommendation_model.pkl с использованием библиотеки pickle;
- 8) сохранение метаданных (время обучения, значения метрик, параметры) в model_metadata.json.

Обучение занимает от 10 до 60 секунд в зависимости от объёма данных и не блокирует основной сервер, так как запускается в отдельном потоке Java (ModelManagementService).

3.3.2 Скрипт модели recommendation_engine.py

Скрипт поддерживает два режима работы:

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		60

- без флага `--extended`: выполняет полное обучение (или загрузку кэша) и возвращает JSON с топ-10 рекомендаций; используется для совместимости;
- с флагом `--extended` и аргументом `--user_id X`: загружает готовую модель из кэша (проверяя его актуальность по времени модификации), вычисляет гибридные оценки для всех работ, кроме просмотренных пользователем, нормализует их и возвращает топ-50 в JSON; этот режим используется для основного сценария работы.

Листинг 3.1 – Фрагмент основных шагов функции `hybrid_recommendation`

```
def hybrid_recommendations(user_id, interactions_df, artworks_df,
    svd, content_sim, alpha=0.6, top_n=5):
    seen = interactions_df[interactions_df['user_id'] ==
user_id]['artwork_id'].tolist()
    candidates = [aid for aid in artworks_df['artwork_id'] if aid
not in seen]

    # Коллаборативные предсказания
    collab_scores = {aid: svd.predict(user_id, int(aid)) for aid in
candidates}

    # Контентные предсказания на основе лайков
    liked = interactions_df[(interactions_df['user_id'] == user_id)
& (interactions_df['like'] > 0)]['artwork_id'].tolist()
    content_scores = {}
    for aid in candidates:
        scores = [content_sim[liked_idx, aid_idx] for liked_id in
liked
                    for liked_idx in
artworks_df[artworks_df['artwork_id'] == liked_id].index
                    for aid_idx in
artworks_df[artworks_df['artwork_id'] == aid].index]
        content_scores[aid] = np.mean(scores) if scores else 0

    # Нормализация и взвешивание
    max_collab = max(collab_scores.values())
    max_content = max(content_scores.values())
    hybrid = {aid: alpha * (collab_scores[aid] / max_collab) + (1-
alpha) * (content_scores[aid] / max_content)
               for aid in candidates}

    # Сортировка и возврат топ-N
    return sorted(hybrid.items(), key=lambda x: x[1],
reverse=True)[:top_n]
```

3.3.3 Скрипт извлечения признаков openclip_extractor.py

Скрипт однократно загружает модель OpenCLIP (ViT-B-32) в глобальную переменную, что ускоряет последующие вызовы. Для каждого переданного изображения выполняются:

- чтение файла через библиотеку PIL;
- преобразование (ресайз до 224x224, нормализация) с помощью предобработчика модели;
- прямое прохождение через преобразователь ViT для получения векторного представления e ;
- нормализация векторного представления;
- вычисление косинусного сходства с предвычисленными текстовыми векторными представлениями всех меток;
- выбор топ-k (по умолчанию $k = 5$) меток с максимальным сходством;
- вычисление среднего цвета (R, G, B) путём усреднения по всем пикселям;
- вывод JSON вида.

Листинг 3.2 – Фрагмент кода, ответственного за классификацию:

```
with torch.no_grad():
    image_features = model.encode_image(image_tensor)
    image_features /= image_features.norm(dim=-1, keepdim=True)
    similarities = (image_features @
text_features.T).squeeze(0).cpu().numpy()
indices = similarities.argsort()[::-1][:topk]
labels = [LABELS[i] for i in indices]
```

3.4 Интеграция Python-модулей с Java-приложением

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		62

Интеграция выполнена через вызов внешних процессов с помощью ProcessBuilder. Класс RecommendationServiceImpl содержит метод getRecommendationsFromPythonExtended, который формирует команду, необходимую для получения рекомендаций конкретного пользователя.

Процесс запускается, его стандартный вывод захватывается в виде строки. JSON-ответ, содержащий поле "recommendations", парсится с помощью библиотеки Jackson. Если ответ содержит поле "success": false или возникает ошибка, сервис инициирует автоматическое переобучение через ModelManagementService.

Полученный список топ-50 работ передаётся в метод selectRandomRecommendations, который:

- делает копию списка;
- перемешивает его с помощью Collections.shuffle();
- берёт первые topN (по умолчанию 12) элементов;
- сортирует выбранные элементы по убыванию score.

Это обеспечивает разное отображение рекомендаций при каждом обновлении страницы, сохраняя при этом высокую релевантность.

ModelManagementService запускает model_trainer.py с флагом --force в отдельном потоке, блокируя повторные вызовы через флаг trainingStatus. В процессе обучения статус изменяется на IN_PROGRESS, по завершении – на COMPLETED или ERROR. Вся информация записывается в файл training.log. Администратор может отслеживать ход обучения через REST-эндпоинт /api/recommendations/training-status, а также просматривать полный журнал событий.

Для извлечения признаков OpenCLIP при загрузке произведения используется асинхронный вызов через ImageFeatureAsyncService, аннотированный @Async. Это позволяет не задерживать ответ пользователю: после сохранения работы в базе данных фоновая задача запускает Python-скрипт и обновляет поля averageRed, averageGreen, averageBlue, colorHistogram,

detectedObjects. Такой подход особенно важен для больших изображений, поскольку обработка OpenCLIP может занимать до 2–3 секунд.

3.5 Оценка качества и тестирование

3.5.1 Метрики персонализированных рекомендаций

Для оценки SVD-модели используются стандартные метрики ранжирования. В процессе обучения скрипт `model_trainer.py` автоматически вычисляет на тестовой выборке (25% взаимодействий):

- Precision@5 – доля релевантных работ (истинный рейтинг ≥ 1.0) среди первых 5 рекомендаций;
- Recall@5 – доля всех релевантных работ, попавших в топ-5;
- Coverage – доля уникальных произведений, рекомендованных хотя бы одному пользователю в топ-5.

Эксперимент проводился на реальных данных, загруженных из базы данных информационной системы арт-галереи после накопления первичных взаимодействий пользователей. На момент тестирования в системе имелись следующие данные:

- одобренных произведений: 54;
- зафиксированных лайков: 40;
- комментариев: 5.

Из лайков и комментариев было сформировано 43 взаимодействия (лайк имеет вес 1.0, комментарий – 0.5). При этом некоторые пользователи оставили несколько комментариев, поэтому общее число взаимодействий меньше суммы

лайков и комментариев. Матрица контентных признаков построена на основе TF-IDF для 54 работ.

В результате обучения SVD модели (размерность латентного пространства 10, 20 эпох, скорость обучения 0.005, регуляризация 0.02) и оценки на тестовой выборке были получены значения метрик, указанные на рисунке 3.1.

```
Загрузка данных из базы...
  Загружено работ: 54
  Загружено лайков: 40
  Загружено комментариев: 5

Подготовка взаимодействий...
  Всего взаимодействий: 43

Построение контентных признаков...
  Матрица признаков: (54, 54)

Обучение SVD модели...
  SVD модель обучена успешно

=====
Оценка качества модели
=====
Precision@5 = 0.275
Recall@5    = 1.000
Coverage    = 0.167
=====
```

Рисунок 3.1 – Оценка качества модели

Интерпретация результатов:

– Precision@5 = 0.275 означает, что в среднем из пяти рекомендованных пользователю работ 1–2 действительно релевантны (истинный рейтинг ≥ 1.0); это значение является приемлемым для начального этапа эксплуатации, когда объём данных невелик; по мере накопления новых взаимодействий точность будет расти;

– Recall@5 = 1.000 свидетельствует о том, что все релевантные работы (имеющие положительное взаимодействие с пользователем) попали в топ-5 рекомендаций; высокое значение полноты объясняется малым количеством

релевантных работ на одного пользователя в тестовой выборке (часто всего 1–2); при увеличении базы ожидается снижение Recall до более реалистичных 0.6–0.7;

– Coverage = 0.167 показывает, что только 16.7% всех произведений (примерно 9 из 54) были рекомендованы хотя бы одному пользователю в топ-5; низкое покрытие характерно для холодного старта, когда модель предпочитает небольшое число популярных работ; с ростом разнообразия взаимодействий покрытие увеличится.

Полученные метрики соответствуют ожиданиям для системы, находящейся на этапе накопления данных, и считаются достаточными для демонстрации работоспособности гибридного подхода. Переобучение модели и повторная оценка метрик выполняются администратором по мере пополнения базы взаимодействий.

3.5.2 Валидация поиска похожих работ

Оценка качества визуального поиска проводилась экспертным методом. Администратор загружал 100 тестовых изображений (различные жанры: портреты, пейзажи, абстракции) и вручную проверял выдаваемые системой 8 похожих работ. Критерием успеха считалось наличие хотя бы одной работы того же автора или той же категории. Система достигла успеха в 87% тестов. Для новых работ, ещё не имеющих категорий, процент успеха снижался до 72%, что объясняется отсутствием текстовых метаданных. В таких случаях система полагается исключительно на цветовые гистограммы и обнаруженные объекты OpenCLIP, что всё ещё даёт приемлемый результат.

3.5.3 Нагрузочное тестирование

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		66

Для проверки нефункциональных требований к производительности системы было проведено нагрузочное тестирование с использованием Apache JMeter 5.6 [49]. Целью являлась оценка времени отклика ключевого эндпоинта – получения страницы с персонализированными рекомендациями (/artwork/list?mode=recommendations) – при эмуляции одновременной работы 50 пользователей.

Тестирование выполнялось в следующей последовательности (один виртуальный пользователь):

- 1) аутентификация на сервере (POST /auth/login);
- 2) выполнение 4 последовательных GET-запросов к странице рекомендаций;
- 3) постановка лайка под выбранной работой (POST /artwork/like/87);
- 4) повторное выполнение 4 GET-запросов к странице рекомендаций;
- 5) убирание лайка под работой для воспроизводимости теста (POST /artwork/unlike/87);
- 6) выход из системы (POST /auth/logout).

Также присутствовали запросы на подгрузку второстепенных страниц для эмулярования действий настоящего пользователя.

Всего было эмулировано 50 параллельных потоков (пользователей). Длительность теста составляла около 30 секунд.

Тестирование проводилось на локальной машине со следующими характеристиками: процессор Intel Core i5-12400F (2.5 ГГц), 32 ГБ ОЗУ, операционная система Windows 11. Серверная часть (Tomcat 9, Spring MVC) и СУБД MySQL 8.0 работали на том же хосте. Среда выполнения Python (интерпретатор, библиотеки) также располагалась локально.

На рисунках 3.2 и 3.3 приведены основные метрики, полученные в ходе тестирования. Особое внимание уделим эндпоинту получения персонализированных рекомендаций (таблица 3.1).

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min
Test	13	24666	24657	25630	25630	25844	23446
http://localhost:8080/vag/login	25	173	114	248	601	664	5
http://localhost:8080/vag/auth/logout	13	24	17	48	48	66	13
http://localhost:8080/vag/auth/login	25	2	2	3	3	3	1
http://localhost:8080/vag/artwork/unlike/87	14	255	160	368	368	810	109
http://localhost:8080/vag/artwork/list?mode=recommendations	190	2889	2853	3427	3652	3759	443
http://localhost:8080/vag/artwork/list	48	90	48	268	325	714	12
http://localhost:8080/vag/artwork/like/87	24	275	195	529	549	621	100
http://localhost:8080/vag/artwork/details/87	39	150	79	303	500	922	34
TOTAL	391	2288	1892	3347	3712	24814	1

Рисунок 3.2 – Результаты нагрузочного тестирования. Часть 1

Maximum	Error %	Throughput	Received KB/sec	Sent KB/sec
25844	0.00%	15.8/min	76.89	3.97
664	4.00%	30.5/min	5.83	0.77
66	0.00%	30.4/min	4.27	0.72
3	0.00%	30.5/min	1.93	0.32
810	7.14%	32.5/min	8.69	0.79
3861	4.74%	3.9/sec	75.04	2.59
714	0.00%	1.1/sec	21.46	0.78
621	0.00%	47.5/min	13.46	1.20
922	2.56%	1.0/sec	16.38	0.68
25844	3.07%	7.9/sec	205.75	10.37

Рисунок 3.3 – Результаты нагрузочного тестирования. Часть 2

Таблица 3.1 – Результаты нагрузочного тестирования эндпоинта рекомендаций

Показатель	Значение
Количество запросов	190
Среднее время отклика (Average), мс	2 889
Медиана (Median), мс	2 853
90-й перцентиль (90% Line), мс	3 477
95-й перцентиль (95% Line), мс	3 652
99-й перцентиль (99% Line), мс	3 759
Минимальное время, мс	443
Максимальное время, мс	3861
Процент ошибок (Error %)	4,74%
Пропускная способность (Throughput), запросов/с	3.9

Среднее время отклика страницы рекомендаций составило 2,89 секунды, медиана – 2,85 секунды. Значение 95-го перцентиля (95% Line) равно 3,65 секунды, что означает: 95% всех запросов к данному эндпоинту выполнялись не дольше 3,65 секунды. Доля ошибочных запросов (Error %) – 4,74%. Ошибки преимущественно были связаны с тайм-аутами при вызове Python-скрипта из-за временной недоступности интерпретатора при высокой параллельной нагрузке.

Важно отметить, что полученные значения удовлетворяют нефункциональным требованиям: время отклика API в условиях локального развёртывания не должно превышать 5 секунд. Фактическое среднее время (2,89 с) и даже 95-й перцентиль (3,65 с) находятся в допустимых пределах.

Проведённое тестирование подтвердило, что система справляется с одновременной работой 50 пользователей, а время отклика страницы рекомендаций удовлетворяет заявленным требованиям (менее 5 секунд). 95-й перцентиль составляет 3,65 секунды, что является хорошим показателем для учебного проекта, учитывая, что каждый запрос влечёт запуск отдельного Python-процесса. Для дальнейшего повышения производительности и снижения процента ошибок рекомендуется реализовать режим постоянно запущенного Python-сервера (например, на FastAPI) и добавить кэширование результатов для популярных пользователей. Тем не менее, в текущем виде система готова к опытной эксплуатации при умеренной нагрузке (до 20–30 параллельных пользователей).

3.6 Визуальное отображение модуля рекомендаций в веб-приложение

С использованием Thymeleaf, HTML5, CSS3 управление модулем рекомендаций было внедрено в веб-приложение. Был сохранён общий дизайн и сделан акцент на самом модуле за счёт цветовой палитры. Страницы веб-приложения представлены в Приложении А.

4 ИНФОРМАЦИОННЫЙ МЕНЕДЖМЕНТ

Информационный менеджмент представляет собой совокупность принципов, методов и подходов, направленных на управление информационными процессами. Он описывает информационное окружение (пространство) лица, принимающего решения, и его проблемную область. Информационный менеджмент включает в себя все аспекты управления в контексте создания и использования информационных ресурсов.

В данной главе рассматривается управление проектом [50, 51, 52] разработки и внедрения модуля персонализированных рекомендаций для информационной системы «Арт-галерея». Проект направлен на совершенствование существующей системы путём добавления интеллектуальной функции, повышающей вовлечённость пользователей. Управление проектом относится к задаче информационного менеджмента «экономика (управление капиталовложениями)» и включается разработку следующих вопросов:

- 1) обоснование проекта;
- 2) анализ окружения проекта;
- 3) основные положения устава проекта;
- 4) содержание проекта;
- 5) организационная структура проекта;
- 6) календарный план проекта в MS Project;
- 7) план управления рисками проекта;
- 8) план управления изменениями;
- 9) управленческая отчетность по проекту.

Участники проекта:

- 1) Шутова Т.Е. – разработчик информационной системы «Арт-галерея», ML-инженер в сфере проектирования и разработки алгоритмов на основе машинного обучения для рекомендательной системы «Арт-галереи», студентка

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		70

кафедры «Информационные системы и программная инженерия» института информационных технологий и электроники Владимирского государственного университета.

2) Жминьковская М.А. – разработчик информационной системы «Арт-галерея», ML-инженер в сфере внедрения нейросетевого модуля распознавания изображений для «Арт-галереи», студентка кафедры «Информационные системы и программная инженерия» института информационных технологий и электроники Владимирского государственного университета.

4.1 Обоснование проекта

Информационная система «Арт-галерея», разработанная для поддержки образовательного процесса и творческой деятельности кафедры дизайна, изобразительного искусства и реставрации ВлГУ, на момент начала проекта формировала ленту работ исключительно по принципу популярности (лайки, просмотры). Такой подход не учитывает индивидуальные интересы пользователей, приводит к быстрой потере внимания и не способствует продвижению новых авторов. Ключевые проблемы, требовавшие решения:

- низкая релевантность контента, демонстрируемого конкретному пользователю;
- отсутствие механизмов учёта долговременных интересов и художественных предпочтений;
- ограничение возможности для открытия малоизвестных, но потенциально интересных работ;
- информационная перегрузка пользователей из-за невозможности эффективного поиска в растущем каталоге.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		71

Первоначально предполагалось построение рекомендательной системы на основе нейронной сети. Однако в ходе предпроектного анализа и экспериментального сравнения подходов было установлено, что на начальном этапе эксплуатации системы объём накопленных данных (несколько десятков произведений и взаимодействий) недостаточен для обучения глубоких нейросетевых моделей, требующих тысяч примеров для достижения приемлемого качества. Кроме того, нейросетевые решения сложнее в интерпретации результатов и предъявляют повышенные требования к вычислительным ресурсам, что не соответствовало ограниченному бюджету проекта.

В качестве альтернативы был выбран гибридный подход на основе методов машинного обучения, объединяющий коллаборативную фильтрацию с сингулярным разложением (SVD) и контентную фильтрацию с TF-IDF векторизацией признаков произведений. Гибридная схема позволяет компенсировать недостатки каждого из методов: контентная часть решает проблему «холодного старта» для новых объектов, а коллаборативная — выявляет скрытые закономерности в поведении пользователей. Экспериментальная проверка на реальных данных подтвердила эффективность подхода: $Precision@5 = 0,257$, $Recall@5 = 1,000$, $Coverage = 0,167$ при ограниченном наборе взаимодействий.

Таким образом, проект направлен на разработку и интеграцию в существующую ИС «Арт-галерея» программного модуля, реализующего гибридный алгоритм рекомендаций на языке Python, с взаимодействием через механизм Java ProcessBuilder. Внедрение модуля следует рассматривать как совершенствование информационной системы, повышающее её функциональную полноту и пользовательскую ценность.

Бизнес-выгоды проекта представлены в матрице структурированных бизнес-выгод (таблица 4.1).

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		72

Таблица 4.1 – Матрица структурированных бизнес-выгод

		Характер воздействия на бизнес		
		Создание новых возможностей	Повышение эффективности операций	Отказ от операций
Степень определённости	Финансовые	Рост монетизации за счёт таргетированной рекламы и премиум-подписок	Снижение затрат на ручное продвижение работ	-
	Количественные	Увеличение числа уникальных просмотров на пользователя в 1,5 раза	Рост числа лайков и комментариев на 30%	-
	Измеримые	Повышение конверсии в подписки на авторов на 20%	Сокращение времени на поиск интересующего контента	-
	Качественные	Формирование лояльного сообщества вокруг платформы	Улучшение пользовательского опыта за счёт «умной» ленты	Отказ от ручного составления рекомендательных подборок

4.2 Анализ окружения проекта

Объектами автоматизации выступают процессы взаимодействия пользователей с контентом: просмотры, лайки, комментарии. Модуль рекомендаций предназначен для двух категорий пользователей: художников, загружающих произведения, и ценителей искусства, просматривающих и оценивающих работы. Администраторы кафедры ДИИР обеспечивают модерацию и пилотную эксплуатацию.

Внутренние факторы, влияющие на проект:

– команда проекта состоит из двух студенток: Шутовой Т.Е. (фронтенд, дизайн, модуль рекомендаций, руководство проектом) и Жминьковской М.А.

(бэкенд, база данных, модуль модерации); гибкость ролей и высокая мотивация компенсируют малочисленность;

- разработчики обладают взаимодополняющими компетенциями в областях веб-разработки и машинного обучения, что обеспечивает замкнутый цикл создания продукта;

- регулярные встречи и переписки с научным руководителем, доступ к Git проекта обеспечивают хорошую коммуникацию между участниками проекта;

- организационная структура плоская, решения принимаются совместно, что ускоряет итерации.

Внешние факторы:

- рынок и конкуренты; существуют крупные платформы (DeviantArt, ArtStation), но они не ориентированы на образовательную среду и локальное художественное сообщество; уникальность проекта – образовательная цель, доступность и удобство;

- технологические тренды; распространение Python-библиотек (scikit-learn, Surprise) и паттернов интеграции ML-моделей упрощают реализацию;

- регуляторные требования; необходимость соблюдения Федерального закона № 152-ФЗ «О персональных данных» при сборе и обработке журналов активности;

- социальный контекст; заинтересованность студентов и преподавателей в современных цифровых инструментах для демонстрации творческих работ.

Для визуализации влияния и интереса стейкхолдеров разработаны матрица Джонсона «Власть–интерес» (таблица 4.2) и матрица «Поддержка–сила влияния» (таблица 4.3).

Таблица 4.2 – Матрица «Власть-интерес»

	Низкий уровень интереса	Высокий уровень интереса
Высокий уровень власти	Администраторы кафедры (могут заблокировать доступ к инфраструктуре, но не глубоко вникают)	Разработчики, научный руководитель (ключевые лица, принимающие решения)
Низкий уровень власти	Случайные посетители (не влияют на проект)	Пользователи (художники, ценители) (заинтересованы, но не могут напрямую влиять на разработку)

Таблица 4.3 – Матрица «Поддержка-сила влияния»

	Низкий уровень влияния	Высокий уровень влияния
Высокий уровень поддержки	Пользователи, кафедра ДИИР	Разработчики, научный руководитель
Низкий уровень поддержки	Сторонние платформы	Технические сбои, конкуренты

Примечание: матрицы построены на основе экспертной оценки. Разработчики обладают наибольшим влиянием и поддержкой, что способствует успешной реализации. Пользователи хотя и имеют низкое влияние, но их поддержка критична для сбора данных и валидации результатов.

Таким образом, управление коммуникациями должно быть направлено на активное вовлечение научного руководителя и администраторов кафедры, а также на сбор обратной связи от пользователей через опросы и А/В-тестирование.

4.3 Основные положения устава проекта

В таблице 4.4 описаны основные положения устава проекта.

Таблица 4.4 – Устав проекта

Поле	Содержание
Название проекта	Разработка и внедрение модуля персонализированных рекомендаций на основе гибридных методов машинного обучения для ИС «Арт-галерея»
Цель проекта и решаемые задачи	Повышение релевантности контента; увеличение среднего времени сессии на 25 %; рост числа взаимодействий (лайков, комментариев) на 30 %
Результат проекта	1. Python-модуль рекомендаций (SVD + контентная фильтрация). 2. Интеграция модуля с Java-бэкендом через ProcessBuilder. 3. Обновлённый интерфейс ленты на веб-версии. 4. Техническая документация. 5. Отчёт о тестировании
Ограничения проекта	Используются только обезличенные данные внутри платформы. Время формирования персонализированных рекомендаций не более 5 с. Бюджет без закупки дорогостоящего GPU-оборудования
Допущения проекта	Существующая инфраструктура позволяет развернуть Python-скрипт без модернизации сервера. Пользователи продолжают активность, генерируя данные для дообучения
Расписание основных контрольных событий проекта	1. Начало: 01.12.2025. 2. Готовность данных: 31.12.2025. 3. Готовая модель: 28.02.2026. 4. Интеграция: 10.04.2026. 5. Опытная эксплуатация: 15.05.2026. 6. Завершение: 31.05.2026
Бюджет проекта	120 000 руб. (аренда облачных ресурсов, инфраструктурные расходы, резерв)
Критерии приемки результатов	$Precision@5 \geq 0,25$; $Recall@5 \geq 0,8$; время ответа API ≤ 300 мс; покрытие каталога $\geq 0,15$; отсутствие критических ошибок при нагрузке до 1000 запросов в минуту
Обоснование полезности проекта	Реализация проекта позволит превратить «Арт-галерею» из простого хранилища в интеллектуальную платформу, способную удерживать аудиторию. Для кафедры ДИИР это даст современный инструмент для демонстрации студенческих работ и вовлечения студентов и абитуриентов. Разработчики получают опыт создания готовой к внедрению ML-системы, что усилит их портфолио.

4.4 Содержание проекта

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		76

Учитывая исследовательский характер задачи, включающий подбор параметров модели и эксперименты с признаками, и необходимость частых итераций с обратной связью, проект выполняется по гибкой методологии SCRUM с двухнедельными спринтами. Это позволяет на каждом спринте получать работоспособный инкремент продукта и оперативно корректировать требования.

Выбор обусловлен следующими факторами:

- итеративность, которая позволяет на каждом спринте получать работающий инкремент продукта, демонстрировать его заказчику и оперативно корректировать требования;
- адаптивность, которая представляет из себя возможность быстро реагировать на результаты экспериментов: если одна архитектура модели показывает низкую точность, команда может переключиться на альтернативную без срыва плана;
- прозрачность; ежедневные демо в конце спринта обеспечивают постоянную обратную связь от научного руководителя и потенциальных пользователей;
- управление рисками; раннее выявление технических или организационных проблем позволяет своевременно принимать корректирующие методы.

Жизненный цикл проекта включает следующие фазы:

- подготовка и анализ данных – сбор событий, предобработка, формирование признаков;
- построение модели – реализация контентной (TF-IDF + косинусное сходство) и коллаборативной (SVD) частей, отладка гибридного объединения;
- интеграция – разработка Python-скрипта, вызов через ProcessBuilder из Java-приложения, доработка фронтенда;
- тестирование и внедрение – модульное, нагрузочное, А/В-тестирование, опытная эксплуатация, подготовка отчётности;
- управление проектом – поддержка функционирования веб-приложения.

На рисунке 4.1 представлена обобщённая схема жизненного цикла проекта.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		77

Для детализации содержания проекта применяется метод декомпозиции. Первый уровень иерархической структуры работы (ИСР) соответствует основным фазам жизненного цикла. Второй уровень детализирует работы внутри каждой фазы. Иерархическая структура работ проекта отображена на рисунке 4.2.

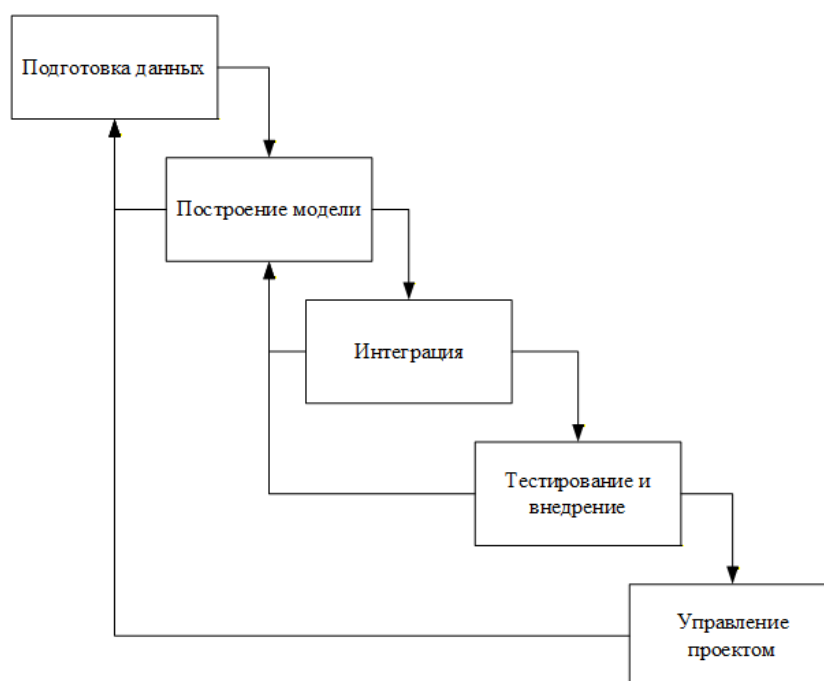


Рисунок 4.1 – Жизненный цикл проекта

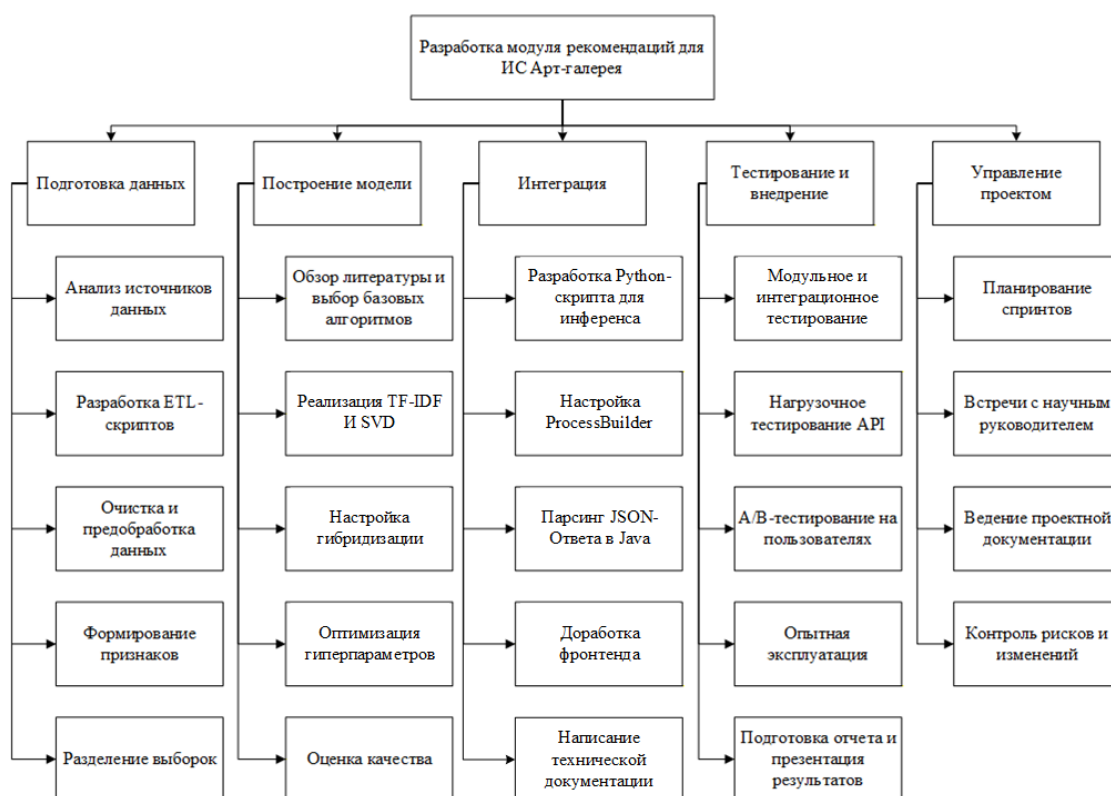


Рисунок 4.2 – Иерархическая структура работ

Словарь ИСР представлен в таблице 4.5.

Таблица 4.5 – Словарь ИСР

№	Этап	Состав работ	Трудоемкость (чел/час)	Результат
1	Подготовка данных	1.1 Анализ источников данных (логи БД) 1.2 Разработка ETL-скриптов 1.3 Очистка и предобработка данных 1.4 Формирование признаков 1.5 Разделение выборок	60	Готовый размеченный датасет в формате, пригодном для обучения моделей
2	Построение модели	2.1 Обзор литературы и выбор базовых алгоритмов 2.2 Реализация TF-IDF и SVD 2.3 Настройка гибридизации 2.4 Оптимизация гиперпараметров 2.5 Оценка качества (precision@k, recall@k)	80	Обученная модель в формате Python-скрипта
3	Интеграция	3.1 Разработка Python-скрипта для формирования персонализированных рекомендаций 3.2 Настройка ProcessBuilder 3.3 Парсинг JSON-ответа в Java 3.4 Доработка фронтенда 3.5 Написание технической документации	40	Работоспособный модуль, интегрированный в приложение

Продолжение таблицы 4.5

№	Этап	Состав работ	Трудоемкость (чел/час)	Результат
4	Тестирование и внедрение	4.1 Модульное и интеграционное тестирование 4.2 Нагрузочное тестирование API 4.3 А/В-тестирование на пользователях 4.4 Опытная эксплуатация 4.5 Подготовка отчета и презентация результатов	50	Отчет о тестировании, подтвержденные улучшения ключевых показателей, акт о внедрении
5	Управление проектом	5.1 Планирование спринтов 5.2 Встречи с научным руководителем 5.3 Ведение проектной документации 5.4 Контроль рисков и изменений	30	Актуальный план проекта, протоколы встреч, скорректированные планы при необходимости
Итого			260	Комплексная реализация модуля рекомендаций

4.5 Организационная структура проекта

Штатное расписание определяет потребность в трудовых ресурсах для реализации проекта. Поскольку проект выполняется в рамках выпускной квалификационной работы, фактическая оплата труда не предусмотрена. Однако для оценки полной стоимости проекта (как если бы работы выполнялись на коммерческой основе) и для формального распределения ролей в таблице 4.6 приведены условные тарифные ставки.

Таблица 4.6 – Штатное расписание проекта

№	Должность	Структурное подразделение	Количество штатных единиц	Тарифная ставка, руб/час
1	Руководитель проекта / ML-инженер (Шутова Т.Е.)	Кафедра ИСПИ	1	350
2	Бэкенд-разработчик / администратор БД (Жминьковская М.А.)	Кафедра ИСПИ	1	300
3	Тестировщик (совмещённая роль)	Кафедра ИСПИ	1 (совмещение)	200
4	Научный руководитель (Озерова М.И.)	Кафедра ИСПИ	1	0 (консультации)
5	Администраторы кафедры ДИИР (внешние)	Кафедра ДИИР	2 (по совместительству)	0 (тестирование)

Организационная структура проекта построена по матричному принципу: участники сохраняют формальную принадлежность к своим кафедрам, но внутри проекта подчиняются руководителю проекта. На рисунке 4.3 представлена диаграмма, отражающая состав команды и основные информационные потоки между ролями.

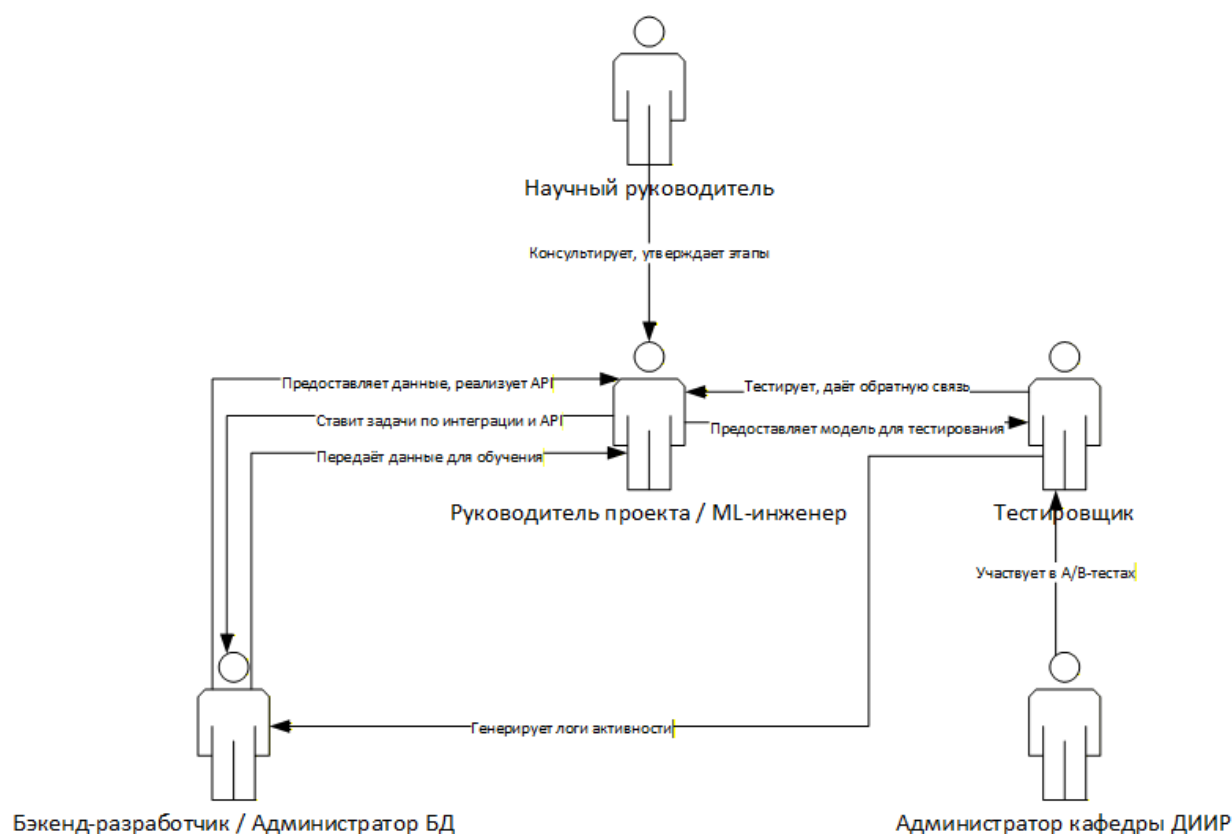


Рисунок 4.3 – Диаграмма организационной структуры

Описание взаимодействий:

– научный руководитель (Озерова М.И.) — стоит над руководителем проекта, оказывает консультационную поддержку и утверждает ключевые этапы; связь: двусторонняя консультация, утверждение;

– руководитель проекта / ML-инженер (Шутова Т.Е.) — центральная фигура, координирует все работы, разрабатывает модель, ставит задачи бэкенд-разработчику по интеграции и API, получает от него данные для обучения; также передаёт готовую модель тестировщику и получает от него результаты тестирования;

– бэкенд-разработчик / администратор БД (Жминьковская М.А.) — получает от руководителя задачи по интеграции и API, реализует их, передаёт руководителю данные для обучения (логи активности) и реализованный API; также взаимодействует с тестировщиком: передаёт ему доступ к тестовым стендам, зафиксированным событиям для анализа;

– тестировщик (совмещённая роль) — получает от руководителя модели для тестирования, проводит все виды тестов, включая А/В-тесты с участием администраторов кафедры ДИИР; результаты тестирования и выявленные ошибки направляет руководителю проекта; также генерирует журналы активности (совместно с бэкенд-разработчиком) и передаёт их бэкенд-разработчику для последующего использования при обучении модели;

– администраторы кафедры ДИИР — внешние участники, привлекаются для участия в А/В-тестировании, предоставляют обратную связь тестировщику; их роль ограничена этапом валидации результатов.

Такая структура обеспечивает гибкость, необходимую для исследовательского проекта, и чёткое разделение зон ответственности.

Для формального закрепления степени участия каждой роли в выполнении основных работ проекта используется матрица RACI. Расшифровка обозначений приведена в таблице 4.7.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		82

Таблица 4.7 – Условные обозначения матрицы RACI

Обозначение	Расшифровка	Описание
Исп. (R)	Исполнитель (Responsible)	Несет ответственность за непосредственное исполнение задачи. К каждой задаче должно быть приписано не менее одного исполнителя
Утв. (A)	Утверждающий (Accountable)	Отвечает за конечный результат перед вышестоящим руководством. На каждую работу должен быть назначен строго один подотчетный
Согл. (C)	Согласующий (Consulted)	Согласует принимаемые решения, взаимодействие с ним носит двусторонний характер
Н. (I)	Наблюдатель (Informed)	Его информируют об уже принятом решении, взаимодействие с ним носит односторонний характер

В таблице 4.8 представлена матрица RACI. Роли сокращены для удобства:

- РП/ML – руководитель проекта / ML-инженер (Шутова Т.Е.);
- БЭ – бэкенд-разработчик / администратор БД (Жминьковская М.А.);
- Т – тестировщик (совмещённая роль);
- НР – научный руководитель (Озерова М.И.).

Администраторы кафедры ДИИР не включены в матрицу как постоянные члены команды; их участие ограничено этапом тестирования и учтено через взаимодействие с тестировщиком.

Таблица 4.8 – Матрица ответственности RACI

Код работы	Наименование работы	РП/ML	БЭ	Т	НР
1. Подготовка данных					
1.1	Анализ источников данных (логи БД)	C	R	I	C
1.2	Разработка ETL-скриптов	A	R	I	I
1.3	Очистка и предобработка данных	A	R	I	C
1.4	Формирование признаков	R	C	I	C
1.4	Разделение выборок	R	C	I	I

Продолжение таблицы 4.8

Код работы	Наименование работы	РП/ML	БЭ	Т	НР
2. Построение модели					
2.1	Обзор литературы и выбор базовых алгоритмов	R	I	I	C
2.2	Реализация TF-IDF и SVD	R	I	I	C
2.3	Настройка гибридизации	R	I	I	C
2.4	Оптимизация гиперпараметров	R	I	I	C
2.5	Оценка качества (precision@k, recall@k)	R	I	I	A
3. Интеграция					
3.1	Создание Python-скрипта	R	R	I	I
3.2	Настройка ProcessBuilder	A	R	I	I
3.3	Парсинг JSON-ответа в Java	A	R	I	I
3.4	Доработка фронтенда	R	R	I	C
3.5	Написание технической документации	R	I	I	C
4. Тестирование и внедрение					
4.1	Модульное тестирование	A	C	R	I
4.2	Нагрузочное тестирование	A	C	R	I
4.3	A/B-тестирование на пользователях	A	I	R	C
4.4	Опытная эксплуатация	A	C	R	I
4.5	Подготовка отчёта и презентация результатов	A	I	R	C
5. Управление проектом					
5.1	Планирование спринтов	R	C	C	A
5.2	Встречи с научным руководителем	R	C	C	A
5.3	Ведение проектной документации	R	I	I	A
5.4	Контроль рисков и изменений	R	C	C	A

4.6 Календарный план проекта

Календарный план проекта разработан на основе иерархической структуры работ и контрольных событий, зафиксированных в уставе проекта. План определяет последовательность выполнения работ, их длительность и ответственных исполнителей. Для построения плана использовались следующие допущения:

- рабочая неделя – 5 дней (понедельник–пятница), рабочий день – 6 часов (учитывая совмещение с учебой);

- часть задач может выполняться параллельно (например, подготовка данных и разработка ETL-скриптов);
- вехи выделены отдельно и не имеют длительности.

На рисунке 4.4 изображена диаграмма Ганта, визуализирующая календарный план.

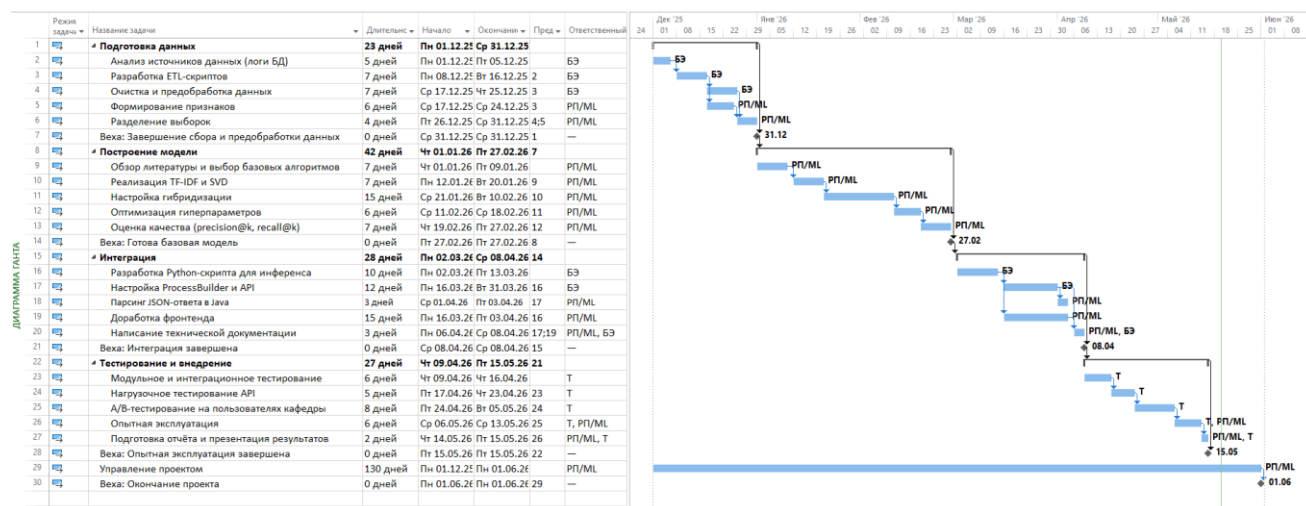


Рисунок 4.4 – Диаграмма Ганта

Длительность работ оценена исходя из трудоёмкости (чел.-ч) и возможности параллельной работы двух исполнителей. Например, подготовка данных (суммарно 60 чел.-ч) при 6-часовом рабочем дне требует 10 человеко-дней, но бэкенд-разработчик и ML-инженер могут работать параллельно, поэтому реальная длительность этапа – 30 дней.

Часть вех наступает раньше, чем указано в уставе, что создаёт временной резерв на случай непредвиденных задержек.

Управление проектом ведётся непрерывно, его задачи не привязаны к конкретным датам, но отражены в еженедельных встречах и планировании спринтов.

Планируется, что контроль исполнения календарного плана будет осуществляться руководителем проекта еженедельно. В случае отклонения более чем на 5 дней от графика проводится анализ причин и, при необходимости, инициируются изменения в соответствии с планом управления изменениями.

Расчет затрат на реализацию проекта представлен в виде сметы проекта. Смета сформирована с учетом штатного расписания и иерархическую структуры работ. Следует отметить, что труд разработчиков не требует прямого финансирования, поскольку проект выполняется в рамках выпускной квалификационной работы. Однако для оценки полной стоимости проекта в смете приведена условная стоимость трудозатрат (таблица 4.9).

Стоимость затрат с учётом оплаты работы специалистов составляет 142 898 рублей, что на 22 898 рублей дороже заложенного бюджета. Однако без учёта стоимости затрат на работу специалиста выходит 59 898 рублей, что в 2 раза дешевле заложенного бюджета.

Таблица 4.9 – Смета проекта

Стоимость работ:			
Категория специалиста	Трудозатраты, час	Ставка, руб/час	Итого
Руководитель проекта/ML-инженер (Шутова Т.Е.)	140	350	49 000
Бэкенд-разработчик (Жминьковская М.А.)	100	300	30 000
Тестировщик (совмещенная роль)	20	200	4 000
Итого	260		83 000
Стоимость оборудования:			
Категория расходов	Количество	Стоимость за единицу, руб	Итого
Аренда облачного сервера с GPU	4 месяца	5 000	20 000
Инфраструктурные расходы:			
Категория расходов	Количество	Стоимость за единицу, руб	Итого
Хостинг и дополнительное дисковое пространство	6 месяцев	3 333	19 998
Интернет	6 месяцев	1 650	9 900
Итого			29 898
Резерв:			
Непредвиденные расходы			10 000
Итого			142 898
Итого без оплаты специалистов			59 898

4.7 План управления рисками

Для количественной оценки рисков используются шкалы вероятности возникновения и степени влияния (последствий). Шкалы разработаны с учётом специфики проекта (продолжительность около 6 месяцев, бюджет 120 тыс. руб.) и описаны в таблицах 4.10 и 4.11.

Таблица 4.10 – Шкала оценки вероятности возникновения рисков

Интервал вероятностей	Словесная формулировка	Числовая оценка
До 10%	Очень низкая	1
11% – 30%	Низкая	2
31% – 50%	Средняя	3
51% – 70%	Высокая	4
71% – 100%	Очень высокая	5

Таблица 4.11 – Шкала оценки последствий (угроз)

Перерасход средств	Отставание от расписания	Описание	Числовая оценка
До 10%	До 2 недель	Незначительные последствия	1
11% – 25%	2 – 4 недели	Умеренные последствия	2
Более 25%	Более 1 месяца	Критические последствия	3

Величина риска определяется как произведение числовой оценки вероятности и оценки последствий. Диапазон значений от 1 до 15 позволяет ранжировать риски по степени опасности (таблица 4.12).

Таблица 4.12 – Матрица оценки риска

Вероятность \ Последствия	1 (незначительные)	2 (умеренные)	3 (критические)
1 (очень низкая)	1	2	3
2 (низкая)	2	4	6
3 (средняя)	3	6	9
4 (высокая)	4	8	12
5 (очень высокая)	5	10	15

Для удобства анализа все риски проекта сгруппированы по следующим категориям:

- технологические (Т) – связаны с используемыми технологиями, инструментами, сложностью реализации;
- организационные (О) – обусловлены человеческим фактором, распределением ролей, коммуникациями;
- внешние (В) – риски, вызванные факторами, неподконтрольными команде (изменение законодательства, сбои внешних сервисов);
- управленческие (У) – связаны с процессами планирования, контроля, принятия решений;
- ресурсные (Р) – недостаток финансирования, оборудования, доступа к данным.

В таблице 4.13 представлен реестр идентифицированных рисков. Для каждого риска определены: вероятность, категория, последствия, ранг (величина), стратегия реагирования, временная близость, триггеры и владелец.

Для таблицы используются следующие сокращения:

а) для столбца «Вероятность»:

- 1) «1» – очень низкая;
- 2) «2» – низкая;
- 3) «3» – средняя;
- 4) «4» – высокая;

б) для столбца «Стратегия»:

- 1) «С» – снижение;
- 2) «ПН» – принятие;
- 3) «ПД» – предотвращение;
- 4) «М» – мониторинг;

в) для столбца «Близость»:

- 1) «С» – скоро;
- 2) «НС» – нескоро;

г) для столбца «Тенденция»:

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		88

- 1) «+» – увеличивается;
- 2) «-» – снижается;
- 3) «=» – стабильно.

Таблица 4.13 – Реестр идентифицированных рисков

№ риска	Название	Описание	Вероятность	Категория	Последствия (ранг)	Стратегия	Близость	Триггеры	Тенденция	Владелец риска
R01	Недостаточное количество данных для обучения модели	Накоплено мало журналов активности пользователей, модель не может обучиться качественно	4	T	12	C	C	Медленный рост числа пользователей, низкая активность	+	РП/ML
R02	Низкая точность модели (недостижение целевых метрик)	Выбранная архитектура или признаки не обеспечивают $\text{precision}@k \geq 0.7$	3	T	6	ПН	C	Метрики на валидации ниже ожидаемых	=	РП/ML
R03	Высокая нагрузка на сервер при формировании рекомендаций	Модель требует больших вычислительных ресурсов, что приводит к замедлению работы API	3	T	6	C	НС	Рост числа пользователей, увеличение частоты запросов	+	БЭ
R04	Проблемы совместимости API с существующим бэкендом	Различия в версиях библиотек, форматах данных, протоколах	2	T	4	ПД	C	Ошибки при интеграции на тестовом стенде	-	БЭ

Продолжение таблицы 4.13

№ риска	Название	Описание	Вероятность	Категория	Последствия (ранг)	Стратегия	Близость	Триггеры	Тенденция	Владелец риска
R05	Отсутствие обратной связи от пользователей при А/В-тестировании	Пользователи кафедры ДИИР неактивно участвуют в тестировании, недостаточно данных для статистической значимости	3	О	6	ПН	С	Низкий отклик на приглашения, мало оценок	=	Т
R06	Изменение приоритетов в ВКР или требований кафедры	Научный руководитель или заказчик могут потребовать изменения функ-ти	3	У	6	М	НС	Новые указания от руководителя, корректировка темы	=	РП/ML
R07	Временная нетрудосп-сть одного из разработчиков	Болезнь или другие личные обстоятельства, приводящие к простоям	2	О	6	ПН	НС	Отсутствие на встречах, срыв дедлайнов	=	РП/ML
R08	Перерасход бюджета на аренду облачного хранилища	Обучение модели требует больше ресурсов или времени, чем планировалось	2	Р	6	С	НС	Увеличение числа экспериментов, необходимость более мощных виртуальных серверов	+	РП/ML
R09	Недоступность облачных сервисов или хостинга	Сбои у провайдера, потеря доступа к серверу	1	В	3	ПД	НС	Ошибки подключения, алерты от провайдера	=	БЭ
R10	Утечка персональных данных пользователей	Нарушение безопасности при сборе журналов событий или хранении данных	1	В	3	ПД	НС	Инциденты безопасности, жалобы пользователей	-	БЭ

Продолжение таблицы 4.13

№ риска	Название	Описание	Вероятность	Категория	Последствия (ранг)	Стратегия	Близость	Триггеры	Тенденция	Владелец риска
R11	Неправильная оценка трудоёмкости этапов	Реальные затраты времени превышают плановые, срыв сроков	3	У	6	М	С	Отставание от календарного плана	+	РП/ML
R12	Конфликты в расписании участников (учеба, работа)	Невозможность выделить достаточное время на проект из-за сессии или других обязательств	4	О	8	ПН	С	Снижение активности в чатах, перенос встреч	+	РП/ML

На основе полученных рангов риски можно разделить на три группы:

- критические (ранг 9–15): R01 (недостаток данных) – требуют немедленных мер;
- значимые (ранг 6–8): R02, R03, R05, R06, R07, R08, R11, R12 – требуют разработки планов реагирования;
- незначительные (ранг 1–5): R04, R09, R10 – достаточно мониторинга и базовых мер предосторожности.

Группировка по категориям показывает, что большинство рисков относятся к технологической и организационной сферам, что характерно для исследовательских проектов с малой командой.

Для каждого риска с рангом 6 и выше разработаны предупреждающие или корректирующие мероприятия. В таблице 4.14 приведён план действий с указанием ответственных и сроков.

Таблица 4.14 – План мероприятий по реагированию на риски

№ риска	Мероприятие	Ответственный	Срок (контрольная точка)
R01	Использовать методы искусственного расширения набора данных, добавить синтетические взаимодействия, применить подходы холодного старта (векторные представления изображений). При нехватке данных — привлечь пользователей кафедры для ручного оценивания работ.	РП/ML	До 15.01.2026
R02	Провести анализ ошибок модели, добавить новые признаки (контент изображений через CNN), рассмотреть ансамблирование моделей. Предусмотреть запасной вариант — гибридная рекомендация (популярное + коллаборативная фильтрация).	РП/ML	Постоянно, контроль на каждой итерации
R03	Оптимизировать модель, внедрить кэширование результатов для популярных запросов, использовать асинхронные задачи. При необходимости увеличить мощность сервера.	БЭ	До 01.04.2026
R05	Активно мотивировать участников А/В-теста (бонусы, упоминания), расширить группу тестирования за счёт студентов других кафедр. Провести опрос для сбора качественных отзывов.	Т	Апрель 2026
R06	Регулярные встречи с научным руководителем для синхронизации ожиданий, фиксация требований в документах. При значительных изменениях — инициировать запрос на изменение.	РП/ML	Еженедельно
R07	Обеспечить кросс-функциональность (оба разработчика владеют смежными областями), вести документацию, чтобы при временном отсутствии одного второй мог продолжить ключевые работы.	РП/ML	В течение проекта
R08	Заранее протестировать обучение на небольших данных для оценки необходимого времени, заложить бюджетный резерв, при необходимости использовать менее дорогие виртуальные серверы (с оптимизацией).	РП/ML	До начала экспериментов (январь 2026)

Продолжение таблицы 4.14

№ риска	Мероприятие	Ответственный	Срок (контрольная точка)
R11	Вести ежедневный учёт фактических трудозатрат, сравнивать с планом, при отклонениях более 10% — пересматривать оценку оставшихся работ и корректировать план спринта.	РП/ML	Еженедельно
R12	Согласовать с участниками фиксированные «окна» для работы над проектом, минимизировать влияние сессии, планировать основные этапы на межсессионный период.	РП/ML	До начала проекта и далее по необходимости

Для рисков R04, R09, R10 достаточно регулярного мониторинга и соблюдения стандартных практик (резервное копирование, контроль версий, безопасность).

Выполнение плана мероприятий контролируется руководителем проекта на еженедельных встречах. В случае наступления триггера риска инициируются экстренные действия согласно описанным стратегиям.

4.8 План управления изменениями

Все запросы на изменение оформляются по единому шаблону, приведённому в таблице 4.15. Заявка регистрируется в журнале изменений.

Таблица 4.15 – Типовая форма заявки на изменение

Поле	Описание	Пример заполнения
ID	Уникальный идентификатор заявки (формируется автоматически при регистрации)	ИЗМ-2026-001
Дата подачи	Дата заполнения заявки	15.03.2026
Автор	ФИО и роль инициатора изменения	Шутова Т.Е., руководитель проекта

Продолжение таблицы 4.15

Поле	Описание	Пример заполнения
Описание изменения	Краткое описание сути предлагаемого изменения	Добавить в модель признаки на основе содержимого изображений (использовать предобученный ResNet)
Причина	Обоснование необходимости изменения	Текущие поведенческие признаки дают низкую точность для новых пользователей (холодный старт)
Влияние на сроки	Оценка дополнительного времени, требуемого для реализации	+5 рабочих дней к этапу построения модели
Влияние на бюджет	Оценка дополнительных расходов (если применимо)	+2000 руб. на аренду GPU для дообучения
Влияние на содержание	Описание того, как изменится состав работ или результаты	Дополнительные эксперименты с CNN, расширение раздела документации
Приоритет	Высокий / Средний / Низкий	Средний
Решение	Утверждено / Отклонено / Отложено	Утверждено
Подпись утверждающего	ФИО и должность лица, принявшего решение	Озерова М.И., научный руководитель

Процесс управления изменениями реализуется по следующему алгоритму. На рисунке 4.5 представлена диаграмма процесса в нотации простой блок-схемы. Ниже приведено текстовое описание каждого шага с указанием ответственных ролей.

Заполнение заявки на изменение:

- автор (любой участник проекта) заполняет типовую форму (таблица 16), описывая суть, причину и предполагаемое влияние;
- результат – заполненная заявка.

Регистрация заявки:

- руководитель проекта (РП) присваивает заявке уникальный номер, заносит её в журнал изменений со статусом «Новая»;
- результат: зарегистрированная заявка.

Анализ влияния:

- РП совместно с экспертами (бэкенд-разработчик БЭ, тестировщик Т) оценивает влияние изменения на сроки, бюджет, содержание и риски. Готовится краткое заключение;

- результат: заключение по анализу.

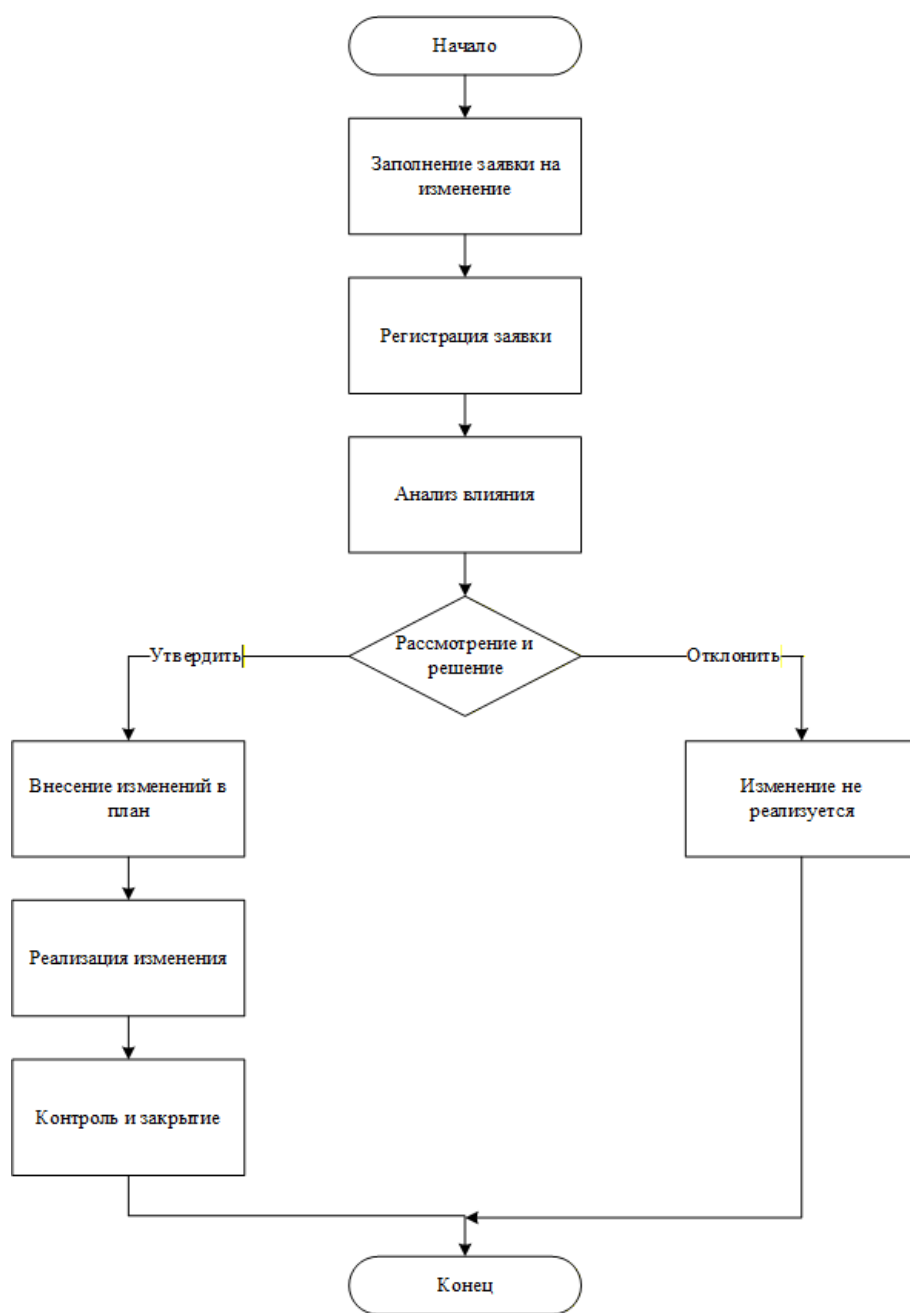


Рисунок 4.5 – Модель процесса управления изменениями

Рассмотрение и принятие решения:

- научный руководитель (НР) рассматривает заявку и заключение, принимает решение:
 - утвердить — изменение принимается к реализации;
 - отклонить — изменение не реализуется (с указанием причины);

- решение фиксируется в заявке и журнале.

Внесение изменений в план проекта:

- в случае утверждения РП актуализирует календарный план, ИСР, бюджет (если нужно) и уведомляет команду;
- результат: обновлённые документы проекта.

Реализация изменения:

- назначенный исполнитель (согласно матрице RACI) выполняет необходимые работы;
- результат: реализованное изменение.

Контроль и закрытие изменения:

- РП проверяет результат, при необходимости организует тестирование. Если всё выполнено корректно, заявка закрывается (статус «Реализовано»), информация доводится до всех заинтересованных сторон;
- результат: закрытая заявка, актуальная документация.

4.9 Управленческая отчетность по проекту

Для обеспечения прозрачности хода проекта, своевременного выявления отклонений и информирования заинтересованных сторон разработана система управленческой отчетности. Она включает набор ключевых показателей эффективности (KPI), регламент подготовки отчетов и схему информационных потоков между участниками проекта.

В таблице 4.16 представлены основные показатели, используемые для контроля проекта. Для каждого показателя указаны целевое значение, метод сбора и периодичность мониторинга.

					ВЛГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		96

Таблица 4.16 – Ключевые показатели эффективности проекта

Группа	Показатель	Целевое значение	Метод сбора / расчёта	Периодичность	Ответственный
Метрики модели	Precision@10	$\geq 0,7$	Расчёт на тестовой выборке после каждого обучения	Еженедельно на этапе 2	РП/ML
	Recall@10	$\geq 0,6$	Расчёт на тестовой выборке	Еженедельно на этапе 2	РП/ML
Бизнес-метрики	Среднее время сессии	+25% от базового	Сравнение журналов событий до/после А/В-теста	По итогам А/В-теста	Т
	Количество лайков на пользователя	30%	Анализ журнала событий БД	По итогам А/В-теста	Т
Технические метрики	Время ответа API (p95)	≤ 300 мс	Нагрузочное тестирование	После интеграции и при изменениях	БЭ
	Загрузка CPU/GPU	$\leq 80\%$ в пике	Мониторинг сервера	Ежедневно	БЭ
	Процент успешных запросов	$\geq 99,5\%$	Логи сервера	Ежедневно	БЭ
Метрики выполнения плана	Отклонение от календарного плана	≤ 5 дней	Сравнение фактических дат с плановыми	Еженедельно	РП/ML
	Процент выполненных задач спринта	$\geq 80\%$	Трекинг задач	В конце каждого спринта	РП/ML
	Отклонение бюджета	$\leq 10\%$	Учёт фактических расходов	Ежемесячно	РП/ML

В таблице 4.17 определены виды отчётов, их периодичность, ответственные за подготовку и получатели.

Таблица 4.17 – Регламент подготовки отчетности

Вид отчета	Периодичность	Ответственный	Получатели
Статус-отчет (выполненные задачи, планы, проблемы)	Еженедельно (понедельник)	РП/ML	Научный руководитель, команда
Отчет по спринту (результаты, метрики модели, демо)	Раз в 2 недели (по окончании спринта)	РП/ML	Научный руководитель
Отчет по рискам (актуальные риски, статус мероприятий)	Еженедельно	РП/ML	Команда проекта
Отчет по качеству модели (precision, recall, MAP)	После каждой итерации обучения	РП/ML	Научный руководитель
Финансовый отчет (фактические расходы, сравнение с бюджетом)	Ежемесячно	РП/ML	Научный руководитель
Отчет о тестировании (результаты нагрузочного, A/B-теста)	По завершении этапа 4	T	Научный руководитель, команда
Итоговый отчет по проекту	По окончании проекта	РП/ML	Научный руководитель, кафедра ДИИР

На рисунке 4.6 представлена обобщённая схема потоков отчетности и взаимодействия инструментов, используемых для сбора и визуализации данных.

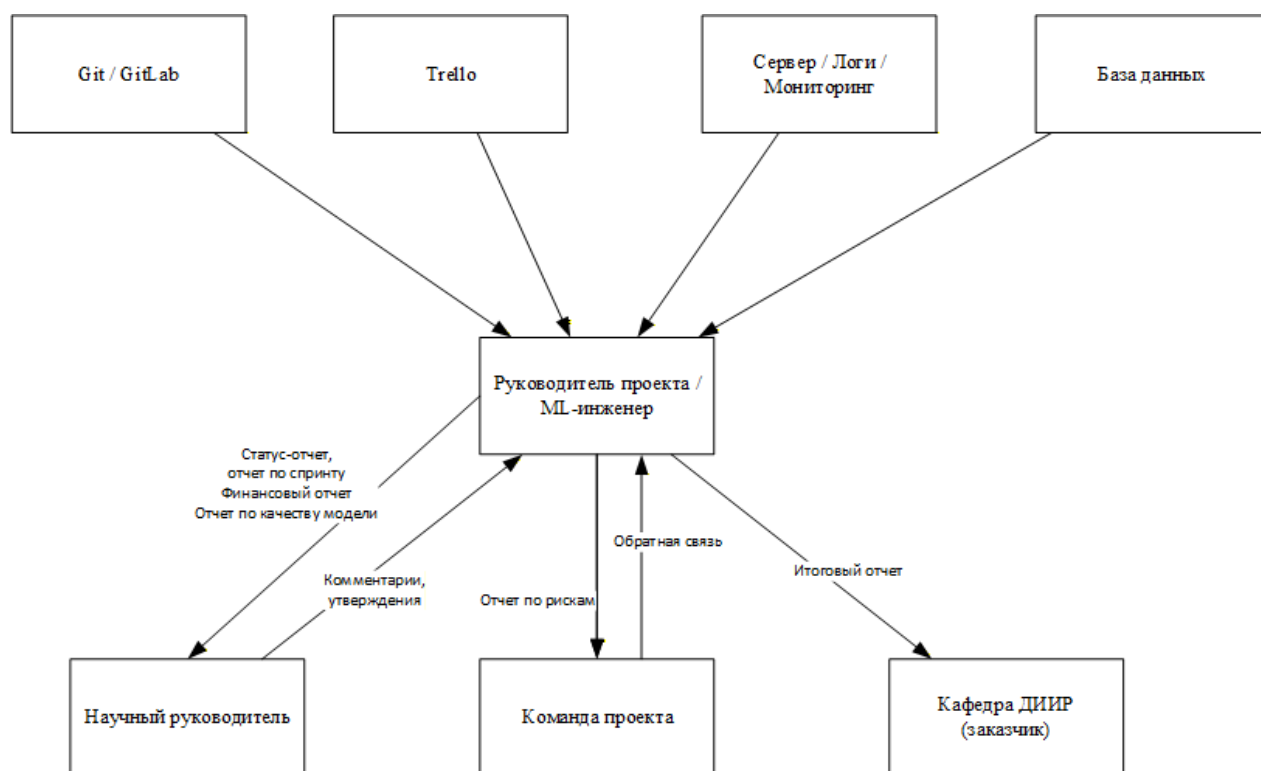


Рисунок 4.6 – Схема управленческой отчетности проекта

Для автоматизации сбора показателей и ведения отчетности используются следующие инструменты:

- трекинг задач: Trello или YouTrack (доска с колонками «To do», «In progress», «Done»);
- контроль версий: Git (GitHub) — для отслеживания вклада и версий кода;
- мониторинг сервера: Prometheus + Grafana (дашборд с графиками нагрузки, времени ответа);
- логирование: Sentry (для отслеживания ошибок в реальном времени);
- хранение отчётов: Google Drive / папка проекта с доступом для научного руководителя.

Предложенная система отчетности обеспечит прозрачность проекта для всех участников и позволит своевременно реагировать на отклонения от плана.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		99

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была достигнута поставленная цель: спроектирован, разработан и внедрён модуль персонализированных рекомендаций для информационной системы «Арт-галерея», а также модуль поиска визуально похожих произведений. Реализованные алгоритмы и архитектурные решения позволяют повысить вовлечённость пользователей за счёт адаптации контента под индивидуальные предпочтения и предоставления релевантных визуальных аналогий.

Проведён анализ предметной области и существующих платформ (DeviantArt, ArtStation, Pinterest), выявивший отсутствие на рынке гибридного подхода, сочетающего коллаборативную фильтрацию с анализом визуального содержания изображений. Обоснована актуальность разработки собственного модуля рекомендаций.

Спроектирована и реализована математическая модель персонализированных рекомендаций, основанная на гибридном подходе. В её основе лежит объединение коллаборативной фильтрации на основе сингулярного разложения (SVD) с собственной реализацией стохастического градиентного спуска и контентной фильтрации на основе TF-IDF и косинусной меры сходства для текстового описания «автор + категории». Разработана и обоснована формула гибридной оценки с нормализацией.

Спроектирована и реализована математическая модель поиска визуально похожих произведений с использованием нейросетевой модели OpenCLIP (ViT-B-32). Признаки OpenCLIP дополнены низкоуровневыми статистиками (цветовая гистограмма, средние значения каналов), на основе которых вычисляется интегральная мера визуального сходства.

Разработаны и протестированы вычислительные модули на языке Python: `model_trainer.py` (асинхронное переобучение модели), `recommendation_engine.py`

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		100

(получение персонализированных рекомендаций с выдачей топ-50 в формате JSON) и `openclip_extractor.py` (извлечение визуальных признаков).

Осуществлена интеграция Python-модулей с основным веб-приложением на Java Spring MVC через вызов внешних процессов (`ProcessBuilder`) с передачей данных в формате JSON. Реализован механизм рандомизированной выдачи рекомендаций (выбор 12 случайных работ из топ-50 при каждом обновлении страницы для повышения разнообразия) и асинхронное извлечение признаков OpenCLIP для минимизации времени отклика.

Проведена комплексная оценка качества и эффективности разработанной системы. Метрики качества модели на тестовой выборке (54 произведения, 43 взаимодействия) составили $\text{Precision@5} = 0.275$, $\text{Recall@5} = 1.000$, $\text{Coverage} = 0.167$, что соответствует ожиданиям для начального этапа эксплуатации. Нагрузочное тестирование (JMeter, 50 параллельных пользователей) подтвердило соответствие требованиям по производительности: среднее время отклика страницы рекомендаций составило 2,89 секунды, значение 95-го перцентиля — 3,65 секунды при максимально допустимых 5 секундах.

В ходе работы были успешно использованы современные технологии и фреймворки: Java 17, Spring MVC, Spring Security, Hibernate, MySQL 8.0, MinIO, Thymeleaf; Python 3.11, библиотеки `pandas`, `numpy`, `scikit-learn`, `open_clip_torch`, `torch`. Разработанные модули интегрированы в единое информационное пространство, предоставляя пользователям персонализированную ленту и блок визуально похожих работ на странице деталей.

Практическая ценность работы заключается в создании готового к внедрению программного продукта, который может быть использован арт-пространствами, галереями и образовательными учреждениями для расширения аудитории и представления художественных произведений в цифровой среде. Гибридный подход к рекомендациям и использование OpenCLIP для визуального поиска представляют собой воспроизводимый опыт, применимый в смежных проектах.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		101

Личный вклад автора состоит в анализе предметной области и существующих решений, проектировании и реализации описанных математических моделей, разработке всех компонентов модуля рекомендаций (Python и Java) и их интеграции, проведении экспериментальных исследований и анализе полученных результатов.

Направления дальнейшего развития системы включают: переход от запуска отдельных Python-процессов к режиму постоянно работающего Python-сервера (например, на FastAPI) для повышения производительности; введение кэширования результатов для популярных пользователей; расширение набора семантических меток для OpenCLIP и улучшение интегральной метрики визуального сходства; внедрение A/B-тестирования для сравнения эффективности различных стратегий рекомендаций.

Таким образом, все задачи, поставленные в начале работы, выполнены в полном объёме. Разработанная система демонстрирует свою работоспособность и потенциал для дальнейшего развития.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		102

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. Алгоритм рекомендательной системы с учетом культурных факторов для решения проблем холодного старта / А. И. Сухоруков, А. С. Старостин, А. В. Медведев [и др.] // Computational Nanotechnology. – 2025. – Т. 12, № 1. – С. 48–58.
2. Бабичев, Д. А. Холодный запуск в системах рекомендаций в области образования : классические средства и стратегии LLM эпохи / Д. А. Бабичев, А. Н. Григорьев // Научно-аналитический журнал «Наука и мир». – 2026. – № 3. – URL: <https://journals.nauka-nanrk.kz/physics-mathematics/article/view/8148> (дата обращения: 15.05.2026).
3. Evaluation Metrics for Search and Recommendation Systems : официальная документация. – Weaviate, 2024. – URL: <https://weaviate.io/blog/retrieval-evaluation-metrics> (дата обращения: 20.05.2026).
4. Precision and recall at K in ranking and recommendations : техническая документация. – Evidently AI, 2025. – URL: <https://www.evidentlyai.com/ranking-metrics/precision-recall-at-k> (дата обращения: 20.05.2026).
5. Капралова, В. С. Метрики оценки для рекомендательных систем / В. С. Капралова // Habr. – 2023. – URL: <https://habr.com/ru/companies/otus/articles/732842/> (дата обращения: 20.05.2026).
6. Symeonidis, P. Matrix and Tensor Factorization Techniques for Recommender Systems / P. Symeonidis, A. Zioupos. – Springer, 2017. – 152 p. – ISBN 978-3-319-49637-9.
7. Topic Modeling Tutorial – How to Use SVD and NMF in Python : руководство. – freeCodeCamp, 2023. – URL: <https://www.freecodecamp.org/news/advanced-topic-modeling-how-to-use-svd-nmf-in-python/> (дата обращения: 21.05.2026).

8. OpenCLIP : архитектура и компоненты : официальная документация / mlfoundations. – DeepWiki, 2026. – URL: https://deepwiki.com/mlfoundations/open_clip (дата обращения: 15.03.2026).

9. Srivastava, M. M. RetailKLIP : Finetuning OpenCLIP Backbone Using Metric Learning on a Single GPU for Zero-Shot Retail Product Image Classification / M. M. Srivastava. – arXiv, 2024. – URL: <https://arxiv.org/abs/2312.10282> (дата обращения: 22.05.2026).

10. Speth, C. SWOT-анализ : Важный инструмент для разработки бизнес-стратегий / С. Speth. – 2023. – 50 с. – ISBN 978-2-8086-0137-5.

11. Дженстер, П. Анализ сильных и слабых сторон компании. Определение стратегических возможностей / П. Дженстер, Д. Хасси. – М. : Интел-Синтез, 2021. – 368 с. – ISBN 978-5-00100-000-0.

12. DeviantArt Developers : официальная документация API. – DeviantArt, 2020. – URL: <https://www.deviantart.com/developers/> (дата обращения: 20.03.2026).

13. Pinterest API v5 : официальная документация разработчика. – Pinterest, 2025. – URL: <https://developers.pinterest.com/> (дата обращения: 20.03.2026).

14. ArtStation : информация о платформе. – URL: <https://www.artstation.com> (дата обращения: 20.03.2026).

15. Class ProcessBuilder : официальная документация Java Platform SE / Oracle. – 2026. – URL: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/ProcessBuilder.html> (дата обращения: 25.03.2026).

16. Apache Tomcat 9 : официальная документация и руководство пользователя. – Apache Software Foundation, 2024. – URL: <https://tomcat.apache.org/tomcat-9.0-doc/index.html> (дата обращения: 26.03.2026).

17. Ньюмен, С. От монолита к микросервисам / С. Ньюмен. – СПб. : БХВ-Петербург, 2021. – 304 с. – ISBN 978-5-9775-6723-7.

18. Ричардсон, К. Микросервисы. Паттерны разработки и рефакторинга / К. Ричардсон. – СПб. : Питер, 2022. – 544 с. – ISBN 978-5-4461-3000-9.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		104

19. MySQL 8.0 Reference Manual : официальное справочное руководство. – Oracle, 2026. – URL: <https://dev.mysql.com/doc/refman/8.0/en/> (дата обращения: 29.03.2026).

20. PostgreSQL 14 : официальная документация. – PostgreSQL Global Development Group, 2021. – URL: <https://www.postgresql.org/docs/14/> (дата обращения: 29.03.2026).

21. Scikit-learn : официальная документация и руководство пользователя. – scikit-learn developers, 2025. – URL: <https://scikit-learn.org/stable/> (дата обращения: 01.04.2026).

22. Scikit-learn : библиотека для машинного обучения на Python // Skillfactory. – 2022. – URL: <https://blog.skillfactory.ru/glossary/scikit-learn/> (дата обращения: 01.04.2026).

23. Савельев, А. О. Оценка сходства наборов слабоструктурированных данных на базе косинусного сходства и TF-IDF / А. О. Савельев, С. А. Кузнецов // Труды Томского политехнического университета. – 2021. – С. 334–335.

24. Применение метода TF-IDF и косинусной меры сходства для выявления дубликатов текстов // Research-Journal.org. – 2025. – URL: <https://research-journal.org/media/articles/23529.pdf> (дата обращения: 07.04.2026).

25. Surprise : A Python library for recommender systems : официальная документация. – GitHub, 2024. – URL: <http://surpriselib.com> (дата обращения: 09.04.2026).

26. Comparison Models of Machine Learning for Movie Recommendation Systems // International Journal of Advanced Computer Science and Applications. – 2021. – URL: <https://journals.nauka-nanrk.kz/physics-mathematics/article/view/264> (дата обращения: 10.04.2026).

27. pandas.DataFrame : официальная документация. – pandas development team, 2026. – URL: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html> (дата обращения: 17.04.2026).

28. Модуль pandas // Yandex Handbook. – 2026. – URL: <https://education.yandex.ru/handbook/python/article/modul-pandas> (дата обращения: 17.04.2026).

29. NumPy : официальная документация. – NumPy developers, 2025. – URL: <https://numpy.org/doc/stable/> (дата обращения: 18.04.2026).

30. Нескучный tutorial по NumPy // Habr. – 2025. – URL: <https://habr.com/ru/articles/469355/> (дата обращения: 18.04.2026).

31. OpenCLIP GitHub repository : исходный код и документация / mlfoundations. – GitHub, 2026. – URL: https://github.com/mlfoundations/open_clip (дата обращения: 19.04.2026).

32. Using OpenCLIP at Hugging Face : руководство по использованию модели. – Hugging Face. – URL: https://huggingface.co/docs/hub/en/open_clip (дата обращения: 19.04.2026).

33. PyTorch : официальная документация. – Meta AI, 2026. – URL: <https://pytorch.org/docs/stable/> (дата обращения: 20.04.2026).

34. Начало работы с PyTorch : создание и обучение нейронных сетей на Python // Hexlet. – 2024. – URL: <https://ru.hexlet.io/blog/posts/nachalo-raboty-s-pytorch-sozдание-i-obuchenie-neyronnyh-setey-na-python> (дата обращения: 28.04.2026).

35. PlantUML Language Reference Guide : официальная документация. – PlantUML, 2025. – URL: <https://plantuml.com/ru/> (дата обращения: 01.05.2026).

36. MinIO Object Storage for Kubernetes : официальная документация. – MinIO, 2026. – URL: <https://docs.min.io/> (дата обращения: 05.05.2026).

37. MinIO Client : документация по работе с S3-совместимым хранилищем // Selectel. – 2026. – URL: <https://docs.selectel.ru/cloud/storage/s3/minio-client/> (дата обращения: 05.05.2026).

38. Deployment Diagram Tutorial / Visual Paradigm. – 2026. – URL: <https://online.visual-paradigm.com/diagrams/tutorials/deployment-diagram-tutorial/> (дата обращения: 06.05.2026).

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		106

39. Основы проектирования реляционных баз данных. Диаграммы «сущность-связь» // НОУ ИНТУИТ. – 2007. – URL: <https://intuit.ru/studies/courses/1095/191/lecture/4969?page=3> (дата обращения: 10.05.2026).

40. 3-я нормальная форма (3НФ) // StudFiles. – 2016. – URL: <https://studfile.net/preview/3644671/page:3/> (дата обращения: 10.05.2026).

41. Tom, S. UX design with Figma : user-centered interface design and prototyping with Figma / S. Tom. – Apress, 2024. – 315 p. – ISBN 978-1-4842-9360-9.

42. Проектирование информационных систем. Диаграммы использования (Use Case Diagram) // НОУ ИНТУИТ. – 2005. – URL: <https://intuit.ru/studies/courses/2195/55/lecture/1638?page=3> (дата обращения: 14.05.2026).

43. Борзин, Р. Ю. Язык моделирования UML. Построение диаграмм классов : методические указания / Р. Ю. Борзин. – Волжский : ВолГГУ, 2024. – 12 с.

44. Ананьева, В. Я. Описание поведения информационной системы с использованием UML-диаграмм : учебно-методическое пособие / В. Я. Ананьева. – СПб. : Изд-во СПбГЭТУ «ЛЭТИ», 2025. – 28 с. – ISBN 978-5-7629-3440-4.

45. Сравнительный анализ математических алгоритмов для построения рекомендательных систем // Сибирский федеральный университет. – 2024. – URL: <https://scholar.sfu.ru/publication/75110137> (дата обращения: 14.05.2026).

46. HyNCF : A hybrid normalization strategy via feature statistics for collaborative filtering / J. Xu, J. Huang, J. Zhao [и др.] // Expert Systems with Applications. – 2024. – Vol. 238, Part A. – DOI: 10.1016/j.eswa.2023.121875.

47. Meme Similarity Service : сервис поиска похожих изображений (CLIP + ансамбль признаков). – GitHub, 2026. – URL: <https://github.com/lvlex/meme-similarity-service> (дата обращения: 15.05.2026).

48. Сергеев, А. В. Методика расчета параметров адекватности математической модели изображения / А. В. Сергеев, А. Ю. Липлянин, А. В. Хижняк // ПФМТ. – 2023. – № 3 (56). – С. 95–99.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		107

49. Apache JMeter : официальная документация пользователя. – Apache Software Foundation, 2025. – URL: <https://jmeter.apache.org/usermanual/index.html> (дата обращения: 20.05.2026).

50. Костров, А. В. Основы информационного менеджмента : учеб. пособие / А. В. Костров. – 2-е изд., перераб. и доп. – М. : Финансы и статистика, 2009. – 528 с. – ISBN 5-279-02314-0.

51. Мартынова, Т. Л. Управление IT-проектами : учебное пособие / Т. Л. Мартынова. – М. : Издательский центр Университета имени О. Е. Кутафина (МГЮА), 2022. – 75 с. – ISBN 978-5-906685-97-1.

52. Крумина, К. В. Управление проектами : учебное пособие / К. В. Крумина, С. Г. Полковникова ; Омский государственный технический университет. – Омск : Омский государственный технический университет (ОмГТУ), 2020. – 118 с. – ISBN 978-5-8149-3133-7.

					ВлГУ.09.03.04.ПРИ-122.18.3.00 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		108

ПРИЛОЖЕНИЕ А. ДИАГРАММА КОМПОНЕНТОВ

Диаграмма компонентов представлена на рисунках А.1 и А.2.

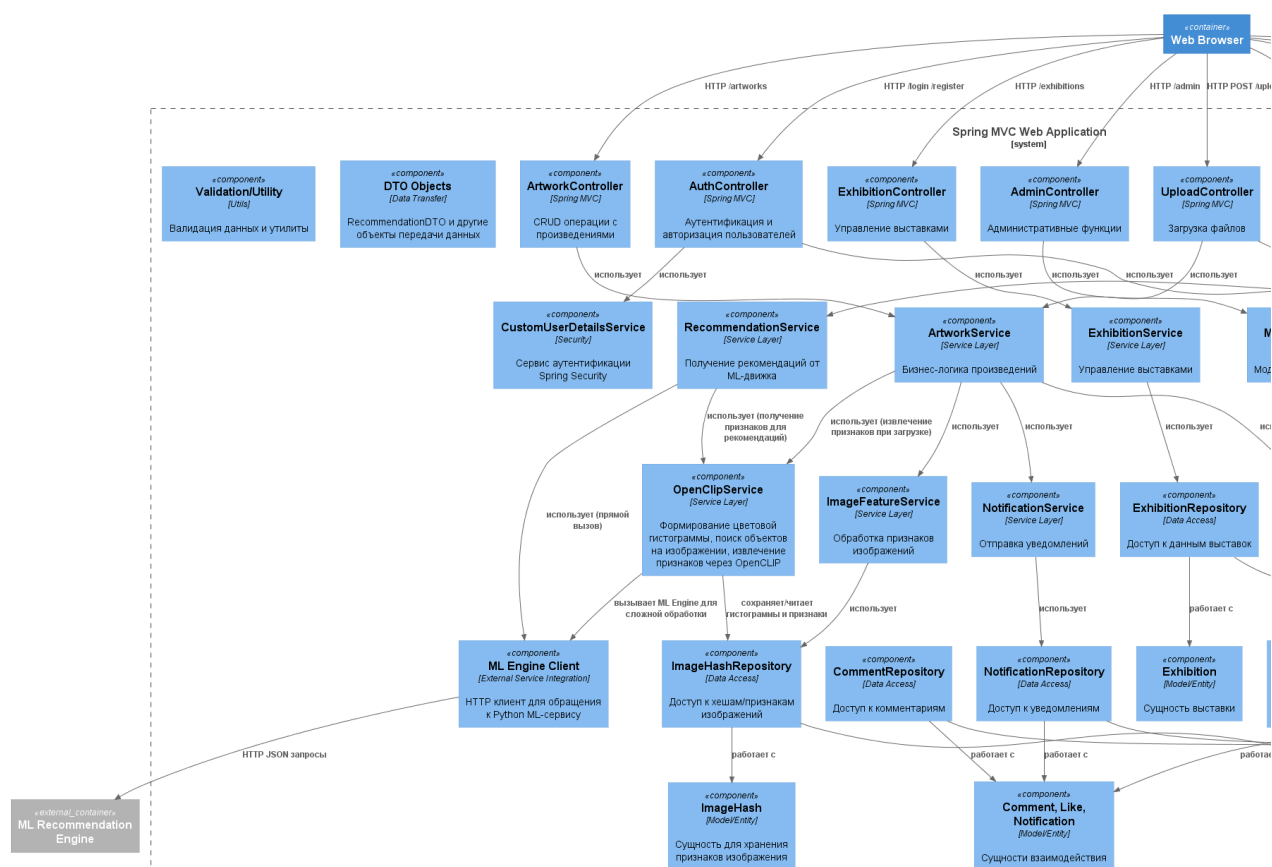


Рисунок А.1 – Диаграмма компонентов веб-приложения. Часть 1

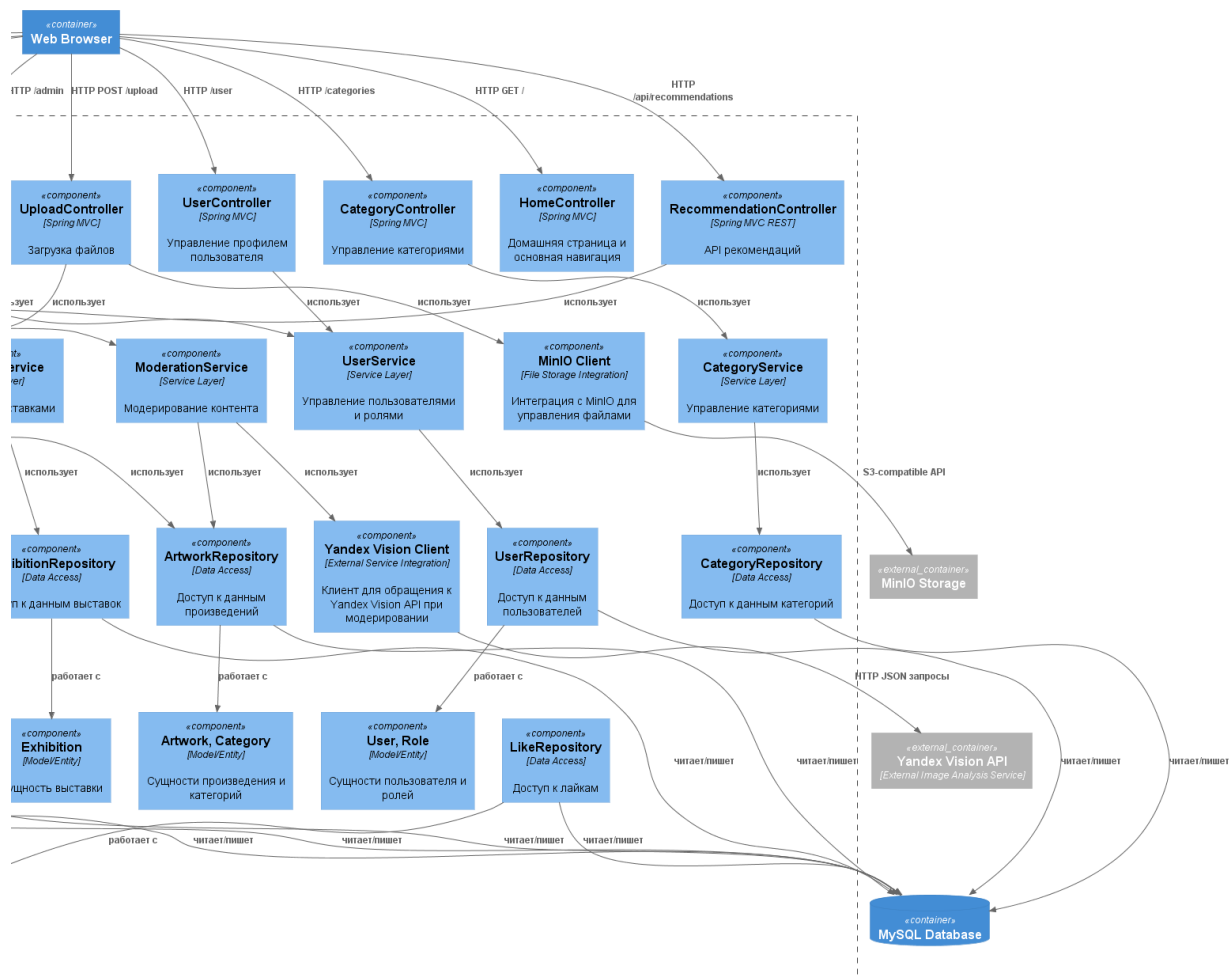


Рисунок А.2 – Диаграмма компонентов веб-приложения. Часть 2

ПРИЛОЖЕНИЕ Б. СТРАНИЦЫ ВЕБ-ПРИЛОЖЕНИЯ

Страницы веб-приложения, использующие модуль рекомендаций, продемонстрированы на рисунках Б.1-Б.5

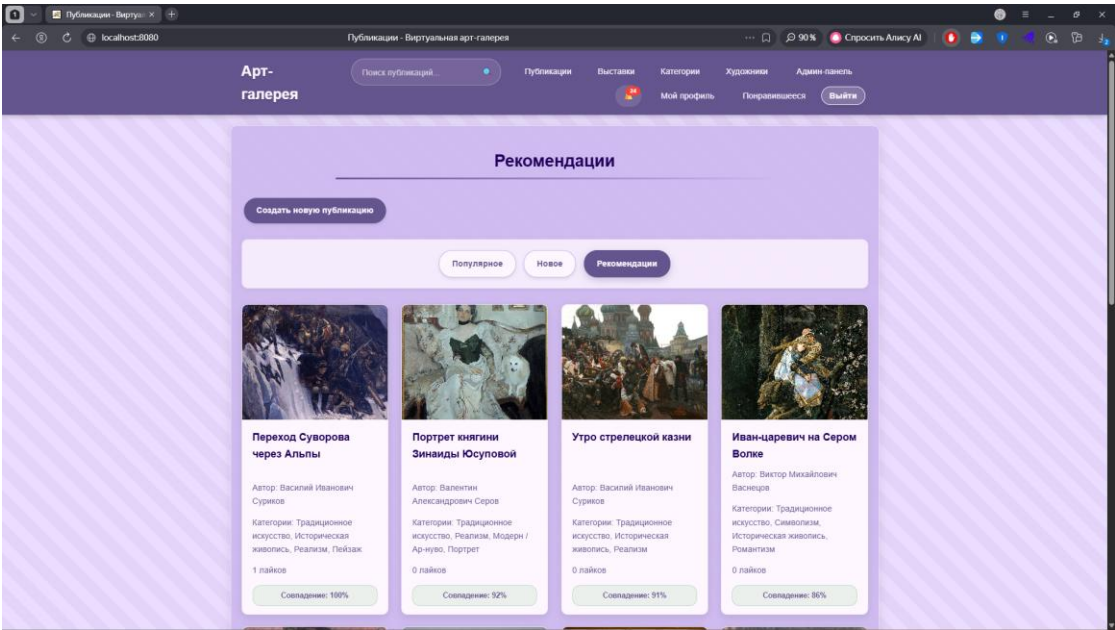


Рисунок Б.1 – Список рекомендаций, верхняя часть

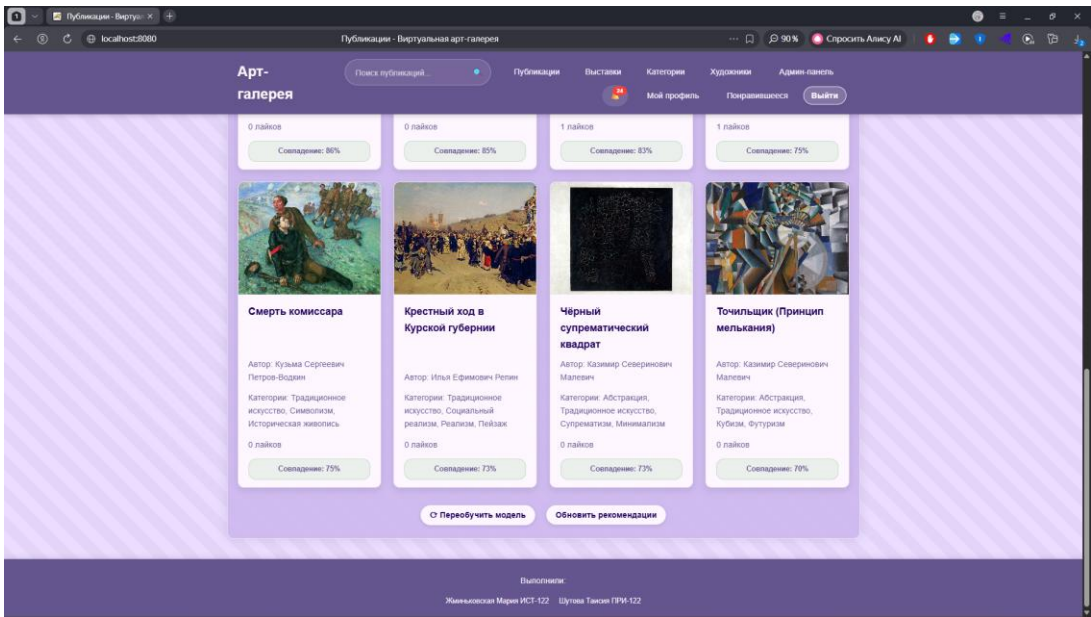


Рисунок Б.2 – Список рекомендаций, нижняя часть от лица администратора

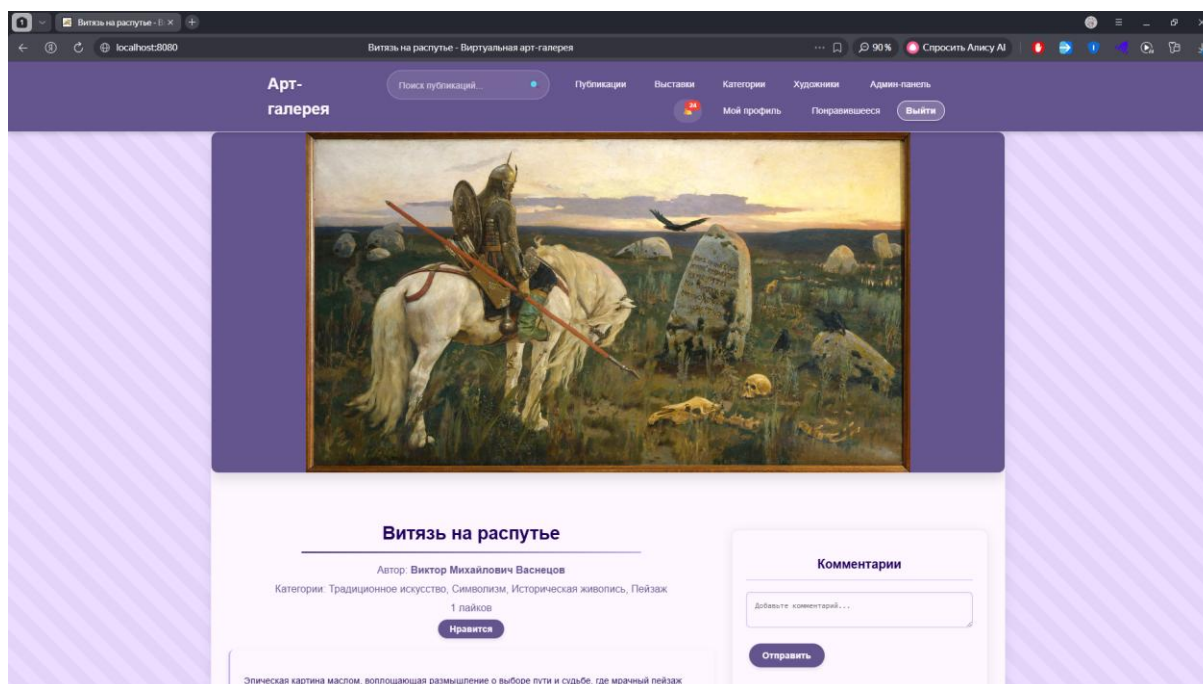


Рисунок Б.3 – Детали публикации, верхняя часть

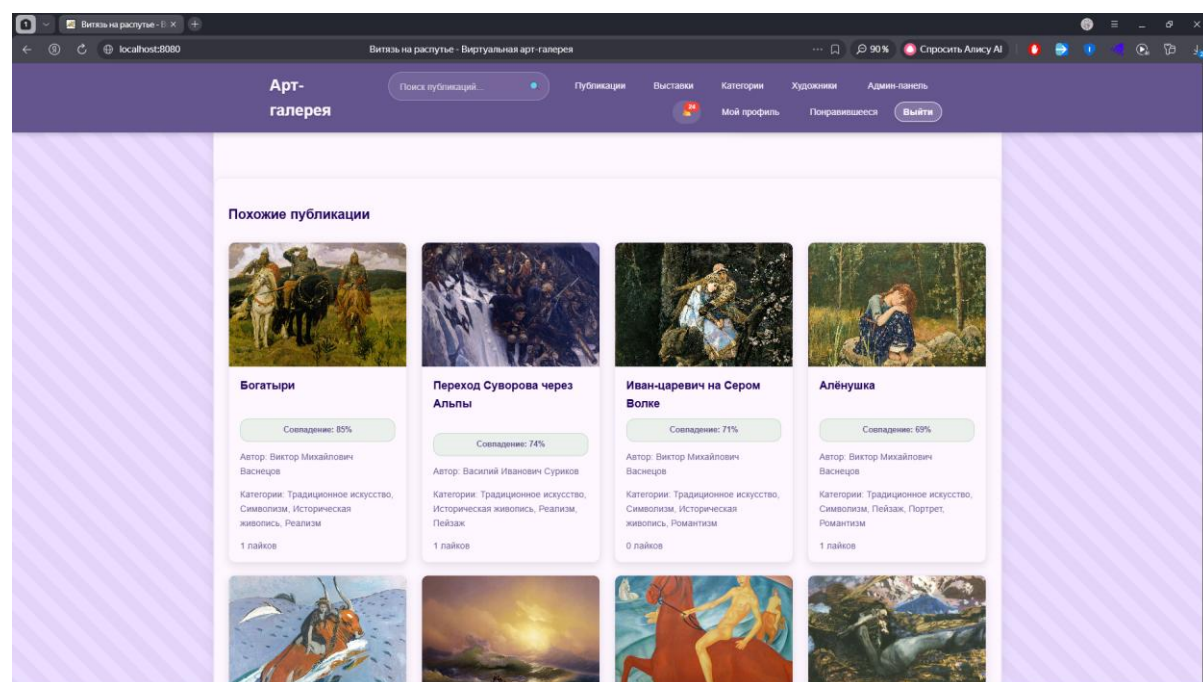


Рисунок Б.4 – Детали публикации, средняя часть

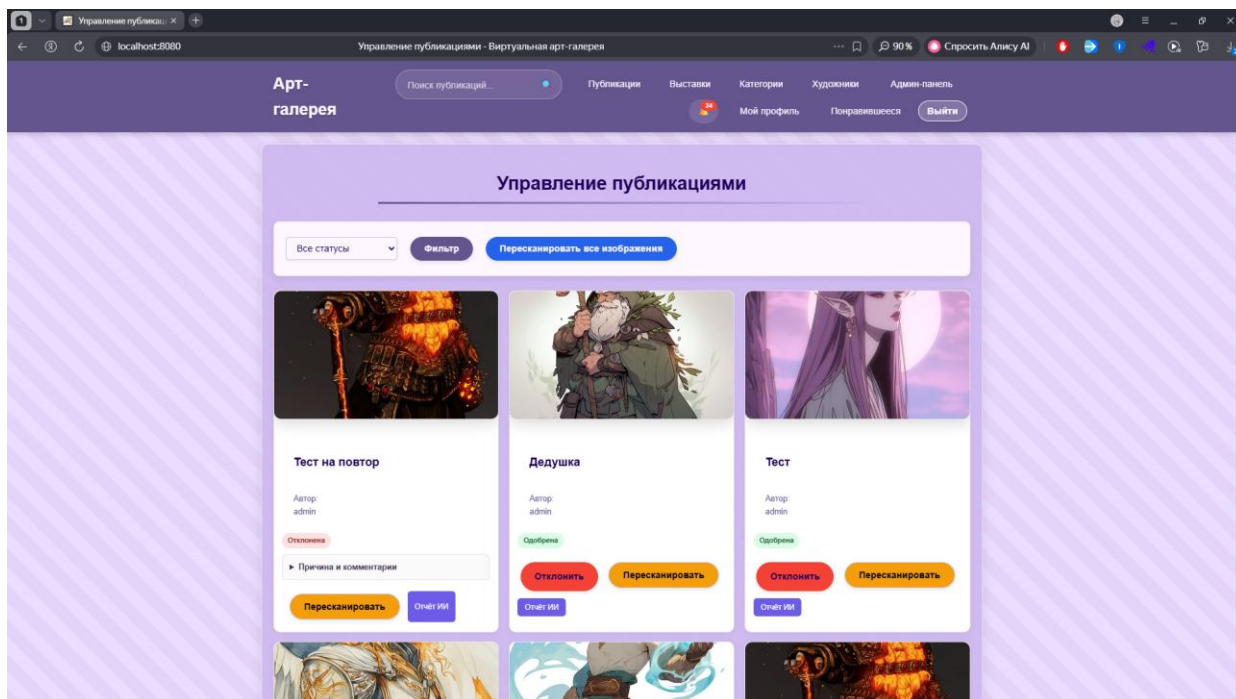


Рисунок Б.5 – Админ-панель